

Study Guide: Introduction to Algorithms (CLRS 4th Edition)

Generated 2026-06-03. Subject: Computer Science / Algorithms. Exam format: Mixed (MCQ, short answer, problem-solving). Coverage: comprehensive. Target length: ~3000 lines.

Chapter-by-Chapter Breakdown

Part I & II: Foundations, Sorting, Order Statistics

Ch 1 – The Role of Algorithms in Computing

Key Terms: - **Algorithm:** well-defined computational procedure that takes input and produces output in finite time - **Computational problem:** specifies desired input/output relationship for problem instances - **Instance:** input needed to compute a solution - **Correct algorithm:** halts and outputs correct solution for every instance - **Data structure:** way to store and organize data to facilitate access and modifications - **NP-complete:** set of problems with no known efficient algorithm; if one has efficient algorithm, all do - **Approximation algorithm:** gives good but not necessarily optimal solution - **Online algorithm:** receives input over time, must decide without knowing future data - **Keys:** numbers to be sorted; **Satellite data:** data associated with keys; **Record:** key + satellite data

Key Concepts: - Sorting problem: Input sequence $a_1, \dots, a_n$, Output permutation with $a_{p(1)} \leq a_{p(2)} \leq \dots \leq a_{p(n)}$ - Algorithm efficiency depends on input size; rate of growth (order of growth) is critical - Insertion sort $\approx c \cdot n^2$; Merge sort $\approx c \cdot n \lg n$; for large n , merge sort wins regardless of constants - Example: 10M numbers – fast computer (insertion sort) 5.56 days vs slow computer (merge sort) 0.39 hours

Exercises/Problems: 1.1-1 through 1.1-6; 1.2-1 through 1.2-3; Problem 1-1 (comparison of running times)

Ch 2 – Getting Started

Named Entities: - **Insertion sort:** efficient for small n ; incremental method - **Loop invariant:** property used to prove correctness – Initialization, Maintenance, Termination - **Pseudocode conventions:** indentation for blocks, // comments, arrays 1-origin typically, : for subarrays - **RAM model:** random-

access machine; each instruction/data access takes constant time
 - **Input size:** for sorting, number of items n ; for integers, total bits
 - **Running time:** number of instructions/data accesses executed
 - **Order of growth:** leading term, ignore constant coefficients and lower-order terms
 - **Divide-and-conquer:** break into subproblems, solve recursively, combine solutions
 - **Merge sort:** divide-and-conquer sorting algorithm
 - **Incremental method:** insert each element into its proper place
 - **Inversions:** pair (i, j) where $i < j$ and $A[i] > A[j]$
 - **Binary search:** $\Theta(\lg n)$ search on sorted array
 - **Selection sort:** find smallest, exchange with $A[1]$; $\Theta(n^2)$

Algorithm: INSERTION-SORT(A, n)

```

for i = 2 to n
    key = A[i]
    j = i - 1
    while j > 0 and A[j] > key
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key

```

- Best case: already sorted $\rightarrow \Theta(n)$
- Worst case: reverse sorted $\rightarrow \Theta(n^2)$
- Average case: $\Theta(n^2)$

Loop Invariant (Insertion Sort): At start of each for loop iteration, subarray $A[1:i-1]$ consists of elements originally in $A[1:i-1]$ but in sorted order.

Algorithm: MERGE(A, p, q, r)

```

nL = q - p + 1
nR = r - q
let L[0:nL-1] and R[0:nR-1] be new arrays
copy A[p:q] into L, A[q+1:r] into R
i = 0, j = 0, k = p
while i < nL and j < nR
    if L[i] ≤ R[j]
        A[k] = L[i]; i = i + 1
    else A[k] = R[j]; j = j + 1
    k = k + 1
copy remainder of L or R back into A

```

- MERGE runs in $\Theta(n)$ time for $n = r - p + 1$ elements

Algorithm: MERGE-SORT(A, p, r)

```

if p ≥ r: return
q = ⌊(p+r)/2⌋
MERGE-SORT(A, p, q)

```

MERGE-SORT(A, q+1, r)
 MERGE(A, p, q, r)

- Divide: $\Theta(1)$; Conquer: $2 T(n/2)$; Combine: $\Theta(n)$
- Recurrence: $T(n) = 2T(n/2) + \Theta(n)$
- Solution: $T(n) = \Theta(n \lg n)$

Recursion Tree Analysis: - $\lg n + 1$ levels; each level costs $c \cdot n$; leaf level costs $c \cdot n$ - Total: $c \cdot n \lg n + c \cdot n = \Theta(n \lg n)$

Comparisons (Insertion vs Merge Sort):

Property	Insertion Sort	Merge Sort
Worst-case time	$\Theta(n^2)$	$\Theta(n \lg n)$
Best-case time	$\Theta(n)$	$\Theta(n \lg n)$
In-place (uses extra arrays)	Yes	No
Small n	Faster (smaller constants)	Slower
Large n	Slower	Faster

Key Formulas: - Sum $1..n$: $n(n+1)/2$; Sum $1..n i^2$: $n(n+1)(2n+1)/6$
 - Summation of arithmetic series: $\sum_{i=1}^n i = n(n+1)/2$

Exercises/Problems: 2.1-1 through 2.3-8; Problems 2-1 (insertion sort on small arrays in merge sort), 2-2 (bubblesort), 2-3 (Horner's rule), 2-4 (inversions)

Ch 3 – Characterizing Running Times

Asymptotic Notation - Formal Definitions:

- **Θ -notation** (tight bound): $\Theta(g(n)) = \{f(n): \exists c_1, c_2, n_0 > 0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}$
- **O -notation** (upper bound): $O(g(n)) = \{f(n): \exists c, n_0 > 0 \text{ such that } 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0\}$
- **Ω -notation** (lower bound): $\Omega(g(n)) = \{f(n): \exists c, n_0 > 0 \text{ such that } 0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0\}$
- **o -notation** (non-tight upper bound): $o(g(n)) = \{f(n): \exists c > 0, n_0 > 0 \text{ such that } 0 \leq f(n) < c g(n) \text{ for all } n \geq n_0\}$
 - $f(n) = o(g(n))$ means $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$
- **ω -notation** (non-tight lower bound): $\omega(g(n)) = \{f(n): \exists c > 0, n_0 > 0 \text{ such that } 0 \leq c g(n) < f(n) \text{ for all } n \geq n_0\}$
 - $f(n) = \omega(g(n))$ means $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$

Theorem 3.1: $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

Properties: - Transitivity: all five notations are transitive - Reflexivity: Θ, O, Ω are reflexive - Symmetry: $f = \Theta(g)$ iff $g = \Theta(f)$
 - Transpose symmetry: $f = O(g)$ iff $g = \Omega(f)$; $f = o(g)$ iff $g = \omega(f)$

Analogy: $f = O(g) \sim a \leq b$; $f = \Omega(g) \sim a \geq b$; $f = \Theta(g) \sim a = b$; $f = o(g) \sim a < b$; $f = \omega(g) \sim a > b$

Not Trichotomy: Not all functions are asymptotically comparable (e.g., n and $n^{1+\sin n}$)

Insertion Sort Analysis with Asymptotic Notation: - Worst-case: $\Theta(n^2)$ – inner loop at most $n-1$ times, each constant - Lower bound proof: $n/3$ largest in first $n/3$ positions $\rightarrow$ each moves through $n/3$ positions $\rightarrow \Omega(n^2)$ - Best-case: $\Theta(n)$

Standard Functions: - **Monotonicity:** monotonically increasing/decreasing; strictly increasing/decreasing - **Floors/Ceilings:** $\lfloor x \rfloor$ greatest integer $\leq x$; $\lceil x \rceil$ least integer $\geq x$; $\lfloor x \rfloor + \lceil x \rceil = n$ for integer n - **Modular arithmetic:** $a \bmod n = a - n \lfloor a/n \rfloor$; $a \equiv b \pmod{n}$ if $(a \bmod n) = (b \bmod n)$ - **Polynomials:** $p(n) = \sum_{i=0}^d a_i n^i = \Theta(n^d)$ if $a_d > 0$; polynomially bounded if $f(n) = O(n^d)$ - **Exponentials:** $a^0 = 1$; $a^1 = a$; $a^{-1} = 1/a$; $(a^b)^c = a^{bc}$; $a^b a^c = a^{b+c}$ - For $a > 1$: $\lim_{n \rightarrow \infty} n^d / a^n = 0$ (exponential dominates polynomial) - $e^x = \sum_{i=0}^{\infty} x^i / i!$ for all x ; $e^x \geq 1+x$; $e^x = 1+x+\Theta(x^2)$ as $x \rightarrow 0$ - **Logarithms:** $\lg n = \log_2 n$; $\ln n = \log_e n$; $\lg_b n = (\lg n) / \lg b$ - $\log_b(a^n) = n \log_b a$; $\log_a n = \log_b n / \log_b a$; $a^{\log_b n} = n^{\log_b a}$ - $\lg n = o(n^\epsilon)$ for any $\epsilon > 0$ (polylog grows slower than any positive polynomial) - **Factorials:** $n! = 1 \cdot 2 \cdot 3 \cdots n$; $n! \leq n^n$; $n! = \omega(2^n)$; $n! = o(n^n)$; $\lg(n!) = \Theta(n \lg n)$ - **Stirling's approximation:** $n! = \sqrt{2\pi n} (n/e)^n (1 + \Theta(1/n))$ - **Iterated logarithm:** $\lg^* n = \min\{i \geq 0: \lg^{(i)} n \leq 1\}$ - $\lg^2 = 1$, $\lg^4 = 2$, $\lg^{16} = 3$, $\lg^{65536} = 4$, $\lg^{(2^{65536})} = 5$ - **Fibonacci numbers:** $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ - Golden ratio $\phi = (1+\sqrt{5})/2 = 1.618\dots$, $\bar{\phi} = (1-\sqrt{5})/2 = -0.618\dots$ - $F_n = (\phi^n - \bar{\phi}^n) / \sqrt{5} = \Theta(\phi^n / \sqrt{5})$; $F_n \geq \phi^{n/2}$

Problems: 3-1 (asymptotic behavior of polynomials), 3-2 (relative growths), 3-3 (ordering by growth rates), 3-4 (notation properties), 3-5 (manipulating asymptotic notation), 3-6 (variations on O and Ω), 3-7 (iterated functions)

Ch 4 – Divide-and-Conquer

Recurrence Basics: - **Recurrence:** equation describing function in terms of its value on smaller arguments - **Base case:** $n < n_0 \rightarrow T(n) = \Theta(1)$ (algorithmic recurrence) - **Recursive case:** for $n \geq n_0$ - **Well-defined recurrence:** at least one function satisfies it - **Algorithmic recurrence:** satisfies $T(n) = \Theta(1)$ for $n < n_0$ and recursion terminates

Conventions: omit base cases (assume algorithmic); floors/ceilings can be ignored for most analyses

Matrix Multiplication Algorithms:

- **MATRIX-MULTIPLY (naive):** triply nested for loops $\rightarrow \Theta(n^3)$
- **MATRIX-MULTIPLY-RECURSIVE:** divide into $n/2 \times n/2$ submatrices (4 submatrices each for A, B, C)
 - 8 recursive multiplications + $\Theta(1)$ index calculations

- Recurrence: $T(n) = 8T(n/2) + \theta(1) \rightarrow T(n) = \theta(n^3)$

- **Strassen's Algorithm:**

- Only 7 recursive multiplications instead of 8
- 10 matrix sums/differences ($S_1 \dots S_{10}$), then 7 products ($P_1 \dots P_7$)
- Requires 18 matrix additions total
- Recurrence: $T(n) = 7T(n/2) + \theta(n^2) \rightarrow T(n) = \theta(n^{\lg 7}) = \theta(n^{2.81})$
- Steps:
 1. Partition ($\theta(1)$)
 2. Create 10 sum/difference matrices $S_1 \dots S_{10}$ ($\theta(n^2)$)
 3. Recursively compute 7 products $P_1 \dots P_7$ ($7T(n/2)$)
 4. Combine results into $C_{11}, C_{12}, C_{21}, C_{22}$ ($\theta(n^2)$)
- $P_1 = A_{11} \cdot S_1$ where $S_1 = B_{11} - B_{21}$
- $P_2 = S_1 \cdot B_{11}$ where $S_1 = A_{11} + A_{21}$
- $P_3 = S_1 \cdot B_{21}$ where $S_1 = A_{11} + A_{21}$
- $P_4 = A_{11} \cdot S_2$ where $S_2 = B_{11} - B_{21}$
- $P_5 = S_2 \cdot S_2$ where $S_2 = A_{11} + A_{21}$, $S_2 = B_{11} + B_{21}$
- $P_6 = S_2 \cdot S_2$ where $S_2 = A_{11} - A_{21}$, $S_2 = B_{11} + B_{21}$
- $P_7 = S_2 \cdot S_2$ where $S_2 = A_{11} - A_{21}$, $S_2 = B_{11} + B_{21}$

Recurrence Solving Methods:

1. Substitution Method: - Guess form of solution using symbolic constants - Use mathematical induction to prove correct, find constants - Can subtract lower-order term to fix induction - Never use asymptotic notation in inductive hypothesis (must name constants)

2. Recursion-Tree Method: - Model recurrence as tree with node costs - Sum costs per level, then total - Good for generating guesses verified by substitution

3. Master Method (Theorem 4.1): - For recurrences: $T(n) = aT(n/b) + f(n)$, where $a > 0$, $b > 1$ - **Watershed function:** $n^{\lg_b a}$ - **Case 1:** $f(n) = O(n^{\lg_b a - \epsilon})$ for $\epsilon > 0 \rightarrow T(n) = \theta(n^{\lg_b a})$ - (Costs increase geometrically from root to leaves; leaves dominate) - **Case 2:** $f(n) = \theta(n^{\lg_b a} \lg^k n)$ for $k \geq 0 \rightarrow T(n) = \theta(n^{\lg_b a} \lg^{k+1} n)$ - (Each level costs about the same; $k=0 \rightarrow \theta(n^{\lg_b a} \lg n)$) - **Case 3:** $f(n) = \Omega(n^{\lg_b a + \epsilon})$ for $\epsilon > 0$ AND regularity condition $af(n/b) \leq cf(n)$ for $c < 1 \rightarrow T(n) = \theta(f(n))$ - (Costs decrease geometrically; root dominates)

Master Theorem Examples:

f(n)	Case	Solution	a	b	$n^{\lg_b a}$
n^2	1	$\theta(n^2)$	9	3	n^2
n	1	$\theta(n^2)$	1	3/2	$n^1 = 1$
$2(k=0)$	1	$\theta(\lg n)$	3	4	$n^{\lg 3}$
2	2	$\theta(n \lg n)$	2	2	$n^1 = n$
2	2	$\theta(n \lg^2 n)$	2	2	$n^1 = n$
2	2	$\theta(n \lg n)$	8	2	n^3
$\theta(1)$	3	$\theta(n^3)$	7	2	$n^{\lg 7}$

$\approx n^{\{2.807\}} \mid n^2 \mid 1 \mid \Theta(n^{\{\lg 7\}}) \mid \mid T(n)=2T(n/2)+n/\lg n \mid 2 \mid 2 \mid$
 $n \mid n/\lg n \mid \text{gap} \mid \Theta(n \lg \lg n) \mid$

4. Akra-Bazzi Method: - For $T(n) = \sum_{i=1}^k a_i T(n/b_i) + f(n)$
 - Find p such that $\sum_{i=1}^k a_i b_i^{-p} = 1$ - Solution: $T(n) = \Theta(n^p (1 + \int_1^n f(u)/u^{p+1} du))$ - More general than master theorem (handles unequal subproblem sizes) - Requires polynomial-growth condition on $f(n)$ to ignore floors/ceilings - $f(n)$ satisfies polynomial-growth if $f(\Theta(n)) = \Theta(f(n))$ - Examples: $n^\alpha \lg^\beta n \lg^\gamma n$ satisfy it; exponentials do not

Gap Cases (Master theorem doesn't apply): - When $f(n)$ grows between cases (e.g., $n/\lg n$ – slower than n but not polynomially slower) - When regularity condition fails

Problems: 4-1 through 4-7 (recurrence examples, parameter-passing costs, change of variables, Fibonacci numbers, chip testing, Monge arrays)

Ch 5 – Probabilistic Analysis and Randomized Algorithms

Key Terms: - **Probabilistic analysis:** using probability to analyze problems/algorithms, computing average-case running time - **Randomized algorithm:** behavior determined by input + values from random-number generator - **Expected running time:** expectation over algorithm's random choices (not input distribution) - **Average-case running time:** expected value over distribution of possible inputs - **Indicator random variable:** $I\{A\} = 1$ if A occurs, 0 otherwise - **Uniform random permutation:** each of $n!$ permutations equally likely

Lemma 5.1: $E[X_A] = \Pr\{A\}$ for indicator random variable $X_A = I\{A\}$

Linearity of expectation: $E[\sum X_i] = \sum E[X_i]$ (holds even with dependence)

Hiring Problem: - Hire first candidate better than current best - **Worst case:** candidates in increasing order $\rightarrow$ hire n times $\rightarrow O(c_h n)$ - **Expected hires:** $\approx \ln n$ (harmonic series $H_n = \sum_{i=1}^n 1/i = \ln n + O(1)$) - **Probabilistic analysis:** assumes candidates arrive in random order $\rightarrow E[\text{\#hires}] = H_n \approx \ln n$ - **Analysis:** $X_i = I\{\text{candidate } i \text{ hired}\}$; $\Pr\{\text{candidate } i \text{ hired}\} = 1/i$; $E[X] = \sum 1/i = \ln n + O(1)$ - **Randomized version:** randomly permute candidates first $\rightarrow$ expected hiring cost $O(c_h \ln n)$

Random Number Generator: - $\text{RANDOM}(a,b)$: returns integer between a and b inclusive, each equally likely, independent

RANDOMLY-PERMUTE(A, n):

```
for i = 1 to n
    swap A[i] with A[RANDOM(i,n)]
```

- Produces uniform random permutation with probability $1/n!$ for any permutation
- Loop invariant: prior to i -th iteration, $A[1:i-1]$ contains any $(i-1)$ -permutation with prob $(n-i+1)!/n!$

Birthday Paradox: - With $k \geq 23$ people, probability $\geq 1/2$ that two share birthday ($n=365$) - General: $k \geq \sqrt{(2n \ln 2)}$ for probability $\geq 1/2$ - With at least $\sqrt{(2n)}$ people, expected matching pairs ≥ 1

Balls and Bins / Coupon Collector: - Expected tosses to get a ball in given bin: b (geometric with $p=1/b$) - Expected tosses to fill all b bins: $b \cdot H_b \approx b \ln b$ - Partition into stages; stage i has probability $(b-i+1)/b$ of hit

Streaks (consecutive heads in n coin flips): - Expected length of longest streak of heads: $\Theta(\lg n)$ - Upper bound $O(\lg n)$: $\Pr\{\text{streak of length} \geq 2 \lg n \text{ starts anywhere}\} < 1/n$ - Lower bound $\Omega(\lg n)$: partition into groups of $\lceil (\lg n)/2 \rceil \rightarrow$ likely at least one all-heads group

Online Hiring Problem (Secretary Problem): - Strategy: skip first k candidates, then hire first candidate better than all seen - Optimal $k = n/e \approx 0.368n$ - Probability of hiring best candidate: at least $1/e \approx 0.368$

Exercises/Problems: 5.1-1 through 5.4-8; Problems 5-1 (probabilistic counting), 5-2 (searching unsorted array)

Ch 6 – Heapsort

Heap Data Structure: - **Binary heap:** nearly complete binary tree, represented as array $A[1:n]$ - **Array representation:** root $A[1]$; $\text{PARENT}(i) = \lfloor i/2 \rfloor$; $\text{LEFT}(i) = 2i$; $\text{RIGHT}(i) = 2i+1$ - **Heap attribute:** $A.\text{heap-size} (\leq n)$ - **Height:** $\Theta(\lg n)$ for n -element heap - **Max-heap property:** $A[\text{PARENT}(i)] \geq A[i]$ (largest at root) - **Min-heap property:** $A[\text{PARENT}(i)] \leq A[i]$ (smallest at root) - Leaves: indices $\lfloor n/2 \rfloor + 1$ through n

MAX-HEAPIFY(A, i):

```
l = LEFT(i); r = RIGHT(i)
largest = i
if l ≤ heap-size and A[l] > A[largest]: largest = l
if r ≤ heap-size and A[r] > A[largest]: largest = r
if largest ≠ i:
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

- Maintains max-heap property; assumes children are max-heaps

- Running time: $O(\lg n) = O(h)$ where h = height of node
- Recurrence: $T(n) \leq T(2n/3) + O(1) \rightarrow$ case 2 of master $\rightarrow O(\lg n)$

BUILD-MAX-HEAP(A, n):

```

heap-size = n
for i = ⌊n/2⌋ down to 1:
    MAX-HEAPIFY(A, i)

```

- Bottom-up; starts from last non-leaf
- Analysis: $\sum_{h=0}^{\lceil \lg n \rceil} \frac{n}{2^{h+1}} O(h) = O(n)$ (linear time)

HEAPSORT(A, n):

```

BUILD-MAX-HEAP(A, n)
for i = n down to 2:
    exchange A[1] with A[i]
    heap-size = heap-size - 1
    MAX-HEAPIFY(A, 1)

```

- BUILD-MAX-HEAP: $O(n)$; $n-1$ calls to MAX-HEAPIFY: $O(n \lg n)$
- Total: $O(n \lg n)$; sorts in place

Priority Queues (based on max-heap):	Operation	Running Time
Description	MAXIMUM(S)	$O(1)$
Return element with largest key	EXTRACT-MAX(S)	$O(\lg n)$
Remove and return max element	INSERT(S, x, k)	$O(\lg n)$
Insert element with key k	INCREASE-KEY(S, x, k)	$O(\lg n)$
Increase element's key to k		

MAX-HEAP-EXTRACT-MAX(A): Gets max, puts last element at root, calls MAX-HEAPIFY

MAX-HEAP-INCREASE-KEY(A, x, k): Increases key, then bubbles up toward root until parent $\geq$ node

MAX-HEAP-INSERT(A, x, n): Adds leaf with key $-\infty$, then calls INCREASE-KEY

Problems: 6-1 (building heap using insertion), 6-2 (d-ary heaps), 6-3 (Young tableaux)

Ch 7 – Quicksort

Algorithm QUICKSORT(A, p, r):

```

if p < r:
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)

```

Algorithm PARTITION(A, p, r):


```

x = A[r]           // pivot
i = p - 1
for j = p to r-1:
    if A[j] ≤ x:
        i = i + 1
        exchange A[i] with A[j]
exchange A[i+1] with A[r]
return i + 1

```

- Running time: $\Theta(n)$ where $n = r - p + 1$
- Always picks $A[r]$ as pivot; maintains 4 regions:
 1. $A[p:i] \leq x$ (low side)
 2. $A[i+1:j-1] > x$ (high side)
 3. $A[j:r-1]$ unknown
 4. $A[r] = x$ (pivot)

Performance of Quicksort:

Case	Split	Recurrence	Solution
Worst	$\Theta : n-1$ every time	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$
Best	$n/2 : n/2$ every time	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \lg n)$
Constant proportional	e.g., 9:1	$T(n) = T(9n/10) + T(n/10) + \Theta(n)$	$\Theta(n \lg n)$
Average (random)	mix of good/bad	$\Theta(n \lg n)$ expected	$\Theta(n \lg n)$

Worst case occurs when input already sorted (or reverse sorted) and pivot is always last element.

Randomized Quicksort:

```

RANDOMIZED-PARTITION(A, p, r):
    i = RANDOM(p, r)
    exchange A[r] with A[i]
    return PARTITION(A, p, r)

```

- Expected running time: $\Theta(n \lg n)$ (assuming distinct elements)
- **Key Lemma 7.2:** Two elements $z_i < z_j$ are compared iff the first pivot chosen from $Z_{[ij]} = \{z_i, \dots, z_j\}$ is either z_i or z_j
- **Lemma 7.3:** $\Pr\{z_i \text{ compared with } z_j\} = 2/(j-i+1)$
- **Theorem 7.4:** Expected running time $\Theta(n \lg n)$
 - $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2/(j-i+1) = \sum_{k=1}^{n-1} \sum_{i=1}^{n-k} 2/(k+1) < 2n \cdot H_n = O(n \lg n)$
- **Space:** $\Theta(n)$ worst-case stack depth (can reduce to $\Theta(\lg n)$ with tail recursion)

Hoare Partition (Problem 7-1): Original version; uses $A[p]$ as pivot; two pointers i, j moving inward

Problems: 7-1 (Hoare partition), 7-2 (equal elements), 7-3 (alternative analysis), 7-4 (stooge sort), 7-5 (stack depth), 7-6 (median-of-3 partition), 7-7 (fuzzy sorting)

Ch 8 – Sorting in Linear Time

Comparison Sorts: Determine order only by comparing elements (insertion sort, merge sort, heapsort, quicksort)

Decision-Tree Model: - Full binary tree; internal nodes = comparisons $i:j$; leaves = permutations - Each comparison sort corresponds to a decision tree - Height of tree = worst-case #comparisons - Any correct comparison sort must have $\geq n!$ reachable leaves

Theorem 8.1: Any comparison sort requires $\Omega(n \lg n)$ comparisons in the worst case. - Proof: $n! \leq l \leq 2^h$; $h \geq \lg(n!) = \Omega(n \lg n)$ by Stirling's approximation

Corollary 8.2: Heapsort and merge sort are asymptotically optimal comparison sorts.

Counting Sort: - Assumes input integers in range $0..k$ - Stable sort (preserves relative order of equal keys) - Uses auxiliary array $C[0:k]$

```
COUNTING-SORT(A,n,k):
    let B[1:n] and C[0:k] be new arrays
    for i=0 to k: C[i] = 0
    for j=1 to n: C[A[j]] = C[A[j]] + 1
    // C[i] now = count of elements = i
    for i=1 to k: C[i] = C[i] + C[i-1]
    // C[i] now = count of elements ≤ i
    for j=n down to 1:
        B[C[A[j]]] = A[j]
        C[A[j]] = C[A[j]] - 1
    return B
```

- Time: $\Theta(k + n)$; if $k = O(n)$, then $\Theta(n)$
- Not a comparison sort; uses array indexing instead

Radix Sort: - Sorts d -digit numbers by sorting least significant digit first (stable sort required) - Uses counting sort as stable sort per digit

```
RADIX-SORT(A,n,d):
    for i=1 to d:
        use a stable sort to sort A on digit i
```

- **Lemma 8.3:** $\Theta(d(n+k))$ time where each digit has k possible values
- **Lemma 8.4:** For n b -bit numbers with r bits per digit: $\Theta((b/r)(n+2^r))$

- Optimal $r \approx \lg n$; when $b=O(\lg n)$, radix sort runs in $\Theta(n)$ time

Bucket Sort: - Assumes input uniform over $[0,1)$ - Divides $[0,1)$ into n equal buckets; puts elements into buckets; sorts each with insertion sort; concatenates

BUCKET-SORT(A, n):

```

let B[0:n-1] be new array
for i=0 to n-1: B[i] = empty list
for i=1 to n: insert A[i] into B[ $\lfloor n \cdot A[i] \rfloor$ ]
for i=0 to n-1: sort list B[i] with insertion sort
concatenate lists B[0], ..., B[n-1]
```

- Average-case: $\Theta(n)$ ($E[n^2] = 2 - 1/n$ from binomial distribution)
- Worst-case: $\Theta(n^2)$ (all elements in one bucket)

Sorting Algorithms Comparison Table:

Algorithm	Worst-case	Average-case/Expected	In-place	Stable
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$	Yes	Yes
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	No	Yes
Heapsort	$O(n \lg n)$	$\Theta(n \lg n)$ (expected)	Yes	No
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$	No	Yes
Counting sort	$\Theta(k+n)$	$\Theta(k+n)$	No	Yes
Radix sort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	No	Yes
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (avg-case)	No	Yes

0-1 Sorting Lemma: If an oblivious compare-exchange algorithm correctly sorts all 0-1 sequences, it sorts all inputs.

Problems: 8-1 (probabilistic lower bounds), 8-2 (sorting in place in linear time), 8-3 (variable-length items), 8-4 (water jugs), 8-5 (average sorting/k-sorted), 8-6 (lower bound on merging), 8-7 (0-1 sorting lemma, columnsort)

Ch 9 – Medians and Order Statistics

Definitions: - **i -th order statistic:** i -th smallest element -

Minimum: 1st order statistic - **Maximum:** n -th order statistic -

Median: lower median at $\lfloor (n+1)/2 \rfloor$; upper median at $\lceil (n+1)/2 \rceil$

Minimum (and Maximum): - Finding minimum: $n-1$ comparisons (optimal – every non-winner must lose at least once) - Finding both min and max: naive $2n-2$ comparisons - **Improved:** process in pairs $\rightarrow$ at most $3 \lceil n/2 \rceil$ comparisons - Compare pair elements: 1 comparison; smaller vs min: 1; larger vs max: 1; total 3 per 2 elements

Randomized Selection – RANDOMIZED-SELECT(A, p, r, i):

```

if p == r: return A[p]
q = RANDOMIZED-PARTITION(A, p, r)
k = q - p + 1
```

```

if i == k: return A[q]
else if i < k: return RANDOMIZED-SELECT(A,p,q-1,i)
else: return RANDOMIZED-SELECT(A,q+1,r,i-k)

```

- Like quicksort but recurses on only one side
- **Worst-case:** $\Theta(n^2)$ (always partition around largest/smallest)
- **Expected:** $\Theta(n)$
 - If pivot in middle half (between $1/4$ and $3/4$), at most $3/4$ elements remain
 - $\Pr\{\text{helpful partitioning}\} \geq 1/2$ (Lemma 9.1)
 - Expected comparisons: $< 2n \sum_{k=0}^{\infty} \log_{4/3} n^k$
 $(3/4)^k = O(n)$
 - **Theorem 9.2:** Expected running time $\Theta(n)$

Deterministic (Worst-Case Linear) Selection – SELECT(A,p,r,i):

```

while (r-p+1) mod 5 ≠ 0:
    find minimum, if i=1 return; else p++, i--
g = (r-p+1)/5
for j = p to p+g-1:
    sort 5-element group in place
x = SELECT(A, p+2g, p+3g-1, ⌊g/2⌋) // median of medians
q = PARTITION-AROUND(A,p,r,x)
k = q-p+1
if i==k: return A[q]
else if i<k: return SELECT(A,p,q-1,i)
else: return SELECT(A,q+1,r,i-k)

```

- Divides elements into groups of 5; finds median of each group; recursively finds median of medians
- **Pivot guarantees:** at least $3g/2$ elements $\leq x$ and $\geq x \rightarrow$ both subproblems size $\leq 7n/10$
- **Recurrence:** $T(n) = T(n/5) + T(7n/10) + \Theta(n)$
- **Theorem 9.3:** $T(n) = \Theta(n)$ (proved by substitution $T(n) \leq cn$)
 - $T(n) \leq c(n/5) + c(7n/10) + \Theta(n) = 9cn/10 + \Theta(n) = cn - cn/10 + \Theta(n) \leq cn$ for sufficiently large c

Key Difference: Selection doesn't require sorting all elements, so avoids $\Omega(n \lg n)$ lower bound of comparison sorts.

Problems: 9-1 (largest i numbers), 9-2 (simplified randomized select), 9-3 (weighted median / post-office location), 9-4 (small order statistics), 9-5 (alternative analysis of randomized select), 9-6 (select with groups of 3) ## Part III & IV: Data Structures, Design Techniques

Ch 10 – Elementary Data Structures

Arrays (10.1.1) - Stored as contiguous bytes; element i (starting at index s , each b bytes) at $\text{addr } a + b(i-s)$. - $O(1)$ random access. - If elements vary in size, store pointers (uniform pointer size).

Matrices (10.1.2) - Row-major: single-array index = $n(i-1)+j$ (1-origin) or $ni+j$ (0-origin). - Column-major: single-array index = $i + m(j-1)$ (1-origin) or $i + mj$ (0-origin). - Multiple-array representation: one array per row/column of pointers. - Block representation: matrix divided into blocks, each stored contiguously. - Trade-off: single-array is more efficient; multiple-array allows ragged arrays.

Stacks (10.1.3) - LIFO policy. Operations: PUSH, POP, STACK-EMPTY. Attribute: S.top. - Empty when S.top = 0. Underflow on POP of empty stack. Overflow when S.top > S.size. - All operations $O(1)$ time.

Queues (10.1.3) - FIFO policy. Operations: ENQUEUE, DEQUEUE. Attributes: Q.head, Q.tail, Q.size. - Circular array: locations wrap around. Empty when Q.head = Q.tail. Full when Q.head = Q.tail + 1 (mod Q.size). - Each operation $O(1)$ time.

Linked Lists (10.2) - Types: singly/doubly linked, sorted/unordered, circular/not. - Each element: key, next, prev. List: L.head.

Operation	Doubly linked	Singly linked
SEARCH	$\theta(n)$	$\theta(n)$
INSERT (prepend)	$O(1)$	$O(1)$
DELETE (given pointer)	$O(1)$	$\theta(n)$
Access kth	$\theta(k)$	$\theta(k)$

- **Sentinel** (L.nil): dummy node converting list to circular doubly linked. Eliminates boundary checks. Code is simpler but extra memory per list.

Rooted Trees (10.3) - Binary tree: node with p, left, right. Root's parent = NIL. - Left-child, right-sibling representation for unbounded branching: each node has left-child (leftmost child) and right-sibling (next sibling). - $O(n)$ space for n-node tree.

Ch 11 – Hash Tables

Direct-Address Tables (11.1) - Universe $U = \{0, \dots, m-1\}$. Array $T[0:m-1]$; slot k stores element with key k or NIL. - All dictionary operations $O(1)$ worst-case. Impractical when $|U|$ is large.

Hash Tables (11.2) - Hash function $h: U \rightarrow \{0, \dots, m-1\}$. Element with key k stored in slot $h(k)$. - **Collision**: two keys hash to same slot. Resolved by chaining or open addressing. - **Load factor** $\alpha = n/m$ (avg elements per slot).

Chaining (11.2) - Each slot points to linked list of all elements hashing to it. - Operations: CHAINED-HASH-INSERT ($O(1)$), CHAINED-

HASH-SEARCH, CHAINED-HASH-DELETE. - Under **independent uniform hashing**: - Unsuccessful search: $\Theta(1+\alpha)$ avg - Successful search: $\Theta(1+\alpha)$ avg - When $n = O(m)$, $\alpha = O(1) \rightarrow$ all ops $O(1)$ avg.

Hash Functions (11.3) - **Division method**: $h(k) = k \bmod m$. Fast; avoid m near power of 2; prefer prime m . - **Multiplication method**: $h(k) = \lfloor m(kA \bmod 1) \rfloor$, $0 < A < 1$. m not critical. - **Multiply-shift** ($m = 2^w$): $h_a(k) = (ka \bmod 2^w) \gg (w-l)$. Three machine instructions. - **Universal hashing**: family H is universal if for distinct k_1, k_2 , $\Pr[h(k_1) = h(k_2)] \leq 1/m$. - Number-theoretic family: $h_{\{ab\}}(k) = ((ak+b) \bmod p) \bmod m$, $a \in \{1..p-1\}$, $b \in \{0..p-1\}$, p prime $> m$. - Multiply-shift with odd a is $2^w/m$ -universal. - **Properties**: uniform, universal, ϵ -universal, d -independent.

Open Addressing (11.4) - No storage outside table. $\alpha \leq 1$. Probe sequence: $h: U \times \{0..m-1\} \rightarrow \{0..m-1\}$. - Operations: HASH-INSERT, HASH-SEARCH. Deletion uses DELETED marker. - **Linear probing**: $h(k,i) = (h_0(k) + i) \bmod m$. Suffers **primary clustering**. Only m distinct probe sequences. With 5-independent h_0 and $\alpha \leq 2/3$, expected constant time. - **Double hashing**: $h(k,i) = (h_0(k) + i \cdot h_1(k)) \bmod m$. $h_0(k)$ must be relatively prime to m . $\Theta(m^2)$ probe sequences. - Under **independent uniform permutation hashing**: - Unsuccessful search: $\leq 1/(1-\alpha)$ expected probes - Successful search: $\leq (1/\alpha) \ln(1/(1-\alpha))$ expected probes - Insert: $\leq 1/(1-\alpha)$ expected probes

Linear Probing Deletion (11.5.1) - LINEAR-PROBING-DELETE: vacates slot, then shifts later entries backward if their probe order requires it.

Wee Hash Function (11.5.2) - $f_a(k) = \text{swap}((2k^2 + ak) \bmod 2^w)$, where swap exchanges word halves. - $h_{\{a,b,t,r\}}(k) = f_a^{\{r\}}(\{a+2^t\}(k+b) \bmod m)$. - Designed for hierarchical memory; implemented entirely in registers.

Ch 12 – Binary Search Trees

BST Property (12.1) - For any node x : keys in left subtree $\leq x.\text{key} \leq$ keys in right subtree. - Tree walks: inorder (left, root, right); $\Theta(n)$ sorted output), preorder, postorder.

Queries (12.2) - All run in $O(h)$ time on tree of height h : - TREE-SEARCH (x, k): recursive or iterative. Compare k with $x.\text{key}$, go left/right. - TREE-MINIMUM: follow left pointers until NIL. - TREE-MAXIMUM: follow right pointers until NIL. - TREE-SUCCESSOR: if right subtree exists, return minimum of right; else go up until finding a node that is a left child of its parent. - TREE-PREDECESSOR: symmetric.

Insertion (12.3) - TREE-INSERT(T, z): walk down from root with trailing pointer y . Insert z as child of y . $O(h)$.

Deletion (12.3) - Four cases (via TRANSPLANT): 1. z has no left child → replace z by its right child. 2. z has left child but no right child → replace z by left child. 3. z has two children, successor y is z's right child → replace z by y. 4. z has two children, y (successor) is not z's right child → first replace y by y's right child, then replace z by y. - $O(h)$ time.

Theorem 12.2: SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR in $O(h)$. **Theorem 12.3:** INSERT and DELETE in $O(h)$.

Ch 13 – Red-Black Trees

Red-Black Properties (13.1) 1. Every node is red or black. 2. Root is black. 3. Every leaf (NIL) is black. 4. Red node's children are both black. 5. For each node, all paths to descendant leaves have same # of black nodes.

Black-height $bh(x)$: # of black nodes from x (exclusive) down to leaf.

Lemma 13.1: Red-black tree with n internal nodes has height $\leq 2 \lg(n+1)$.

Rotations (13.2) - LEFT-ROTATE(T,x): x's right child y becomes new root of subtree; x becomes y's left child; y's left child becomes x's right child. $O(1)$, preserves BST property. - RIGHT-ROTATE(T,y): symmetric. $O(1)$.

Insertion (13.3) - RB-INSERT: insert as in BST, color z RED, then RB-INSERT-FIXUP. - Fixup loop maintains: (a) z is red; (b) if z.p is root, it's black; (c) at most one violation (property 2 or 4). - **Case 1** (uncle y red): recolor parent, uncle, grandparent; move z up two levels. - **Case 2** (uncle black, z is right child): left rotation → case 3. - **Case 3** (uncle black, z is left child): recolor parent black, grandparent red, right rotation. - $O(\lg n)$ time; ≤ 2 rotations.

Deletion (13.4) - RB-DELETE: similar to TREE-DELETE; tracks y (node removed/moved) and x (y's replacement). If y-original-color is BLACK, call RB-DELETE-FIXUP. - Fixup handles "doubly black" x. Loop moves extra black up. - **Case 1** (sibling w red): recolor + left rotation → cases 2/3/4. - **Case 2** (w black, both w's children black): push black up. - **Case 3** (w black, w.left red, w.right black): right rotation → case 4. - **Case 4** (w black, w.right red): recolor + left rotation; loop terminates. - $O(\lg n)$ time; ≤ 3 rotations.

Ch 14 – Dynamic Programming

DP Framework (4 steps) 1. Characterize structure of optimal solution. 2. Recursively define value of optimal solution. 3.

Compute value (bottom-up typically). 4. Construct optimal solution from computed info.

Key Properties - Optimal substructure: optimal solution contains optimal solutions to subproblems. - **Overlapping subproblems:** same subproblems recur; solve each once. - **Independence:** subproblems don't share resources (unlike longest simple path).

Rod Cutting (14.1) - Revenue: $r_n = \max(p_n, \max_{1 \leq i \leq n-1}(r_i + r_{n-i}))$ - Simpler: $r_n = \max_{1 \leq i \leq n}(p_i + r_{n-i})$, $r_0 = 0$. - Naive recursion: $T(n) = 2^n$. - Top-down memoized: $\Theta(n^2)$. - Bottom-up: BOTTOM-UP-CUT-ROD, $\Theta(n^2)$. - Extended version stores $s[j]$ = optimal first-cut size for reconstruction.

Matrix-Chain Multiplication (14.2) - Input dimensions: $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$. - Cost of multiplying $A_{\{p \times q\}} \times B_{\{q \times r\}}$: pqr scalar multiplications. - Recurrence: $m[i, j] = \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$, $m[i, i] = 0$. - Number of parenthesizations: Catalan numbers $\sim \Omega(4^n / n^{3/2})$. - MATRIX-CHAIN-ORDER: $O(n^3)$ time, $\Theta(n^2)$ space. - $s[i, j]$ records split point k for reconstruction.

Elements of DP (14.3) - Subproblem graph: vertices = subproblems; edges = dependencies; running time $\approx \sum \text{degree}$. - Cut-and-paste proof technique for optimal substructure. - tD/eD classification: table $O(n^t)$, each entry depends on $O(n^e)$ others.

Longest Common Subsequence (14.4) - $X = \langle x_1 \dots x_m \rangle$, $Y = \langle y_1 \dots y_n \rangle$. Z is LCS. - Theorem 14.1 (optimal substructure of LCS): 1. If $x_m = y_n$, then $z_k = x_m = y_n$ and $Z_{\{k-1\}}$ is LCS of $X_{\{m-1\}}$ and $Y_{\{n-1\}}$. 2. If $x_m \neq y_n$ and $z_k \neq x_m$, Z is LCS of $X_{\{m-1\}}$ and Y . 3. If $x_m \neq y_n$ and $z_k \neq y_n$, Z is LCS of X and $Y_{\{n-1\}}$. - Recurrence: $c[i, j] = 0$ if $i=0$ or $j=0$; $= c[i-1, j-1] + 1$ if $x_i = y_j$; $= \max(c[i-1, j], c[i, j-1])$ otherwise. - LCS-LENGTH: $\Theta(mn)$ time, $\Theta(mn)$ space. - Reconstruction: follow $b[i, j]$ arrows ($\rightarrow$ = match, $\uparrow$ = skip X , $\leftarrow$ = skip Y). - Space optimization: only 2 rows needed for length; $\min(m, n)$ entries + $O(1)$.

Optimal BST (14.5) - Keys $k_1 \dots k_n$ (ordered), dummy keys $d_1 \dots d_n$. Probabilities p_i (successful), q_i (unsuccessful). - Expected search cost: $E[\text{search cost}] = \sum (\text{depth}_T(k_i) + 1) \cdot p_i + \sum (\text{depth}_T(d_i) + 1) \cdot q_i$. - Recurrence: $e[i, j] = q_{i-1}$ if $j = i-1$; $= \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\}$ where $w(i, j) = w(i, j-1) + p_j + q_j$. - OPTIMAL-BST: $O(n^3)$ time, $\Theta(n^2)$ space. - Knuth optimization: $\text{root}[i, j-1] \leq \text{root}[i, j] \leq \text{root}[i+1, j] \rightarrow O(n^2)$.

Ch 15 – Greedy Algorithms

Greedy-Choice Property: globally optimal solution can be made by locally optimal choices. **Optimal Substructure:** as in DP.

Activity Selection (15.1) - Set of n activities with start s_i , finish f_i (sorted by f_i). - Goal: max-size set of mutually compatible activities. - Greedy choice: pick activity with earliest finish time. - RECURSIVE-ACTIVITY-SELECTOR: $O(n)$ after sorting. - GREEDY-ACTIVITY-SELECTOR: $O(n)$ iterative. - Proof of optimality: Theorem 15.1 – replacing earliest-finish activity in optimal solution with greedy choice yields another optimal solution.

Elements of Greedy Strategy (15.2) - Steps: 1. Cast as choice + one subproblem. 2. Prove greedy choice is safe. 3. Show optimal substructure. - **0-1 Knapsack**: Greedy (value/weight) fails. Solved by DP: $O(nW)$. - **Fractional Knapsack**: Greedy (value/weight) works. $O(n \lg n)$, or $O(n)$ with selection algorithm.

Huffman Codes (15.3) - **Prefix-free code**: no codeword is prefix of another; represented by full binary tree. - Cost of tree: $B(T) = \sum_{c \in C} c.\text{freq} \cdot d_T(c)$. - HUFFMAN(C): repeatedly merge two lowest-frequency nodes. $O(n \lg n)$ with binary min-heap. - Lemma 15.2 (greedy-choice): Two lowest-frequency characters have codewords of same length differing only in last bit. - Lemma 15.3 (optimal substructure): Merging two lowest-frequency characters and solving reduces to optimal substructure. - Theorem 15.4: HUFFMAN produces optimal prefix-free code.

Offline Caching (15.4) - Cache of k blocks. Sequence of n requests. Goal: minimize cache misses. - **Furthest-in-future**: evict block whose next access is furthest in future. Optimal (Theorem 15.5). - Subproblem (C, i) : process requests $b_i..b_n$ with cache C . - LRU (least-recently-used) is common online heuristic but not optimal.

Ch 16 – Amortized Analysis

Definition: average time per operation over worst-case sequence. No probability.

Aggregate Analysis (16.1) - Show total cost $T(n)$ for n operations; amortized = $T(n)/n$. - **Stack with MULTIPOP**: each object popped at most once per push. Total $O(n)$ for n ops $\rightarrow O(1)$ amortized. - **Binary counter INCREMENT**: bit $A[i]$ flips $\leq n/2^i$ times. Total flips $< 2n \rightarrow O(1)$ amortized.

Accounting Method (16.2) - Assign amortized cost $\hat{c}_i$; maintain credit = $\sum \hat{c}_i - \sum c_i \geq 0$ always. - **Stack**: charge PUSH \$2 (pay \$1 actual + \$1 credit on item), POP \$0, MULTIPOP \$0. Credit on each plate pays for its pop. - **Binary counter**: charge \$2 per 0 $\rightarrow$ 1 bit flip (\$1 actual + \$1 credit on bit). Resetting 1 $\rightarrow$ 0 is free (prepaid).

Potential Method (16.3) - Potential function $\Phi(D_i)$. Amortized: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. - Total amortized = total actual + $\Phi(D_n) - \Phi(D_0)$. Need $\Phi(D_i) \geq \Phi(D_0)$ for all i . - **Stack**: $\Phi =$

#items in stack. PUSH: $\hat{c}=2$; POP/MULTIPOP: $\hat{c}=0$. - **Binary counter:** $\Phi = \#1\text{-bits}$. INCREMENT resets t bits, sets at most 1: $\hat{c} \leq (t+1)+(1-t) = 2$. - Counter starting at b_0 : total actual $\leq 2n + b_0 - b_n$.

Dynamic Tables (16.4) - Table expansion: double size when full. Load factor $\alpha \geq 1/2$. - Aggregate: total cost of n insertions $\leq 3n \rightarrow$ amortized ≤ 3 . - Accounting: charge \$3 per insertion (\$1 for itself, \$1 for its own future move, \$1 for another item's future move). - Potential: $\Phi = 2(\text{num} - \text{size}/2)$. Insertion without expansion: $\Delta\Phi=2 \rightarrow \hat{c}=3$. Insertion with expansion: $\hat{c}=3$. - **Table expansion and contraction:** double on full; halve when load $< 1/4$. Potential: - If $\alpha \geq 1/2$: $\Phi = 2(\text{num} - \text{size}/2)$ - If $\alpha < 1/2$: $\Phi = \text{size}/2 - \text{num}$ - Amortized cost of each operation is $O(1)$. Table load factor always $\geq 1/4$. ## Part V & VI: Advanced Data Structures, Graphs

Overview: Chapters 17-19 cover advanced data structures (augmented RB trees, B-trees, disjoint sets); Chapters 20-25 cover graph algorithms (traversal, MST, shortest paths, flow, matching).

Ch 17 – Augmenting Data Structures

- **Goal:** Store extra info in textbook data structures (red-black trees) to support new ops without breaking asymptotic bounds
- **Four-step method** (Section 17.2):
 1. Choose underlying data structure
 2. Determine additional info to maintain
 3. Verify basic modifying ops can maintain it efficiently
 4. Develop new operations
- Applied to order-statistic trees and interval trees

17.1 – Dynamic order statistics - Order-statistic tree: RB tree with $x.\text{size} = \# \text{internal nodes in subtree rooted at } x$ - $x.\text{size} = x.\text{left.size} + x.\text{right.size} + 1$, sentinel size = 0 - Keys may repeat; rank defined by inorder walk position - **OS-SELECT(x, i):** find i -th smallest key - $r = x.\text{left.size} + 1$ (rank of x in its subtree) - If $i == r \rightarrow$ return x ; if $i < r \rightarrow$ recurse left; if $i > r \rightarrow$ recurse right with $i-r$ - Example (Figure 17.1): OS-SELECT(root, 17): root(26) has rank 13 $\rightarrow$ recurse right with $i=4$; node(41) has rank 6 $\rightarrow$ recurse left with $i=4$; node(30) has rank 2 $\rightarrow$ recurse right with $i=2$; node(38) has rank 2 $\rightarrow$ return 38 - $O(\lg n)$ - **OS-RANK(T, x):** return rank of x in inorder walk - $r = x.\text{left.size} + 1$; $y = x$ - While $y \neq T.\text{root}$: if y is right child $\rightarrow r += y.p.\text{left.size} + 1$; $y = y.p$ - Example (node 38, Fig 17.1): start $r=2$, $y=38$; $y=30$ (right child): $r+=1+1=4$; $y=41$ (left child): $r=4$; $y=26$ (right child): $r+=12+1=17 \rightarrow$ return 17 - Loop invariant: $r = \text{rank of } x.\text{key} \text{ in subtree rooted at } y$ - $O(\lg n)$ (walks up tree to root) - **Maintaining size during insert/delete:** - Phase 1 (path from root): increment/decrement size on each node visited ($O(\lg n)$) - Phase 2 (rotations): at most 2 rotations for insert, 3 for delete; update sizes locally: - LEFT-ROTATE: $y.\text{size} = x.\text{size}$;

$x.size = x.left.size + x.right.size + 1$ - RIGHT-ROTATE: symmetric
 - Overall $O(\lg n)$ per insert/delete - **Key exercises:** - 17.1-3:
 nonrecursive OS-SELECT using while loop - 17.1-4: OS-KEY-
 RANK(T, k) finds rank of key k in $O(\lg n)$ - 17.1-5: i -th successor
 of x in $O(\lg n)$ - 17.1-7: count inversions in array in $O(n \lg n)$
 using order-statistic tree - 17.1-8: count intersecting chords in
 circle using sweep + order-statistic tree ($O(n \lg n)$)

17.2 – How to augment a data structure - General four-step
 process: 1. Choose underlying data structure 2. Determine
 additional information to maintain 3. Verify basic modifying ops
 (insert/delete) can maintain it efficiently 4. Develop new
 operations - **Theorem 17.1 (Augmenting RB trees):** If attribute f
 at node x depends only on x , $x.left$, $x.right$ in $O(1)$ time, then
 insert/delete can maintain f in $O(\lg n)$ - Propagation only up
 ancestors (path to root) - Insert phase 1: compute $x.f$ for new
 node ($O(1)$), propagate up ($O(\lg n)$) - Insert phase 2 (rotations):
 each rotation may need $O(\lg n)$ propagation, but at most 2
 rotations $\rightarrow O(\lg n)$ - Delete: similar, at most 3 rotations $\rightarrow O(\lg n)$
 - Key insight: each node should store info that can be
 recomputed from children in $O(1)$ - Counterexample: storing rank
 directly at each node would need $\Theta(n)$ updates on insert of new
 min - Exercises: - 17.2-1: add pointers to support MIN/MAX/SUCC/
 PRED in $O(1)$ on order-statistic trees - 17.2-2: black-height can
 be maintained easily (local property); depth cannot (depends on
 global path length, but can be maintained with Theorem 17.1 by
 storing parent depth info) - 17.2-3: associative binary operator
 $\square$ over attribute a can be maintained: $x.f = x.left.f \square x.a \square$
 $x.right.f$, $O(1)$ recompute

17.3 – Interval trees - **Interval trichotomy:** any two intervals
 $[t_1, t_2]$, $[t_3, t_4]$ satisfy exactly one of: - They overlap ($l_1 \leq h_2$
 $\square l_2 \leq h_1$) - i is left of i' ($h_1 < l_2$) - i is right of i' ($h_2 <$
 l_1) - **Interval tree:** RB tree keyed on $x.int.low$ (low endpoint) -
 Additional attribute: $x.max = \max\{x.int.high, x.left.max,$
 $x.right.max\}$ - **INTERVAL-SEARCH(T, i):** find any interval
 overlapping i 1. $x = T.root$ 2. While $x \neq nil$ and i does not
 overlap $x.int$: - If $x.left \neq nil$ and $x.left.max \geq i.low \rightarrow x =$
 $x.left$ - Else $\rightarrow x = x.right$ 3. Return x - $O(\lg n)$, correctness
 via Theorem 17.2 - **Theorem 17.2 (INTERVAL-SEARCH correctness):** -
 If search goes left ($x.left.max \geq i.low$): either left subtree has
 overlapping interval, or right subtree has none - If search goes
 right ($x.left.max < i.low$ or $x.left = nil$): left subtree has no
 overlapping interval (all $i'.high < i.low$) - Uses interval
 trichotomy for proof - **INTERVAL-INSERT / INTERVAL-DELETE:** $O(\lg n)$
 via Theorem 17.1 - Exercises: - 17.3-1: LEFT-ROTATE for interval
 tree updates max in $O(1)$: $y.max = x.max$; $x.max = \max(x.int.high,$
 $x.left.max, x.right.max)$ - 17.3-2: find overlapping interval with
 min low endpoint by modifying search - 17.3-3: list all
 overlapping intervals in $O(\min\{n, k \lg n\})$ by successive
 INTERVAL-SEARCH + delete - 17.3-5: MIN-GAP data structure:
 augment RB tree with min gap in subtree; maintain during insert/
 delete

Ch 18 – B-Trees

- **B-tree**: balanced search tree optimized for disk; high branching factor minimizes disk accesses
- Node = disk block; typical branching factor 50–2000
- **Minimum degree $t \geq 2$** :
 - Root: $1 \leq \text{keys} \leq 2t-1$
 - Internal node (non-root): $t-1 \leq \text{keys} \leq 2t-1$
 - Children: internal node with n keys has $n+1$ children
 - Leaves: all at same depth (height h)
- **Height theorem (18.1)**: For $n \geq 1$, $h \leq \log_t((n+1)/2)$
- **B+ tree variant**: satellite info only in leaves; internal nodes hold keys + child pointers only

18.1 – Definition - Node x : $x.n$ keys sorted nondecreasing, $x.\text{leaf}$ boolean - Internal node: $x.c_1 \dots x.c_{n+1}$ child pointers - Keys separate subtree ranges: $k_1 \leq x.\text{key}_1 \leq k_2 \leq x.\text{key}_2 \leq \dots \leq k_{n+1}$
- **2-3-4 tree**: $t=2$ (each internal node has 2, 3, or 4 children)

18.2 – Basic operations - **B-TREE-SEARCH(x, k)**: linear scan within node to find correct child; recurse - Disk accesses: $O(h) = O(\log_t n)$; CPU: $O(th) = O(t \log_t n)$ - Can use binary search within node for $O(\lg n)$ CPU (ex 18.2-6) - Returns (y, i) if found, NIL otherwise - **B-TREE-CREATE(T)**: allocate root node, mark leaf, $n=0$; $O(1)$ disk ops - **Split (B-TREE-SPLIT-CHILD(x, i))**: split full child $y = x.c_i$ ($2t-1$ keys) around median $y.\text{key}_t$ - New node z gets largest $t-1$ keys and corresponding children - y keeps smallest $t-1$ keys - $y.\text{key}_t$ moves up to x at position i - x gains one key and one child - CPU: $O(t)$; disk: $O(1)$ writes for y, z, x
- Figure 18.5: splitting node with $t=4$ (keys A B C D E F G, median D moves up) - **B-TREE-INSERT(T, k)**: single pass top-down - If root full $\rightarrow$ B-TREE-SPLIT-ROOT: create new empty root, split old root as its child, height++ - Call B-TREE-INSERT-NONFULL on nonfull root - **B-TREE-INSERT-NONFULL(x, k)**: - If x is leaf: shift keys right, insert k - Else: find child to descend to; if that child is full, split it first; recurse - $O(h)$ disk, $O(th)$ CPU - **Example (Figure 18.7)**: $t=3$, inserting F S Q K C L H T V W M R N P A B X Y D Z E - Root splits when full (after 5 keys) - Internal node splits propagate upward as needed - Height increases only when root splits

18.3 – Deletion (B-TREE-DELETE) - Single pass down (search + delete combined); prevents underflow before recursing into child - **Key property**: ensures node has $\geq t$ keys at time of recursion (not just $t-1$); this extra key allows borrowing if needed - **Case 1**: k in leaf $x \rightarrow$ remove k from x directly (shift remaining keys left) - **Case 2**: k in internal node $x \rightarrow$ three subcases: - 2a: child y (predecessor's subtree) has $\geq t$ keys: find predecessor k' (maximum key in y), replace k with k' , recursively delete k' from y - 2b: child z (successor's subtree) has $\geq t$ keys: symmetric; find successor k' in z , replace k with k' , delete k' from z - 2c: both y and z have $t-1$ keys: merge y, k, z into single node ($2t-2$ keys), free z , recursively delete k from merged

node - **Case 3:** k not in internal node $x \rightarrow$ ensure child c_i (where k belongs) has $\geq t$ keys: - 3a: if left sibling $c_{\{i-1\}}$ has $\geq t$ keys: rotate: sibling's last key up to parent, parent's key down to c_i , sibling's last child becomes c_i 's first child - 3b: if right sibling $c_{\{i+1\}}$ has $\geq t$ keys: symmetric rotation from right - 3c: else (both siblings have $t-1$ keys): merge c_i with one sibling, bring parent separator key down (now $2t-2$ keys); recurse into merged node - **Root handling:** root can have as few as 1 key; when root becomes empty (after merge), remove root, its only child becomes new root (tree height decreases by 1) - **Complexity:** $O(h)$ disk accesses (single pass down); $O(th)$ CPU = $O(t \log_t n)$ - **Example (Figure 18.8):** $t=3$, deleting F, M, G, D, B: - Deleting F: simple leaf deletion - Deleting M: internal node, predecessor L found in left child, replace and delete L - Deleting G: internal node, both children have $t-1 \rightarrow$ merge, then delete from merged node - Deleting D: leaf deletion would cause underflow $\rightarrow$ borrow from left sibling (rotate T through parent) - Deleting B: merge causes root to lose last key $\rightarrow$ new root; tree height decreases

Ch 19 – Data Structures for Disjoint Sets

- **Operations:** MAKE-SET(x), UNION(x, y), FIND-SET(x)
- Parameters: n = #MAKE-SET ops; m = total ops ($m \geq n$); at most $n-1$ UNIONS
- **Application:** CONNECTED-COMPONENTS(G) using MAKE-SET on each vertex + UNION per edge
 - SAME-COMPONENT(u, v): two FIND-SETS

19.2 – Linked-list representation - Each set = linked list with head (rep), tail - MAKE-SET, FIND-SET: $O(1)$ (follow pointer from object to set object, then head) - UNION: append shorter list to longer (weighted-union heuristic) - Without heuristic: worst-case UNION could be $O(n)$ each $\rightarrow O(n^2)$ for $n-1$ UNIONS - With weighted-union: each element's set pointer updated at most $O(\lg n)$ times - Total for m ops with n MAKE-SET: $O(m + n \lg n)$ - **Weighted-union heuristic** (Theorem 19.1): sequence of m ops on n elements takes $O(m + n \lg n)$ - Proof: each element's set pointer changes at most $\lceil \lg n \rceil$ times (each change doubles its set's size)

19.3 – Disjoint-set forests - Each tree = one set; root = representative - **Union by rank:** root with smaller rank points to larger rank; rank $\approx$ height upper bound - Starting rank 0; increases only when merging equal ranks $\rightarrow$ rank+1 - Ranks strictly increase along path from leaf to root - **Path compression:** FIND-SET makes every node on find path point directly to root - Two-pass: recurse up to find root (first pass), then unwind updating pointers (second pass) - Pseudocode: - MAKE-SET(x): $x.p = x$, $x.rank = 0$ - UNION(x, y): LINK(FIND-SET(x), FIND-SET(y)) - LINK(x, y): if $x.rank > y.rank \rightarrow y.p = x$; else $\rightarrow x.p = y$; if $x.rank == y.rank \rightarrow y.rank++$ - FIND-SET(x): if $x \neq x.p \rightarrow x.p =$ FIND-SET($x.p$); return $x.p$ - Running times: - Union by rank alone: $O(m \lg n)$ - Path compression alone: $O(n + f \cdot (1 + \log_{2+f/n} n))$ - Both heuristics: $O(m \alpha(n))$ – nearly linear - **Properties of**

ranks (for analysis, Section 19.4): - Ranks are nonnegative integers, strictly increasing toward root - Number of nodes with rank $k \leq n / 2^k$ (since each node with rank k has subtree of size $\geq 2^k$) - If x is not root, $\text{rank}(x) \leq \text{rank}(x.p) - 1$ - **Lemma 19.4 (Rank properties)**: For any node x , $x.\text{rank} \leq x.p.\text{rank}$, and if $x \neq x.p$ (not root), then path from x to root has at least $x.\text{rank}+1$ nodes - **Lemma 19.5**: Number of nodes with rank r is at most $n/2^r$ - **Ackermann function analysis**: - $A(j) = j+1$; $A_k(j) = A_{k-1}^{j+1}(j)$ (functional iteration) - $A(j) = 2^{j+1}$; $A(j) = 2^{j+1}(j+1)-1$ - $A(1) = 2047$; $A(1) > 10^{10}$ - $\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}$; $\alpha(n) \leq 4$ for all practical n - **Potential function**: $\Phi = \sum_x f(x.\text{rank})$ where f is carefully designed to account for path compression benefit - Each FIND-SET can be charged $O(\alpha(n))$; total $O(m \alpha(n))$ - Complex accounting uses "blocks" of ranks partitioned by A_k levels - Key exercise 19.3-3: sequence achieving $\Omega(m \lg n)$ with union-by-rank alone (no path compression) - make tall trees by always merging equal-rank roots, creating $\lg n$ height

19.4 – Analysis of union by rank with path compression -

Ackermann function: $A(j)$ defined by functional iteration - $A(j) = 2^{j+1}$; $A(j) = 2^{j+1}(j+1)-1$ - $A(1) = 2047$; $A(1) > 10^{80}$ - **$\alpha(n)$** : inverse Ackermann; lowest k s.t. $A_k(1) \geq n$ - $\alpha(n) \leq 4$ for all practical n - **$O(m \alpha(n))$ bound** via potential method (complex amortized analysis) - Properties of ranks: - $x.\text{rank}$ increases only in LINK when equal ranks; monotonic - Number of nodes with rank $k \leq n / 2^k$ - Ranks increase along any path from leaf to root

Ch 20 – Elementary Graph Algorithms

- Graph $G = (V, E)$; $n = |V|$, $m = |E|$ (or just E)

20.1 – Representations - Adjacency-list: array Adj of $|V|$ lists; space $O(V+E)$; edge (u,v) in $\text{Adj}[u]$ - For weighted graph: $\text{Adj}[u]$ stores (v, w) pairs - Simple, compact for sparse graphs - Adding a vertex: $O(1)$; adding an edge: $O(1)$; checking edge existence: $O(\text{degree}(u))$ - **Adjacency-matrix**: $|V| \times |V|$ matrix A with $A[i][j]=1$ if $(i,j) \in E$; space $O(V^2)$ - $O(1)$ edge existence test - Good for dense graphs ($|E| \approx |V|^2$) - Weighted: store weight instead of 0/1 - **Comparison**: adjacency list is more common; matrix used when graph small/dense or $O(1)$ edge test needed repeatedly

20.2 – BFS (Breadth-First Search) - Input: graph G (or adjacency list), source vertex s - Discovers vertices in order of increasing distance (#edges) from s - Attributes: $u.\text{color}$ (WHITE=undiscovered, GRAY=discovered/frontier, BLACK=finished), $u.d$ (distance), $u.\pi$ (predecessor) - $\text{BFS}(G, s)$: 1. For each $u \neq s$: $u.\text{color}=\text{WHITE}$, $u.d=\infty$, $u.\pi=\text{nil}$ 2. $s.\text{color}=\text{GRAY}$, $s.d=0$, $s.\pi=\text{nil}$; $Q = \emptyset$; $\text{ENQUEUE}(Q, s)$ 3. While Q not empty: - $u = \text{DEQUEUE}(Q)$ - For each v in $\text{Adj}[u]$: - If $v.\text{color} == \text{WHITE}$: $v.\text{color}=\text{GRAY}$; $v.d = u.d+1$; $v.\pi = u$; $\text{ENQUEUE}(Q, v)$ - $u.\text{color} = \text{BLACK}$ - $O(V+E)$ (each vertex enqueued/dequeued once; each edge examined once from its source) - **Lemmas**: - Lemma 20.1: $v.d \geq \delta(s,v)$ always; after BFS

completes, $v.d = \delta(s, v)$ for all v - Lemma 20.2: during BFS, queue contains vertices with $d \in \{k, k+1\}$ for some integer k (nondecreasing property) - Proof by induction on queue operations - Lemma 20.3: for any edge (u, v) , $|u.d - v.d| \leq 1$ after BFS - **BFS tree**: vertices with $\pi \neq \text{nil}$ form breadth-first tree; edges are tree edges - If v is reachable, there is a unique simple path in BFS tree from s to v - BFS tree gives shortest paths in unweighted graphs - **Applications**: shortest path in unweighted graphs, connected components, bipartite testing, web crawling, GPS navigation

20.3 – DFS (Depth-First Search) - Explores deeper first; uses recursion (implicit stack) - Attributes: $u.\text{color}$ (WHITE/GRAY/BLACK), $u.d$ (discovery time), $u.f$ (finish time), $u.\pi$ - DFS(G): 1. For each u : $u.\text{color} = \text{WHITE}$; $u.\pi = \text{nil}$ 2. $\text{time} = 0$ 3. For each u : if $u.\text{color} == \text{WHITE} \rightarrow \text{DFS-VISIT}(u)$ - DFS-VISIT(u): 1. $\text{time}++$; $u.d = \text{time}$; $u.\text{color} = \text{GRAY}$ 2. For each v in $\text{Adj}[u]$: if $v.\text{color} == \text{WHITE} \rightarrow v.\pi = u$; DFS-VISIT(v) 3. $u.\text{color} = \text{BLACK}$; $\text{time}++$; $u.f = \text{time}$ - $O(V+E)$ - **Parenthesis theorem (20.7)**: For any u, v , exactly one of:
- $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint $\rightarrow$ neither is ancestor/descendant
- $[u.d, u.f]$ contains $[v.d, v.f]$ $\rightarrow u$ is ancestor of v
- $[v.d, v.f]$ contains $[u.d, u.f]$ $\rightarrow v$ is ancestor of u - Proof: based on timing of color changes; if u is discovered before v ($u.d < v.d$), then v is descendant of u iff $u.f > v.f$ - **Edge classification during DFS**: - **Tree edge**: (u, v) where v was WHITE at time (u, v) examined $\rightarrow v.\pi = u$ - **Back edge**: (u, v) where v is GRAY (ancestor in DFS tree) $\rightarrow$ detects cycles - **Forward edge**: (u, v) where v is BLACK and $u.d < v.d$ (non-tree edge to descendant) - **Cross edge**: (u, v) where v is BLACK and $u.d > v.d$ (non-tree edge to already-finished vertex; can go between different trees) - Undirected graphs: only tree and back edges (no forward/cross) - Directed graph is DAG $\iff$ DFS has no back edges $\iff$ no cycles - **White-path theorem (20.8)**: v is descendant of u in DFS forest $\iff$ at time $u.d$, v is reachable from u along a path of entirely white vertices - Proof: ($\implies$) immediate; ($\impliedby$) by induction on white path; if $u \rightarrow \dots \rightarrow w \rightarrow v$ with w discovered first, then v discovered during w 's DFS-VISIT, making v descendant of u - **Theorem 20.9 (Edge classification in undirected graphs)**: during DFS of undirected graph, every edge is either tree edge or back edge (never forward or cross) - Applications: topological sort, SCC, cycle detection, maze solving, articulation points, bridges, biconnected components - Applications: topological sort, SCC, cycle detection, maze solving

20.4 – Topological sort - **Input**: DAG (Directed Acyclic Graph) - **Output**: linear ordering such that for each $(u, v) \in E$, u appears before v - TOPOLOGICAL-SORT(G): 1. DFS(G) to compute finish times $f[u]$ for all u 2. As each vertex finishes (turns BLACK), insert at front of linked list (or push onto stack) 3. Return linked list (reading front to back gives topological order) - $O(V+E)$ - **Lemma 20.11 (Cycle detection)**: directed graph is DAG $\iff$ DFS has no back edges - **Theorem 20.12 (Correctness)**: TOPOLOGICAL-SORT produces valid linear ordering - Proof: consider any edge (u, v)

in DAG. When (u,v) is explored during DFS from u : - v cannot be GRAY (would be back edge $\rightarrow$ cycle) - If v is WHITE: v becomes descendant of $u \rightarrow v.f < u.f$ - If v is BLACK: already finished $\rightarrow v.f < u.f$ (or we can argue directly from DFS properties) - Since we output in decreasing f time, u appears before v - **Corollary 20.13**: G has cycle $\iff$ DFS has back edge $\iff$ no topological sort exists - **Why DAG required**: any cycle creates impossibility; all edges go same direction in cycle, creating contradiction - **Alternative approach**: Kahn's algorithm (repeatedly remove vertices with indegree 0 using queue) - also $O(V+E)$ - **Applications**: course prerequisite planning, build systems (Makefile), instruction scheduling in compilers, job scheduling with dependencies

20.5 – Strongly Connected Components (SCC) - **SCC**: maximal set of vertices where each can reach each other (mutually reachable) - **Kosaraju's algorithm**: 1. DFS(G) to compute finish times $f[u]$ for all u 2. Compute G^T (transpose: reverse every edge) 3. DFS(G^T) processing vertices in decreasing $f[u]$ order (from step 1) 4. Each DFS tree in step 3 is one SCC component - $O(V+E)$ - **Correctness (Theorem 20.14)**: Kosaraju's algorithm correctly finds SCCs - Key insight: if u and v are in same SCC, they appear in same DFS tree of G^T when processed in decreasing finish order from original DFS - Lemma 20.13: Let C and C' be SCCs with edge (u,v) from C to C' in G ; then $\max_{\{x \in C\}} f[x] > \max_{\{y \in C'\}} f[y]$ (SCCs with edges are visited in topological order) - When processing G^T in decreasing f order, the SCC with highest f (leaf in component DAG) has no incoming edges in $G^{\{SCC\}}$, so its vertices are discovered and finished together - **Component graph $G^{\{SCC\}}$** : one vertex per SCC; edge $(C_i \rightarrow C_j)$ if $\exists$ edge from C_i to C_j in G - $G^{\{SCC\}}$ is always a DAG (if cycles existed, SCCs would merge) - Can compute topological sort of $G^{\{SCC\}}$ using finish times from original DFS - **Applications**: dependency resolution, program optimization (loop detection), social network analysis (community detection), web page ranking

Ch 21 – Minimum Spanning Trees

- Input: connected undirected $G(V,E)$, weight $w: E \rightarrow \mathbb{R}$
- **MST**: spanning tree with minimum total weight
- **Generic-MST (greedy)**:
 1. $A = \emptyset$
 2. While A not spanning tree: find safe edge (u,v) for A ; $A = A \cup \{(u,v)\}$
 3. Return A

21.1 – Growing a minimum spanning tree - **Cut $(S, V-S)$** : partition of V ; edge (u,v) **crosses cut** if $u \in S, v \notin S$ - **Respect**: set A **respects** cut if no edge in A crosses cut - **Light edge**: edge crossing a cut with minimum weight - **Cut property (Theorem 21.1)**: Let $A \subseteq E$ be subset of some MST. Let $(S, V-S)$ be any cut respected by A . If (u,v) is a light edge crossing $(S, V-S)$, then (u,v) is safe for A . - Proof: construct MST T containing A ; if

(u,v) not in T , swap with heavier edge in T crossing cut -
Corollary 21.2: If edge weights are distinct, MST is unique -
Corollary 21.3: Let C be component in forest $G_A = (V,A)$;
 lightest edge from C to rest of V is safe for A - **GENERIC-MST**
loop invariant: A is subset of some MST; add safe edges until
 spanning tree

21.2 – Kruskal and Prim - Kruskal's algorithm: 1. Sort edges by weight 2. Disjoint-set forest: each vertex in own set 3. For each edge (u,v) in sorted order: - If $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$: add edge to MST; $\text{UNION}(u,v)$ - $O(E \lg V)$ (or $O(E \lg E)$ for sorting) - Uses disjoint-set data structure - **Kruskal's algorithm:** 1. Sort edges by weight ($O(E \lg E) = O(E \lg V)$) 2. Initialize disjoint-set forest: $\text{MAKE-SET}(v)$ for each v ($O(V)$) 3. For each edge (u,v) in sorted order (nondecreasing): - If $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$: add edge to MST; $\text{UNION}(u,v)$ - $O(E \lg V)$ (sorting dominates, union/find nearly $O(1)$ per operation) - Grows forest of trees; merges when light edge connects two components - Invariant: A is subset of some MST (cut property: lightest edge crossing cut between components is safe) - **Prim's algorithm:** 1. Start from arbitrary root r ; $\text{key}[r]=0$, for all $v \neq r$: $\text{key}[v]=\infty$, $\pi[v]=\text{nil}$ 2. Min-priority queue $Q = V$ keyed by key values 3. While Q not empty: $u = \text{EXTRACT-MIN}(Q)$; $S = S \cup \{u\}$; for each v in $\text{Adj}[u]$: - If v in Q and $w(u,v) < v.\text{key}$: $v.\text{key} = w(u,v)$; $v.\pi = u$; $\text{DECREASE-KEY}(Q, v, v.\text{key})$ - $O(E \lg V)$ with binary heap; $O(E + V \lg V)$ with Fibonacci heap; $O(V^2)$ for adjacency matrix + array Q - Grows single tree from root like Dijkstra; always adds light edge crossing cut $(S, V-S)$ - Invariant: $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$ - **Correctness:** by cut property; the cut $(S, V-S)$ where S = vertices already removed from Q is respected by A ; EXTRACT-MIN gives vertex with minimum-weight edge crossing cut - **Kruskal vs Prim:** - Kruskal: global sort + disjoint-set; $O(E \lg V)$; good for sparse graphs - Prim: adjacency list + heap: $O(E \lg V)$; adjacency matrix + array: $O(V^2)$; good for dense graphs - Both greedy; both rely on cut property - **If edge weights are unique:** MST is unique

Ch 22 – Single-Source Shortest Paths

- **Variants:** single-destination (reverse edges), single-pair (same as single-source), all-pairs
- **Optimal substructure:** any subpath of a shortest path is itself a shortest path (Lemma 22.1)
 - Proof: if subpath were longer, could replace it for shorter total path
- **Negative-weight cycles:** if reachable from s , shortest path may not exist (can cycle to reduce weight arbitrarily)
- **Relaxation:** for edge (u,v) : if $v.d > u.d + w(u,v)$: $v.d = u.d + w(u,v)$; $v.\pi = u$
 - Key operation in all shortest-path algorithms
 - Only decreases distance estimates (monotonic)
- **Triangle inequality:** $\delta(s,v) \leq \delta(s,u) + w(u,v)$ for all $(u,v) \in E$

22.1 – Bellman-Ford algorithm - Handles negative-weight edges; detects negative-weight cycles reachable from s - **BELLMAN-FORD(G, w, s)**: 1. INIT-SINGLE-SOURCE(G, s): for all v : $v.d = \infty$, $v.\pi = \text{nil}$; $s.d = 0$ 2. For $i = 1$ to $|V|-1$: - For each $(u, v) \in E$: RELAX(u, v, w) - RELAX: if $v.d > u.d + w(u, v)$: $v.d = u.d + w(u, v)$; $v.\pi = u$ 3. For each $(u, v) \in E$: if $v.d > u.d + w(u, v) \rightarrow$ return FALSE (negative-weight cycle detected) 4. Return TRUE - $O(VE)$ - **Loop invariant (passes $1..|V|-1$)**: after i passes, $v.d \leq$ weight of any shortest path from s to v using $\leq i$ edges - Initially ($i=0$): $s.d = 0$ (path of 0 edges) - After i passes: if shortest path to v has $\leq i$ edges, $v.d = \delta(s, v)$ - After $|V|-1$ passes: all shortest paths found (since any path has $\leq |V|-1$ edges) - **Theorem 22.2 (Correctness)**: If no negative-weight cycles reachable from s , after $|V|-1$ passes, $v.d = \delta(s, v)$ for all reachable v - Proof: any shortest path has $\leq |V|-1$ edges (no cycles); by path-relaxation property, relaxing edges in order of each pass gives correct distances - **Corollary 22.3**: If Bellman-Ford returns FALSE, G has negative-weight cycle (detected by 3rd phase) - If TRUE, predecessor subgraph G_π is shortest-paths tree - **Negative-cycle detection**: after $|V|-1$ passes, any further relaxable edge must be on or reachable from a negative-weight cycle

22.2 – Single-source shortest paths in DAGs - DAG-SHORTEST-PATHS(G, w, s): 1. Topological sort of G 2. INIT-SINGLE-SOURCE(G, s) 3. For each u in topological order: for each $(u, v) \in E$: RELAX(u, v, w) - $O(V+E)$ - Works with negative-weight edges (DAG guarantees no cycles) - **Theorem 22.5**: DAG-SHORTEST-PATHS computes correct shortest paths - Proof: by path-relaxation property; edges of any shortest path are relaxed in order due to topological sort

22.3 – Dijkstra's algorithm - Requires nonnegative edge weights - **DIJKSTRA(G, w, s)**: 1. INIT-SINGLE-SOURCE(G, s): $s.d = 0$, all others ∞ 2. $S = \emptyset$ (set of vertices with final distances) 3. $Q = V$ (min-priority queue keyed by d values) 4. While $Q \neq \emptyset$: - $u = \text{EXTRACT-MIN}(Q)$ - $S = S \cup \{u\}$ - For each v in $\text{Adj}[u]$: RELAX(u, v, w) (which may call DECREASE-KEY on Q) - $O(V^2)$ with unsorted array for Q (EXTRACT-MIN $O(V)$ per iteration = $O(V^2)$, each edge relax $O(1) = O(E)$) - $O(E \lg V)$ with binary heap; $O(V \lg V + E)$ with Fibonacci heap - **Loop invariant**: at start of each iteration of while loop, $u.d = \delta(s, u)$ for all $u \in S$ - Initially $S = \emptyset$, invariant vacuously true - Maintained: when u extracted with min d from Q , its d equals shortest path distance - **Theorem 22.6 (Correctness)**: Dijkstra computes correct shortest paths for nonnegative weights - Proof by contradiction: let u be first vertex extracted with $u.d > \delta(s, u)$; let $p = s \rightarrow \dots \rightarrow x \rightarrow y \rightarrow \dots \rightarrow u$ be true shortest path to u , where $x \in S$, $y \notin S$ - Since $x \in S$, $x.d = \delta(s, x)$; after relaxing (x, y) , $y.d = \delta(s, y)$ (convergence property) - Since y is on shortest path to u : $\delta(s, y) \leq \delta(s, u)$, so $y.d \leq \delta(s, u) < u.d$ (by assumption) - But u is min in Q , so $u.d \leq y.d$ - contradiction. Thus $u.d = \delta(s, u)$ - Once $u.d = \delta(s, u)$, it never changes (upper-bound property) - **Comparison with Prim's MST**: structurally identical; Dijkstra uses sum of edge weights from source; Prim

uses min edge weight connecting to tree - **Limitation:** fails with negative edges – vertices might need re-extraction after their d decreases due to negative edge later

22.4 – Difference constraints and shortest paths - System of difference constraints: m constraints, n variables: $x_j - x_i \leq b_k$ - Example: scheduling ($x_j \geq x_i + t \rightarrow x_i - x_j \leq -t$) - Can represent many linear inequality systems - **Constraint graph:** - One vertex v_i per variable x_i - Edge ($v_i \rightarrow v_j$) with weight b_k for each $x_j - x_i \leq b_k$ - Add source vertex v_π with 0-weight edges to all v_i (ensures connectivity, no effect on feasibility) - **Theorem 22.8:** Feasible solution π constraint graph has no negative-weight cycle reachable from v_π - Proof: (i) if cycle exists, sum constraints along cycle gives $0 \leq$ negative sum, contradictory; (ii) shortest distances from v_π satisfy all constraints by triangle inequality - **Solution:** $x_i = \delta(v_\pi, v_i)$ gives feasible assignment (may be negative; can shift by constant) - **Application:** scheduling tasks with time constraints (job j must start $\geq$ job $i + 5$ days, etc.) - Running time: $O(VE)$ using Bellman-Ford (detects infeasibility via negative cycles)

22.5 – Proofs of shortest-paths properties (key correctness lemmas) - **Lemma 22.10 (Triangle inequality):** $\delta(s, v) \leq \delta(s, u) + w(u, v)$ for all $(u, v) \in E$ - Proof: shortest path $s \rightarrow v \leq$ any path, including $s \rightarrow u \rightarrow v$ - **Lemma 22.11 (Upper-bound property):** $v.d \geq \delta(s, v)$ always; once $v.d = \delta(s, v)$, value never changes - **Lemma 22.12 (No-path property):** If v unreachable from s , $v.d = \infty$ forever - **Lemma 22.13 (Convergence property):** If $s \rightarrow u \rightarrow v$ is shortest path and $u.d = \delta(s, u)$ at time of $\text{RELAX}(u, v, w)$, then after that relaxation $v.d = \delta(s, v)$ and never changes - Key to proving Dijkstra and Bellman-Ford - **Lemma 22.14 (Path-relaxation property):** Let $p = \langle v_\pi, \dots, v_k \rangle$ be a shortest path. If edges are relaxed in order $(v_\pi, v_\pi), (v_\pi, v_\pi), \dots, (v_{k-1}, v_k)$ (even with other relaxations interleaved), then $v_k.d = \delta(s, v_k)$. After this, $v_k.d$ never changes - **Lemma 22.15 (Predecessor-subgraph property):** Once $v.d = \delta(s, v)$ for all reachable v , predecessor subgraph $G_\pi = (V_\pi, E_\pi)$ is a shortest-paths tree rooted at s - $V_\pi = \{v \in V : v.\pi \neq \text{nil}\} \cup \{s\}$; $E_\pi = \{(v.\pi, v) : v \in V_\pi - \{s\}\}$

Ch 23 – All-Pairs Shortest Paths

- Goal: shortest path weight $\delta(i, j)$ for every pair (i, j)
- Input: $n \times n$ weight matrix $W = (w_{ij})$ where $w_{ij} = 0$ if $i=j$, $w(u, v)$ if $(u, v) \in E$, ∞ otherwise
- Output: distance matrix $D = (d_{ij}) = \delta(i, j)$; optionally predecessor matrix Π for path reconstruction
- Assume no negative-weight cycles (detected by algorithms)

23.1 – Shortest paths and matrix multiplication - DP formulation: $L^{\{m\}}\{ij\} = \text{minimum weight of path from } i \text{ to } j \text{ using } \leq m \text{ edges}$ - Base: $L^{\{1\}} = W$ (direct edges) - Recurrence: $L^{\{m\}}\{ij\} = \min_{1 \leq k \leq n} \{ L^{\{m-1\}}\{ik\} + w\{kj\} \}$ - Intuition: extend path of $\leq m-1$ edges from i to k by adding edge (k, j) - This is matrix

multiplication in $(\min, +)$ semiring (tropical algebra) - **EXTEND-SHORTEST-PATHS**(L, W): computes $L^{\{m\}}$ from $L^{\{m-1\}}$ and W - For $i=1..n, j=1..n$: $l^{\{ij\}} = \min_{k=1..n} \{ l^{\{ik\}} + w_{\{kj\}} \}$ - $O(n^3)$ per call - **SLOW-ALL-PAIRS-SHORTEST-PATHS**(W): compute $L^{\{1\}}, L^{\{2\}}, \dots, L^{\{n-1\}}$ - $L^{\{n-1\}} = D$ (since any shortest path has $\leq n-1$ edges) - $O(n^4)$ - **FASTER-ALL-PAIRS-SHORTEST-PATHS (repeated squaring)**: - $(\min, +)$ multiplication is associative: $(L \otimes W) \otimes W = L \otimes (W \otimes W)$ - Compute $L^{\{1\}}, L^{\{2\}}, L^{\{4\}}, \dots, L^{\{2^{\lceil \lg(n-1) \rceil}\}}$ by squaring: $L^{\{2m\}} = L^{\{m\}} \otimes L^{\{m\}}$ - $O(n^3 \lg n)$ - Correctness: $L^{\{m\}}$ gives shortest paths with $\leq m$ edges; after squaring enough times ($\geq n-1$), all paths found

23.2 – Floyd-Warshall algorithm - DP formulation: $d^{\{k\}}\{ij\}$ = weight of shortest path from i to j with intermediate vertices only in $\{1, \dots, k\}$ - Recurrence: $d^{\{k\}}\{ij\} = \min(d^{\{k-1\}}\{ij\}, d^{\{k-1\}}\{ik\} + d^{\{k-1\}}\{kj\})$ - Intuition: either don't use vertex k , or use k once as intermediate vertex - Optimal substructure: any subpath of a shortest path is shortest (using intermediate vertices from subset) - Base case: $d^{\{0\}}\{ij\} = w_{\{ij\}}$ (no intermediate vertices) - **FLOYD-WARSHALL**(W): 1. $D^{\{0\}} = W$ ($n \times n$ matrix) 2. For $k = 1$ to n : - For $i = 1$ to n : - For $j = 1$ to n : - $d^{\{k\}}\{ij\} = \min(d^{\{k-1\}}\{ij\}, d^{\{k-1\}}\{ik\} + d^{\{k-1\}}\{kj\})$ 3. Return $D^{\{n\}}$ - $O(n^3)$; $O(n^3)$ even for sparse graphs (uses matrix representation, not adjacency list) - In-place: can overwrite D in place since $d^{\{k\}}\{ik\} = d^{\{k-1\}}\{ik\}$ and $d^{\{k\}}\{kj\} = d^{\{k-1\}}\{kj\}$ (row k , column k unchanged) - **Predecessor matrix Π** : reconstruct actual paths - Initialize $\pi^{\{0\}}\{ij\} = i$ if $i=j$ or $(i,j) \in E$, NIL otherwise - Update: if $d^{\{k-1\}}\{ik\} + d^{\{k-1\}}\{kj\} < d^{\{k-1\}}\{ij\}$ then $\pi^{\{k\}}\{ij\} = \pi^{\{k-1\}}\{kj\}$ - Path from i to j : follow Π from i to j ($i \rightarrow \dots \rightarrow \Pi[i][j] \rightarrow j$) - **Transitive closure** (Section 23.2): - $t^{\{k\}}\{ij\} = 1$ iff path $i \rightarrow j$ exists using vertices $\{1..k\}$ - Recurrence: $t^{\{k\}}\{ij\} = t^{\{k-1\}}\{ij\} \vee (t^{\{k-1\}}\{ik\} \wedge t^{\{k-1\}}\{kj\})$ - Same triple loop as Floyd-Warshall; $O(V^3)$ - Can also use boolean matrix multiplication (bitwise OR/AND) for potential bit-parallel speedup - **Theorem 23.10 (Correctness)**: Floyd-Warshall computes correct shortest paths ($d^{\{n\}}\{ij\} = \delta(i,j)$) if no negative-weight cycles - Proof by induction on k : base $k=0$ holds (direct edges); step: claim holds for paths using $\{1..k-1\}$; path using k either skips k ($d^{\{k-1\}}\{ij\}$) or goes $i \rightarrow k$ and $k \rightarrow j$ ($d^{\{k-1\}}\{ik\} + d^{\{k-1\}}\{kj\}$) - **Negative cycle detection**: if after completion, $d\{ii\} < 0$ for any i , negative cycle exists

23.3 – Johnson's algorithm for sparse graphs - Combines Bellman-Ford + Dijkstra; uses reweighting to eliminate negative edges while preserving shortest paths - **Why needed**: Floyd-Warshall is $O(V^3)$ regardless of sparsity; for sparse graphs we can do better - **Reweighting**: define potential $h: V \rightarrow \mathbb{R}$; new weight $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$ - **Lemma 23.12 (Preservation)**: For any u,v , a path p from u to v has weight $\hat{w}(p) = w(p) + h(u) - h(v)$; thus shortest paths under $\hat{w}$ correspond exactly to shortest paths under w - Proof: telescoping sum: $\sum \hat{w} = \sum (w + h(u) - h(v)) = w(p) + h(u) - h(v)$ - **Lemma 23.13 (Nonnegativity)**: If $h(v) = \delta(s,v)$

(shortest distance from added source s), then $\hat{w}(u,v) \geq 0$ for all edges - Proof: triangle inequality: $\delta(s,v) \leq \delta(s,u) + w(u,v) \rightarrow h(v) \leq h(u) + w(u,v) \rightarrow \hat{w}(u,v) = w(u,v) + h(u) - h(v) \geq 0$ - **JOHNSON(G,w)**: 1. Add new vertex s with 0-weight edges ($s \rightarrow v$) to all v (forming G') 2. Run Bellman-Ford(G', w, s) $\rightarrow h(v) = \delta(s, v)$ for all v ; if negative cycle detected $\rightarrow$ abort 3. Reweight: $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$ (now all $\hat{w} \geq 0$) 4. For each $u \in V$: - Run Dijkstra($G, \hat{w}, u$) $\rightarrow \delta(u,v)$ for all v - Recover original distance: $\delta(u,v) = \delta(u,v) + h(v) - h(u)$ 5. Return distance matrix D - Running time: - Bellman-Ford: $O(V E)$ - $V \times$ Dijkstra: $O(V \cdot E \lg V)$ with binary heap; $O(V^2 \lg V + V E)$ with Fibonacci heap - Total: $O(V E + V^2 \lg V)$ with binary heap; $O(V E + V^2 \lg V)$ with Fibonacci (since $E \geq V-1$) - Best for sparse graphs where $E = o(V^2 / \lg V)$ - **Example (Figure 23.6)**: 5-vertex graph with negative edges; Bellman-Ford from added source s gives h values; all $\hat{w}$ become nonnegative; Dijkstra from each vertex; distances recovered

Ch 24 – Maximum Flow

24.1 – Flow networks - **Flow network**: directed $G(V,E)$ with source s , sink t ; each edge (u,v) has capacity $c(u,v) \geq 0$ - No antiparallel edges (if $(u,v) \in E$, then $(v,u) \notin E$); can transform by splitting one edge with intermediate vertex - No self-loops - $c(u,v) = 0$ if $(u,v) \notin E$ - Every vertex lies on some path $s \rightarrow v \rightarrow t$ (so $|E| \geq |V|-1$) - **Flow**: function $f: V \times V \rightarrow \mathbb{R}$ satisfying: - **Capacity constraint**: $0 \leq f(u,v) \leq c(u,v)$ (flow cannot exceed capacity) - **Flow conservation**: $\sum_{v \in V} f(v,u) = \sum_{v \in V} f(u,v)$ for all $u \in V - \{s,t\}$ (flow in = flow out) - **Skew symmetry**: $f(u,v) = -f(v,u)$ (for analysis convenience) - **Value of flow**: $|f| = \sum_{v \in V} f(s,v) - \sum_{v \in V} f(v,s) =$ net flow out of source - Also = net flow into sink = $\sum_{v \in V} f(v,t) - \sum_{v \in V} f(t,v)$ - **Maximum-flow problem**: find flow f maximizing $|f|$ - **Example (Figure 24.1)**: Lucky Puck Company shipping from Vancouver (s) to Winnipeg (t) through intermediate cities; capacities on edges - **Multiple sources/sinks**: add super-source with ∞ edges to all sources, super-sink with ∞ edges from all sinks - **Properties**: - Flow value equation: $|f| = f(S,T)$ for any cut (S,T) containing s,t (flow decomposition) - $|f| + |f'| = |f + f'|$ for flows (linearity) - Capacity of cut is upper bound on flow across it

24.2 – Ford-Fulkerson method - **FORD-FULKERSON(G,s,t)**: 1. For each $(u,v) \in E$: $f(u,v) = 0$ 2. While $\exists$ path p from s to t in residual network G_f : - $c_f(p) = \min_{(u,v) \in p} c_f(u,v)$ (bottleneck capacity) - For each (u,v) in p : $f(u,v) += c_f(p)$; $f(v,u) -= c_f(p)$ 3. Return f - **Residual network G_f** : edges with residual capacity $c_f(u,v) = c(u,v) - f(u,v)$ - If $f(u,v) > 0$, also include reverse edge (v,u) with $c_f(v,u) = f(u,v)$ - Augmenting path = any path from s to t in G_f - **Augmenting path**: path in G_f ; bottleneck = min residual capacity along path - Augmentation increases flow value by bottleneck amount - **Cuts**: partition (S,T) of V with $s \in S, t \in T$ - Net flow $f(S,T) = \sum_{u \in S, v \in T} f(u,v) - \sum_{u \in S, v \in T} f(v,u)$ - Capacity $c(S,T) = \sum_{u \in S, v \in T} c(u,v)$ - For

any flow f and any cut (S,T) : $|f| = f(S,T) \leq c(S,T)$ - **Max-flow min-cut theorem (24.6)**: For any flow network, these are equivalent: 1. f is maximum flow 2. Residual network G_f has no augmenting path 3. $|f| = c(S,T)$ for some cut (S,T) - Proof structure: - (1) $\Rightarrow$ (2): if augmenting path exists, could increase flow $\rightarrow f$ not max (contradiction) - (2) $\Rightarrow$ (3): let $S =$ vertices reachable from s in G_f ($s \in S$, $t \notin S$ by (2)); then (S,T) is cut; all original edges from S to T are saturated ($c_f=0$), so $f(u,v) = c(u,v)$ for $u \in S$, $v \in T$; all reverse edges are unused ($f(v,u)=0$); thus $|f| = f(S,T) = c(S,T)$ - (3) $\Rightarrow$ (1): for any flow f' and any cut (S,T) : $|f'| \leq c(S,T)$; since $|f| = c(S,T)$, f is max - **Corollary**: The min cut (minimum capacity cut) equals max flow value; the set of vertices reachable from s in residual graph of max flow is a min cut - **Application**: find min cut by computing max flow, then BFS from s in G_f - **Edmonds-Karp algorithm**: Ford-Fulkerson using BFS to find shortest augmenting path (minimum #edges) - $O(V \cdot E^2)$ - **Lemma 24.7**: $\delta_f(s,v)$ (shortest path distance in G_f) never decreases with augmentations; monotonic - Proof: by contradiction; first time distance decreases, examine critical edge - **Theorem 24.8**: Each edge can be critical (bottleneck) at most $O(V)$ times $\rightarrow O(V \cdot E)$ critical edges $\times O(E)$ per BFS = $O(V \cdot E^2)$ - **Lemma 24.9**: If (u,v) becomes critical on augmenting path, it disappears from residual; can reappear only after its reverse edge (v,u) is used, which requires $\delta_f(s,u)$ to increase by at least 2 - **Corollary 24.10**: Number of flow augmentations in Edmonds-Karp is $O(V \cdot E)$ - **Integrality theorem (24.11)**: If all capacities are integers, Ford-Fulkerson produces integer max flow - Corollary: max matching in bipartite graph has integer cardinality (obviously) - **Pathological case**: with irrational capacities, Ford-Fulkerson may never terminate (ex 24.2-11) - **Key concepts for exam**: - How to compute residual network given current flow - How to identify min cut from max flow (reachability from s in G_f) - Relationship between augmenting paths and min cut capacity

24.3 – Maximum bipartite matching - **Bipartite graph**: $G = (V,E)$, $V = L \sqcup R$, $L \cap R = \emptyset$; every edge connects L to R - **Matching M** : set of edges with no shared vertices (each vertex incident to ≤ 1 edge in M) - **Maximum matching**: matching of maximum cardinality $|M|$ - **Perfect matching**: every vertex is incident to exactly one edge in M (requires $|L| = |R|$) - **Reduction to max-flow**: 1. Source s connected to each $u \in L$ (capacity 1) 2. Each $u \in L$ connected to $v \in R$ if $(u,v) \in E$ (capacity 1) 3. Each $v \in R$ connected to sink t (capacity 1) 4. All capacities are 1 (integer) $\rightarrow$ max flow value = size of maximum matching 5. Edges with $f(u,v) = 1$ form the matching - **Correctness**: each vertex can receive at most 1 unit of flow (limited by source/sink edges) $\rightarrow$ no two matching edges share vertex; flow saturates maximum possible edges - Running time: $O(V \cdot E)$ with Ford-Fulkerson (since each augmenting path increases flow by 1, at most $|V|$ augmentations, each $O(E)$); $O(\sqrt{V} \cdot E)$ with Hopcroft-Karp

Ch 25 – Matchings in Bipartite Graphs

25.1 – Maximum bipartite matching (revisited) - Hopcroft-Karp

algorithm: more efficient than Ford-Fulkerson for bipartite matching - $O(E\sqrt{V})$ time (vs $O(VE)$ for unit-capacity Ford-Fulkerson) - **Key idea:** find maximal set of vertex-disjoint shortest augmenting paths in each phase - Phase: BFS from unmatched L vertices builds layered residual graph; DFS finds maximal set of augmenting paths that are vertex-disjoint and shortest - Each phase saturates at least one new edge per vertex - After $O(\sqrt{V})$ phases, matching is maximal - **Algorithm outline:** 1. Start with empty matching M 2. Repeat until no augmenting path: - BFS: from all unmatched vertices in L, compute distance layers; stop at t - If t unreachable $\rightarrow$ done - DFS: repeatedly find augmenting paths using layered graph (avoid reusing vertices) - Each DFS removes found paths from graph 3. Return M - **Analysis:** - Each phase: $O(E)$ (BFS to build layers + DFS to find maximal disjoint paths) - **Lemma 25.1:** After each phase, the shortest augmenting path length strictly increases - **Lemma 25.2:** After k phases, the matching is within $|V|/(k+1)$ of optimal - Only $O(\sqrt{V})$ phases needed: once path length $\geq \sqrt{V}$, at most $\sqrt{V}$ augmentations remain (each increases matching size by 1, and max matching size $\leq V/2$) - Total: $O(\sqrt{V} \cdot E) = O(E\sqrt{V})$ - **Comparison with Ford-Fulkerson:** - Ford-Fulkerson on unit-capacity bipartite network: each augmentation increases flow by 1, at most V augmentations, each $O(E) \rightarrow O(VE)$ - Hopcroft-Karp: uses layered graph to find multiple disjoint augmentations per phase $\rightarrow O(\sqrt{V} E)$ - This is the fastest known algorithm for general bipartite matching in worst case

25.2 – The stable-marriage problem - Input: n men and n women; each person ranks all n members of opposite sex (strict preference order) -

Stable matching: matching M with no **blocking pair** (m,w) where both prefer each other to current partner - If (m,w) both prefer each other over their M-partners, they would “run off together” $\rightarrow$ unstable - **Gale-Shapley algorithm** (deferred acceptance): - While $\exists$ unmatched man m: - m proposes to highest-ranked woman w not yet proposed to - If w unmatched $\rightarrow$ (m,w) engaged - Else if w prefers m to current fiancé m' $\rightarrow$ (m,w) engaged; m' becomes free - Else $\rightarrow$ m remains free (w rejects m) - $O(n^2)$ proposals (each man proposes at most n times) - **Correctness (Theorem 25.2):** Gale-Shapley produces stable matching - Proof: suppose (m,w) is blocking pair; m must have proposed to w before his current partner; at that time w either accepted (prefers m) or rejected (prefers current); if accepted, w would only leave for better; if rejected, w prefers current partner over m - contradicting blocking pair - **Male-optimality (Theorem 25.3):** each man gets best valid partner (partner in some stable matching) - Proof by contradiction: suppose some man does not; consider the last proposal rejection causing this - **Female-pessimality (Theorem 25.4):** each woman gets worst valid partner - Proof: if woman w is matched with m but prefers m' (who is matched with w' in stable matching), then (m,w') would be

blocking pair in male-optimal matching, contradiction - **Rural hospitals theorem**: set of unmatched people is same across all stable matchings; also, hospitals (men/women) that are not fully matched in one stable matching are not fully matched in any - **Lattice structure**: set of stable matchings forms distributive lattice under partial order - **Applications**: NRMP (National Resident Matching Program) – most famous application; college admissions; job placement - **Exercise 25.2-4**: show that Gale-Shapley with men proposing is male-optimal but female-pessimal – prove women get worst possible partner across all stable matchings

25.3 – The Hungarian algorithm for the assignment problem -

Assignment problem: n workers, n jobs; w_{ij} = cost of assigning worker i to job j ; find minimum-cost perfect matching in $K_{n,n}$ - Also called minimum-weight bipartite perfect matching - NP-hard for general graphs (but polynomial for bipartite) - **Hungarian algorithm** (Kuhn-Munkres, 1955-1957): $O(n^3)$ - **Duality framework** (LP duality): - **Primal**: minimize $\sum w_{ij} x_{ij}$ where $\sum_i x_{ij} = \sum_j x_{ij} = 1$, $x_{ij} \in \{0,1\}$ - **Dual**: maximize $\sum \ell(v)$ subject to $\ell(u) + \ell(v) \leq w(u,v)$ for all $(u,v) \in E$ - **Vertex labeling** (potential): $\ell: V \rightarrow \mathbb{R}$ ($\ell(l_i)$ for left, $\ell(r_j)$ for right), feasible if $\ell(u) + \ell(v) \leq w(u,v)$ - **Equality subgraph G_ℓ** : edges where $\ell(u) + \ell(v) = w(u,v)$ - **Theorem 25.9 (Kuhn-Munkres optimality condition)**: If ℓ is feasible and equality subgraph G_ℓ has a perfect matching M , then M is optimal (minimum-weight perfect matching) - Proof: for any perfect matching M' , weight $w(M') \geq \sum \ell(v) = w(M)$; so M is optimal - **Hungarian algorithm**: 1. Initialize feasible labeling: - For minimization: $\ell(l_i) = \min_j w_{ij}$; $\ell(r_j) = 0$ - For maximization: $\ell(l_i) = \max_j w_{ij}$; $\ell(r_j) = 0$ (or negate all weights) - Ensures $\ell(l_i) + \ell(r_j) \leq w_{ij}$ 2. Build equality subgraph G_ℓ ; find maximum matching M in G_ℓ (using augmenting paths / BFS) 3. While M is not perfect: - Let $u \in L$ be unmatched in M - Build alternating BFS tree in G_ℓ from u : $S \subseteq L$ (reachable via alternating paths), $T \subseteq R$ - Compute $\Delta = \min_{l \in S, r \in T} (w(l,r) - \ell(l) - \ell(r))$ - minimum slack - Update labels: $\ell(l) += \Delta$ for all $l \in S$; $\ell(r) -= \Delta$ for all $r \in T$ - Maintains feasibility (edges $S \rightarrow R$ improve; edges $L \rightarrow T$ get worse but feasible; edges within $S \times T$ unchanged) - New tight edges added to G_ℓ (at least one edge crossing $S \rightarrow R$ becomes tight) - Continue alternating tree / expand matching 4. Return M - **Slack optimization ($O(n^3)$ implementation)**: - Maintain $\text{slack}[j] = \min_{i \in S} (w_{ij} - \ell(l_i) - \ell(r_j))$ for each $j \in T$ - When ℓ updated: $\text{slack}[j] -= \Delta$ for $j \in T$; $\text{slack}[j]$ unchanged for $j \notin T$ - When vertex moves into S : update slack for all $j \in T$ - When vertex moves into T : check for 0-slack edges to continue alternating tree - Each phase: $O(n^2)$ to update slacks; $O(n)$ vertices added to S/T per phase; $O(n)$ phases $\rightarrow O(n^3)$ - **Example (Fig 25.7-25.15)**: 5×5 assignment; labels adjusted over 3 phases (augmenting paths); final equality subgraph contains perfect matching - First phase: single augmenting path found - Second phase: label update ($\Delta > 0$) creates new tight edges; another augmenting path - Third phase: final augmenting path completes

perfect matching - **Key insight:** Hungarian algorithm is primal-dual method; labels are dual variables; equalities correspond to complementary slackness ($x_{ij} > 0 \Rightarrow \ell(u_i) + \ell(v_j) = w_{ij}$) - **Theoretical importance:** first polynomial-time algorithm for assignment problem; basis for many combinatorial optimization algorithms (min-cost flow, general matching) - **Relationship to max-flow:** Hungarian uses augmenting paths much like max-flow; the alternating tree is analogous to BFS in residual graph - **Rectangular assignment:** n workers, m jobs, $n \leq m$; add $m-n$ dummy workers with zero-cost edges to all jobs $\rightarrow$ reduces to square case - **Exercise 25.3-3:** prove that Hungarian algorithm works correctly when capacities are integer (integrality property) – matching will be integer

Running-time summary (Ch 17-25):

Algorithm	Time	Key Technique
OS-SELECT/RANK	$O(\lg n)$	Augmented RB tree with subtree sizes
INTERVAL-SEARCH	$O(\lg n)$	Augmented RB tree with max endpoint
B-tree search/insert/delete	$O(t \log_t n)$ CPU, $O(\log_t n)$ disk	High branching factor, splitting
Disjoint-set operations	$O(m \alpha(n))$	Union by rank + path compression
BFS	$O(V+E)$	Queue-based level-order traversal
DFS	$O(V+E)$	Recursive stack-based exploration
Topological sort	$O(V+E)$	DFS + finish-time ordering
SCC (Kosaraju)	$O(V+E)$	Two DFS passes on G and G^T
Kruskal's MST	$O(E \lg V)$	Sort edges + disjoint-set
Prim's MST	$O(E \lg V)$ or $O(V^2)$	Min-priority queue (heap/array)
Bellman-Ford	$O(V \cdot E)$	Edge relaxation repeated $V-1$ times
DAG shortest paths	$O(V+E)$	Topological order + relaxation
Dijkstra	$O(E \lg V)$ or $O(V^2)$	Min-priority queue (nonnegative weights)
Floyd-Warshall	$O(V^3)$	DP over intermediate vertices
Johnson's APSP	$O(V \cdot E + V^2 \lg V)$	Reweighting + $V \times$ Dijkstra
Ford-Fulkerson max flow	$O(E \cdot f^*)$	f^*
Edmonds-Karp max flow	$O(V \cdot E^2)$	BFS for shortest augmenting paths

Algorithm	Time	Key Technique
Hopcroft-Karp matching	$O(E\sqrt{V})$	Layered BFS + maximal DFS phases
Gale-Shapley	$O(n^2)$	Deferred acceptance proposals
Hungarian assignment	$O(n^3)$	Primal-dual with labeling

Exam study tips – key things to practice: 1. **Augmenting RB trees:** given a new query, design the extra attribute; show it can be maintained during rotations in $O(1)$ (like size or max) 2. **B-tree insertion/deletion:** trace through with $t=2$ or $t=3$; know when splitting/merging occurs 3. **Disjoint-set forest:** trace FIND-SET with path compression; know rank updates 4. **Graph traversal:** classify edges in DFS; compute finish times; identify SCCs via Kosaraju 5. **MST:** apply cut property to prove edge safety; trace Kruskal/Prim on small graphs 6. **Shortest paths:** trace Bellman-Ford/Dijkstra/Floyd-Warshall; detect negative cycles 7. **Flow networks:** compute residual capacities; find augmenting paths; identify min cut from max flow 8. **Matching:** reduce bipartite matching to max-flow; trace Gale-Shapley (blocking pairs); trace Hungarian labeling 9. **Complexity:** know when to use each algorithm (e.g., Johnson vs Floyd-Warshall for sparse graphs) 10. **Proofs:** loop invariants (BFS queue, Dijkstra S-set, Bellman-Ford passes), cut-and-paste for optimal substructure, contradiction for stability

Key theorems to remember: - Theorem 17.1: Augmenting RB trees – attribute depends on children in $O(1)$ → maintainable in $O(\lg n)$ - Theorem 17.2: INTERVAL-SEARCH correctness – search always toward overlapping interval if exists - Theorem 18.1: B-tree height $h \leq \log_t((n+1)/2)$ - Max-flow min-cut theorem (24.6): three equivalent conditions for max flow - Cut property (21.1): light edge crossing cut respected by A is safe - Triangle inequality + 5 relaxation lemmas (22.10-22.15): foundation for shortest-path correctness - Theorem 25.9 (Kuhn-Munkres): feasible labeling + perfect matching in equality subgraph → optimal assignment ##
Part VII: Selected Topics – CLRS 4th Ed Chapters 26–35

Ch 26 – Parallel Algorithms

Model: Fork-join parallelism on an ideal parallel computer with sequentially consistent shared memory. Keywords: spawn, sync, parallel. Deleting parallel keywords yields the **serial projection**.

Trace DAG ($G=(V,E)$): Vertices = **strands** (maximal chains of instructions without parallel/procedural control). Edges = dependencies. Spawn creates two outgoing edges – one to child strand, one to continuation strand in parent. Sync joins multiple incoming edges from spawned children back to parent continuation.

- Two strands are **in series** if a directed path connects them.

- Two strands are **in parallel** if no path exists in either direction.

Performance Metrics:

- **Work** (T_1): total time on 1 processor = sum of strand execution times (number of vertices for unit-time strands).
- **Span** (T_∞): fastest possible time on unlimited processors = length of **critical path** (longest weighted directed path in trace DAG).
- **Work Law**: $(T_P \leq T_1/P)$. Each step can do at most (P) units of work.
- **Span Law**: $(T_P \geq T_\infty)$. Unlimited processors cannot beat the critical path.
- **Speedup** = (T_1/T_P) ($\leq P$).
- **Perfect linear speedup** when $(T_1/T_P = P)$.
- **Parallelism** = (T_1/T_∞) = average work per strand on critical path = maximum possible speedup.
- **Slackness** = $((T_1/T_\infty)/P = T_1/(PT_\infty))$. Measures if parallelism exceeds processors.

Greedy Scheduling (Theorem 26.1):

- **Complete step**: (P) strands ready $\rightarrow$ execute any (P) of them.
- **Incomplete step**: $(< P)$ strands ready $\rightarrow$ execute all.
- Greedy bound: $(T_P \leq T_1/P + T_\infty)$.
- Proof: (k) complete steps $\rightarrow$ at most (T_1/P) (each does (P) work). Each incomplete step reduces remaining span by 1 $\rightarrow$ at most (T_∞) steps.
- **Corollary 26.2**: Greedy within factor 2 of optimal $((T_P \leq 2T_\infty))$.
- **Corollary 26.3**: If slackness (s) (i.e., $(P \leq T_1/T_\infty)$), then $(T_P \leq T_1/P)$ (near-perfect linear speedup). Rule of thumb: slackness (s) suffices for good speedup.

Series-Parallel Composition:

- **Series composition**: work adds, span adds.
- **Parallel composition**: work adds, span = max of the two spans.

P-FIB – Parallel Fibonacci:

```
P-FIB(n)
1 if n ≤ 1 return n
2 else x = spawn P-FIB(n-1)
3     y = P-FIB(n-2)
4     sync
5     return x + y
```

- Work: $(T_1(n) = T_1(n-1) + T_1(n-2) + (1) = (\phi^n))$. Grows exponentially (same as serial FIB).
- Span: $(T_\infty(n) = (T_\infty(n-1), T_\infty(n-2)) + (1) = T_\infty(n-1) + (1) = (n))$.
- Parallelism: $((\phi^n/n))$ – grows exponentially with (n) . Even on largest parallel computers, modest (n) gives near-perfect linear speedup.

Parallel Loops: parallel for implemented via recursive binary splitting. Compiler generates divide-and-conquer auxiliary subroutine that spawns left half in parallel with right half, creating a binary tree of parallel execution. Base case = serial loop body.

Matrix-Vector Multiplication $((y = y + Ax))$:

```

P-MAT-VEC(A, x, y, n)
1 parallel for i = 1 to n
2     for j = 1 to n
3         y_i = y_i + a_{ij} x_j

```

- Work: $((n^2))$. Span: $((n + n))$ – serial inner loop contributes $((n))$, binary spawning adds $((n))$. Parallelism: $((n^2/n) = (n))$.

Parallel Merge Sort (P-MERGE): - Merge step: Use binary search to find position of middle element of first subarray in second subarray. Recursively merge left halves in parallel and right halves in parallel. - Merge span: $(T_{\infty}(n) = (n))$ (binary search + recursive parallel merge). Merge work: $(T_1^{\text{merge}}(n) = (n))$. - Full sort: $(T_1(n) = 2T_1(n/2) + (n) = (n \log n))$. $(T_{\infty}(n) = T_{\infty}(n/2) + (n) = (\log^2 n))$. Parallelism: $((n/\log n))$.

Parallel Matrix Multiplication:

```

P-MAT-MUL(A, B, C, n)
1 parallel for i = 1 to n
2     parallel for j = 1 to n
3         c_{ij} = 0
4         for k = 1 to n
5             c_{ij} = c_{ij} + a_{ik} b_{kj}

```

- Work: $((n^3))$. Span: $((n + n))$. Parallelism: $((n^3/n) = (n^2))$.
- Alternative with recursive block matrix multiplication: span reduces to $((n))$, work remains $((n^3))$.

Determinacy Races: Two logically parallel strands access same memory location and at least one write = race. Causes nondeterministic results (output depends on interleaving).

Deterministic computation = race-free. **DAG-race detection algorithm** can verify race-freedom in $(O(T_1))$ time given trace.

Key Exam Questions: - Compute (T_1) , (T_{∞}) , parallelism, slackness for a given fork-join program. - Prove greedy scheduling bound. - Design parallel algorithm using spawn/sync/parallel for; analyze work and span. - Identify determinacy races in pseudocode.

Ch 27 – Online Algorithms

Definition: Input arrives piece-by-piece. **Online algorithm** makes irrevocable decisions with only past knowledge. **Offline algorithm** knows entire input sequence in advance.

Competitive Analysis: For minimization problem, competitive ratio $= (\sup_I A(I)/F(I))$ where (F) is optimal offline algorithm (knows future). Algorithm is **c-competitive** if ratio (c) . Ratio always (≥ 1) .

Elevator vs Stairs Problem: - (k) floors. Stairs: (k) minutes. Elevator: arrives in (m) minutes $((0 \leq m \leq B-1))$, ride = 1 $\rightarrow$ total $(m+1)$. - Optimal offline: wait if $(m < k-1)$, else take stairs. - "Always take stairs": worst when elevator immediate $((m=0)) \rightarrow$ ratio = $(k/1 = k)$. - "Always take elevator": worst when stairs better $((m = B-1)) \rightarrow$ ratio = $((B-1+1)/k = B/k)$. - **Hedging:** Wait (k) minutes, then take stairs if no elevator. Time = (

). Competitive ratio = $(\{ (m+1)/(m+1), 2k/k \} = 2)$ (independent of (k, B)).

Ski-Rental Problem: Rent = $(r)/\text{day}$, Buy = (b) . Optimal offline: ski (d) days. If $(d < b/r) \rightarrow$ rent $((dr))$. If $(d \geq b/r) \rightarrow$ buy $((b))$. - Online strategy: Rent until total rent cost reaches (b) , then buy. Spend at most $(2b - r)$ in worst case. Competitive ratio = $(2 - r/b < 2)$. - Guarantees never spending more than twice what you would have spent if you knew (d) in advance.

Move-to-Front (MTF) List Maintenance: - Maintain sorted linked list. On access to element at position (r) : cost = $(2r-1)$ $((r)$ to search + $(r-1)$ adjacent swaps to move to front). - **Competitive ratio = 4.** Proof uses amortized analysis with potential function = inversion count $(I(L, L'))$ between MTF list and optimal list. - Define sets before (i) th search for element (x) : - BB = elements before (x) in both lists. - BA = elements before (x) in MTF but after (x) in optimal. - AB = elements after (x) in MTF but before (x) in optimal. - Cost analysis: MTF pays $(|| + || + 1)$ (search) + $(||)$ (moves past BA) = $(|| + 2|| + 1)$. Optimal pays at least $(|| + || + 1)$. Inversion count change bounds difference.

Online Caching/Paging: - Cache holds (k) pages. On access to page not in cache: **page fault** $\rightarrow$ must evict a page and load the new one. - **LRU** (Least Recently Used): evict page whose most recent access was furthest in past. **k-competitive**. - **FIFO** (First-In-First-Out): evict oldest page (by load time). **k-competitive**. - **LIFO**: not competitive (adversary can keep accessing a page just evicted). - **LFU**: not competitive. - **Optimal offline** (Belady's Algorithm): evict page that will be used furthest in future. Used only for analysis. - Lower bound: No deterministic online paging algorithm can be better than **k-competitive**. - **Randomized**: Marking algorithm is $(2H_k)$ -competitive (where $H_k = 1 + 1/2 + \dots + 1/k$).

Ch 28 – Matrix Operations

Solving Linear Systems: $(Ax = b)$ with nonsingular $(A^{n \times n})$. Avoid direct inversion (numerically unstable). Use **LUP decomposition**: $(PA = LU)$ where (L) unit lower-triangular, (U) upper-triangular, (P) permutation.

LUP-SOLVE:

LUP-SOLVE(L, U, π, b, n)
 1 let x and y be new vectors of length n

```

2 for i = 1 to n          // forward substitution:  $Ly = Pb$ 
3    $y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} \ell_{ij} y_j$ 
4 for i = n downto 1      // back substitution:  $Ux = y$ 
5    $x_i = (y_i - \sum_{j=i+1}^n u_{ij} x_j) / u_{ii}$ 
6 return x

```

Both forward and back substitution: $((n^2))$ time each.

LU Decomposition (without pivoting): - Partition $(A =)$. - Factor: $[A =$

$] - (A' - vw^T/a_{11}) =$ **Schur complement** of (A) with respect to (a_{11}) . Nonsingular if (A) nonsingular. - Recurse on Schur complement. Algorithm LU-DECOMPOSITION uses iteration (not recursion) for efficiency. $((n^3))$. - Fails if any pivot = 0 along diagonal.

LUP Decomposition (with partial pivoting):

```

LUP-DECOMPOSITION(A, n)
1 let  $\pi[1:n]$  be new array
2 for i = 1 to n:  $\pi[i] = i$ 
3 for k = 1 to n
4   find  $k' \geq k$  maximizing  $|a_{k'k}|$ 
5   if  $a_{k'k} == 0$ : error "singular matrix"
6   swap  $\pi[k], \pi[k']$ 
7   swap rows k and  $k'$  of A
8   for i = k+1 to n
9      $a_{ik} = a_{ik} / a_{kk}$           // store L elements
10    for j = k+1 to n             // update Schur
complement
11       $a_{ij} = a_{ij} - a_{ik} a_{kj}$ 

```

- $((n^3))$ time. Pivoting ensures numerical stability by avoiding small divisors.
- Permutation represented compactly as array (π) .

Matrix Inversion: - From LUP decomposition: solve $(AX_i = e_i)$ for each column (X_i) of (A^{-1}) . Each solve $((n^2)) \rightarrow$ total $((n^3))$. - **Equivalence with matrix multiplication:** - Theorem 28.1: $(M(n) = O(I(n)))$. Given invert $(I(n))$, multiply by embedding (AB) in $(3n \ 3n)$ block matrix. - Theorem 28.2: $(I(n) = O(M(n)))$. Given multiply $(M(n))$, invert using divide-and-conquer on Schur complement of (A^{-1}) for SPD (A) . General case: $(A^{-1} = (A^T A)^{-1} A^T)$.

Symmetric Positive-Definite (SPD) Matrices: $(A = A^T)$ and $(x^T A x > 0)$ for all (x) . - Nonsingular (Lemma 28.3). All pivots $> 0 \rightarrow$ LU succeeds without pivoting. - **Cholesky decomposition:** $(A = L L^T)$ where (L) lower-triangular with positive diagonal. $((n^3)/2)$ - half the work of LU. - $(A^T A)$ is always SPD for full-rank (A) .

Least-Squares Approximation: Overdetermined $(Ax = b)$ (more equations than unknowns). Minimize $(|Ax - b|^2)$. Normal

equations: $(A^T A x = A^T b)$. $(A^T A)$ is SPD $\rightarrow$ solve via Cholesky.

Ch 29 – Linear Programming

Standard Form (maximization): [

] $(A^{m \times n})$, $(b^{m \times 1})$, $(c^{1 \times n})$, $(x^{n \times 1})$. Minimization: maximize $(-c^T x)$.

Converting to Standard Form: - Equality $(a^T x = b) \rightarrow (a^T x \leq b)$ and $(-a^T x \leq -b)$. - Unrestricted variable $(x) \rightarrow (x = x^+ - x^-)$ with (x^+, x^-) . - $(x \geq 0) \rightarrow$ substitute $(x' = -x)$.

Terminology: Feasible solution (satisfies all constraints), feasible region (set of all feasible solutions = convex polyhedron = simplex), optimal solution (feasible with best objective), unbounded (feasible but no finite optimum), infeasible (no feasible solution).

Graphical Method (2 variables): - Plot each constraint as a half-plane. Feasible region = intersection. - Objective $(z = c_1 x_1 + c_2 x_2)$ is a family of parallel lines. Increase (z) until line is tangent to feasible region at a vertex (or edge). - Optimal solution always at a **vertex** of feasible region (if solution exists and bounded).

Simplex Algorithm: - Start at a vertex. In each iteration: move along an edge to adjacent vertex with better (or equal) objective value. - **Pivot:** choose entering variable (most positive reduced cost) and leaving variable (minimum ratio test). - Terminates at local maximum $\rightarrow$ global maximum due to convexity. - **Exponential worst-case** (Klee-Minty cube), but runs in $(O(m + n))$ iterations on average in practice.

Polynomial-Time Algorithms: - **Ellipsoid** (Khachiyan 1979): $(O(n^4 L))$. Uses shrinking ellipsoids to contain optimal. First polynomial LP algorithm, slow in practice. - **Interior-Point Methods** (Karmarkar 1984): $(O(n^{3.5} L))$. Move through interior of feasible region (not along edges). Newton steps toward optimal. Competitive with simplex for large problems.

Duality: - **Primal** (max): $(c^T x)$ s.t. $(Ax \leq b, x \geq 0)$. - **Dual** (min): $(b^T y)$ s.t. $(A^T y \leq c, y \geq 0)$. - **Weak Duality:** For any feasible primal (x) and dual (y) : $(c^T x \leq b^T y)$. Proof: $(c^T x + (A^T y - c)^T x = y^T A x + y^T b = b^T y)$. - **Strong Duality:** If primal has optimal (x^*) and dual has optimal (y^*) , then $(c^T x^* = b^T y^*)$ (provided both feasible). - **Complementary Slackness:** At optimality: $(x_j > 0 \rightarrow (A^T y)_j = c_j)$; $(y_i > 0 \rightarrow (A x)_i = b_i)$.

Integer Linear Programming: Variables restricted to integers. NP-hard. Solved via branch-and-bound (LP relaxation + tree search) or cutting planes (Gomory cuts).

Ch 30 – Polynomials and the FFT

Polynomial Representations: - **Coefficient:** $(a = (a_0, a_1, \dots, a_{n-1}))$ for $(A(x) = \sum_{j=0}^{n-1} a_j x^j)$. - **Evaluation via Horner's rule:** $(A(x_0) = (((a_{n-1}x_0 + a_{n-2})x_0 + \dots)x_0 + a_0))$. - **Addition:** $((n))$. **Multiplication (convolution):** naive $((n^2))$. - **Point-Value:** $(\{(x_k, A(x_k))\}_{k=0}^{n-1})$ with distinct (x_k) . - **Addition:** $((n))$ (sum corresponding (y_k)). - **Multiplication:** $((n))$ pointwise multiply, but need $(2n)$ points for degree-bound $(2n)$ product. - **Interpolation:** unique polynomial from (n) pairs (Theorem 30.1, Vandermonde determinant). - **Conversion strategies:** Evaluate (coefficient $\rightarrow$ point-value) takes $((n^2))$ naively, $((n \log n))$ with FFT. Interpolate (point-value $\rightarrow$ coefficient) similarly.

Vandermonde Matrix: $[V(x_0, \dots, x_{n-1}) =$

$\begin{bmatrix} 1 & x_0 & \dots & x_0^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & \dots & x_{n-1}^{n-1} \end{bmatrix}$ $(V = \sum_{0 \leq j < k < n-1} (x_k - x_j))$ iff all (x_k) distinct $\rightarrow$ invertible.

Lagrange Interpolation: $[A(x) = \sum_{k=0}^{n-1} y_k \ell_k(x)]$ $((n^2))$ to compute coefficients.

Fast Multiplication Strategy (Theorem 30.2): 1. **Extend:** Increase degree-bounds to $(2n)$ (append (n) zero coefficients). 2.

Evaluate: Compute DFT at $(2n)$ complex $((2n))$ th roots of unity via FFT $((n \log n))$. 3. **Pointwise multiply:** $(C(x_k) = A(x_k) B(x_k))$ $((n))$. 4. **Interpolate:** Compute inverse DFT via FFT $((n \log n))$.

Complex Roots of Unity: $(\omega_n = e^{2\pi i/n})$ = principal (n) th root of unity. All (n) roots: $(\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1})$. - **Properties:** - $(\omega_n^n = 1)$. (ω_n^k) for $(0 < k < n)$. - $(\sum_{d|k} \omega_n^d = n^k)$ (Cancellation Lemma). - $(\sum_{d|2k} \omega_n^d = n^k)$ (Halving Lemma). - $(\sum_{j=0}^{n-1} (\omega_n^k)^j = 0)$ for $(k \neq 0)$ (Summation Lemma). - $(\omega_n^{n/2} = -1)$.

DFT: $(y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{kj})$ for $(k = 0, \dots, n-1)$.

FFT (Cooley-Tukey, 1965): Divide-and-conquer. - Split coefficients into even and odd indices: $[A^{\{[0]\}}(x) = \sum_{j=0}^{n/2-1} a_{2j} x^j, A^{\{[1]\}}(x) = \sum_{j=0}^{n/2-1} a_{2j+1} x^j]$ - Then $(A(x) = A^{\{[0]\}}(x^2) + x A^{\{[1]\}}(x^2))$. - For $(k = 0, \dots, n/2-1)$: $[y_k = A^{\{[0]\}}(\omega_n^{2k}) + \omega_n^k A^{\{[1]\}}(\omega_n^{2k})]$ $[y_{k+n/2} = A^{\{[0]\}}(\omega_n^{2k}) - \omega_n^k A^{\{[1]\}}(\omega_n^{2k})]$ - Recurrence: $(T(n) = 2T(n/2) + (n))$.

Inverse DFT: Computed by same FFT algorithm using (ω_n^{-1}) instead of (ω_n) , then scaling each result by $(1/n)$: $[a_j = \sum_{k=0}^{n-1} y_k \omega_n^{-kj}]$

FFT Circuit: Butterfly network with (n) stages, each with $(n/2)$ butterfly operations. Each butterfly: one complex multiply, one add, one subtract.

Ch 31 – Number-Theoretic Algorithms

Bit Complexity: Input size measured in bits. For (n) -bit integers: naive multiply/divide = (n^2) bit ops. Faster: Karatsuba $(n^{\log_2 3})$, FFT-based $(n \log n)$.

Key Concepts: - **Division Theorem (31.1):** Unique $(q = a/n)$, $(r = a - nq)$ with $(0 \leq r < n)$. - **Divisibility:** $(d|a \iff a = kd)$ for some (k) . - **gcd:** $((a,b) = \{d: d|a \wedge d|b\})$. $((0,b) = b)$. - **Theorem 31.2:** $((a,b) = \{ax + by > 0: x,y \in \mathbb{Z}\})$. - **Relatively prime:** $((a,b) = 1)$. - **Theorem 31.8 (Unique Prime Factorization):** Every $(a > 1)$ uniquely expressed as $(\prod p_i^{e_i})$.

GCD Recursion Theorem (31.9): $((a,b) = (b, a \bmod b))$ for $(a, b > 0)$.

Euclid's Algorithm:

```
EUCLID(a, b)
1 if b == 0 return a
2 else return EUCLID(b, a mod b)
```

- $(O(b))$ recursive calls. Worst-case: consecutive Fibonacci numbers $(F_{k+1}, F_k) \rightarrow ((k))$ divisions.

Extended Euclid:

```
EXTENDED-EUCLID(a, b)
1 if b == 0 return (a, 1, 0)
2 else (d', x', y') = EXTENDED-EUCLID(b, a mod b)
3     (d, x, y) = (d', y', x' - ⌊a/b⌋ y')
4     return (d, x, y)
```

- Returns $((d, x, y))$ with $(d = (a,b) = ax + by)$.
- Basis for computing modular inverses: if $((a,n) = 1)$, then $(x \bmod n)$ is the inverse of (a) modulo (n) .

Modular Arithmetic: - **Additive group** $((\mathbb{Z}_n, +_n))$: abelian, identity 0 , inverse of (a) is $(n-a)$. Size (n) . - **Multiplicative group** $(\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n : (a,n) = 1\})$. Size $(\phi(n) = n \prod_{p|n} (1-1/p))$. - **Euler's Theorem:** $(a^{\phi(n)} \equiv 1 \pmod n)$ for $((a,n) = 1)$. - **Fermat's Theorem:** $(a^{p-1} \equiv 1 \pmod p)$ for prime (p) , $(a \not\equiv 0 \pmod p)$.

Group Theory: - **Group:** closure, associativity, identity, inverses. Abelian = commutative. - **Subgroup:** nonempty subset closed under operation. - **Lagrange's Theorem (31.15):** Size of subgroup divides size of group. - **Generator:** (a) generates subgroup $(\langle a \rangle = \{a^k : k \in \mathbb{Z}\})$. Order of (a) = $(|\langle a \rangle|)$.

Chinese Remainder Theorem (31.27–31.29): - For pairwise coprime $(n_1, \dots, n_k)$, let $(N = n_i)$. - System $(x \equiv a_i \pmod{n_i})$ has unique solution modulo (N) . - Solution: $(x \equiv a_i N_i y_i \pmod{N})$ where $(N_i = N/n_i)$ and $(y_i \equiv N_i^{-1} \pmod{n_i})$.

Modular Exponentiation (Repeated Squaring):

```
MODULAR-EXPONENTIATION(a, b, n)
1 d = 1
2 let (b_k ... b_0) be binary of b
3 for i = k downto 0
4     d = (d * d) mod n
5     if b_i == 1
6         d = (d * a) mod n
7 return d
```

- $(O(b))$ modular multiplications, $(O(\log b))$ bit operations for (b) -bit numbers.

RSA Public-Key Cryptosystem: 1. Generate random large primes (p, q) (e.g., 2048 bits each in modern practice). 2. $(n = pq)$, $((\phi(n) = (p-1)(q-1)))$. 3. Choose small odd (e) with $((e, \phi(n)) = 1)$. 4. $(d \equiv e^{-1} \pmod{\phi(n)})$. 5. Public key $(P = (e, n))$. Secret key $(S = (d, n))$. 6. Encryption: $(C \equiv M^e \pmod{n})$. Decryption: $(M \equiv C^d \pmod{n})$.

Correctness (Theorem 31.36): $(M^{\phi(n)} \equiv 1 \pmod{n})$ by Fermat's theorem $(\pmod{p})$ and $(\pmod{q})$ + CRT. - **Security:** Breaking RSA $\approx$ factoring (n) . Best known factoring algorithms subexponential but superpolynomial. Key sizes 2048–4096 bits standard.

Primality Testing: - **Prime Number Theorem:** $(\pi(n) \sim n/\ln n)$. Expected $(\ln n)$ trials to find a prime near (n) . - **Trial division:** test divisibility by all primes $(\leq \sqrt{n})$. Exponential in bit-length. - **Pseudoprime test (base 2):** $(2^{n-1} \equiv 1 \pmod{n})$. If $(= 1)$, composite. If $\neq 1$, probably prime. Fails on base-2 pseudoprimes (341, 561, 645, 1105...). - **Carmichael numbers:** Composite (n) where $(a^{n-1} \equiv 1 \pmod{n})$ for all $(a \in \mathbb{Z}_n^*)$. Example: $561 = 3 \cdot 11 \cdot 17$. Rare but infinite. - **Miller-Rabin Randomized Test:** Write $(n-1 = 2^t u)$, (u) odd. Compute $(x_0 = a^u \pmod{n})$. For $(i = 1, \dots, t)$: $(x_i = x_{i-1}^2 \pmod{n})$. If $(x_0 \neq 1)$ or some $(x_i \neq \pm 1 \pmod{n})$ for $(i < t) \rightarrow$ **PROBABLY PRIME**. Else **COMPOSITE**. One-sided error $\leq (1/4)$ per random base (a) .

Ch 32 – String Matching

Problem: Find all shifts (s) with $(0 \leq s < n-m)$ such that $(P[1:m] = T[s+1:s+m])$. Alphabet (Σ) .

Notation: Prefix $(P[:k] = P[1:k])$. Suffix $(w \text{ suffix of } (x))$: (w) suffix of (x) . Overlapping-suffix lemma (32.1): if $(x \text{ suffix of } (y))$ and $(y \text{ suffix of } (z))$, then either $(x \text{ suffix of } (z))$ or $(y \text{ suffix of } (x))$ (or $(x = y)$).

Algorithm Complexity Table:

	Algorithm	Preprocessing	Matching
Naive	$O(nm)$	$O(1)$	$O((n-m+1)m)$
Rabin-Karp	$O(m)$	$O((n-m+1)m)$ worst; $O(n+m)$ expected	$O(1)$

Finite Automaton | ($O(m|l)$) | ((n)) | | Knuth-Morris-Pratt | ((m)) | ((n)) |

Naive String Matching:

NAIVE-STRING-MATCHER(T, P, n, m)

```

1 for s = 0 to n - m
2     if P[1:m] == T[s+1:s+m]
3         print "Pattern occurs with shift" s

```

- Worst case: ($P = a^m$), ($T = a^n$) $\rightarrow ((n-m+1)m) = (n^2)$ when ($m \approx n/2$).
- Ignores information from earlier comparisons.

Rabin-Karp Algorithm: - View characters as digits in radix- (d) ($(d = |l|)$). Pattern value (p). Text window values (t_s). - Rolling hash formula: ($t_{s+1} = (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$) where ($h = d^{m-1} \bmod q$). - On ($p = t_s$) (hit), check explicitly for spurious hit. - Expected ($O(n + m)$) if valid shifts few and (q) large (e.g., ($q \approx m^2$) gives collision probability ($O(1/m)$)).

Finite Automaton: - Build DFA: states ($0, 1, \dots, m$). State (q) = length of longest pattern prefix matching suffix of text read so far. - ($(q, a) = (P[:q]a)$) where ($(x) = \{k : P[:k] = x\}$). - Theorem 32.4: ($(T[:i]) = (T[:i])$) $\rightarrow$ invariant maintained at each step. - Naive construction: ($O(m^3|l)$). Optimized: ($O(m|l)$). Matching: ((n)). - If state (m) reached $\rightarrow$ pattern found at shift ($i-m$).

Knuth-Morris-Pratt (KMP): - Prefix function: ($= \{k < q : P[:k] = P[q-k+1:q]\}$) (longest proper prefix that is also proper suffix). - Computation (self-matching):

COMPUTE-PREFIX-FUNCTION(P, m)

```

1  $\pi[1] = 0, k = 0$ 
2 for q = 2 to m
3     while k > 0 and P[k+1]  $\neq$  P[q]
4         k =  $\pi[k]$ 
5     if P[k+1] == P[q]
6         k = k + 1
7      $\pi[q] = k$ 
8 return  $\pi$ 

```

- Matching:

KMP-MATCHER(T, P, n, m)

```

1  $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P, m)$ 
2 q = 0
3 for i = 1 to n
4     while q > 0 and P[q+1]  $\neq$  T[i]
5         q =  $\pi[q]$ 
6     if P[q+1] == T[i]
7         q = q + 1
8     if q == m

```

```

9         print "Pattern occurs with shift" i - m
10        q =  $\pi[q]$ 

```

- **Prefix-function iteration lemma** (32.5): ($\wedge^*[q] = \{k < q : P[:k] \text{ is a prefix of } P[:q]\}$) – iterating $()$ generates all proper suffixes of $(P[:q])$ that are also prefixes.
- Corollary 32.7: ($=$) where ($E_{q-1} = \{k \wedge^*[q-1] : P[k+1] = P[q]\}$).
- Amortized analysis: while loop total ($O(m)$) (aggregate method). Matching total ($O(n)$).

Ch 33 – Machine-Learning Algorithms

Three Paradigms: Supervised (labeled training data), unsupervised (no labels), reinforcement (environment feedback).

k-Means Clustering (Unsupervised): - Input: set (S) of (n) points in $(\mathbb{R}^d)$, integer (k) . - Objective: $(f(S, C) = \sum_{x \in S} \|x - c_{\text{closest}}\|^2)$ where centers $(C = \{c^{(1)}, \dots, c^{(k)}\})$. - NP-hard to find global optimum.

Lloyd's Procedure: 1. **Initialize:** Pick (k) points from (S) (randomly) as initial centers. Assign all to cluster 1. 2. **Assign:** For each point (x) , find nearest center (break ties arbitrarily but only reassign if strictly closer to new center). 3. **Stop** if no assignments changed. 4. **Recompute centers:** Each center = centroid (mean) of its cluster points. Go to step 2. - **Theorem 33.1:** Centroid uniquely minimizes $(\sum_{x \in S} \|x - c\|^2)$. - **Theorem 33.2:** Nearest-center rule minimizes (f) for fixed centers. - Each non-final iteration strictly decreases (f) . Finite termination (at most (k^n) possible clusterings). Converges to local minimum. - Running time: $(O(T d k n))$ where (T) = number of iterations.

Preprocessing: Feature scaling (min-max to $[0,1]$ or z-score normalization). Otherwise attributes with larger ranges dominate distance.

Applications: Image segmentation, vector quantization (color palette reduction), customer segmentation.

Multiplicative Weights (Weighted Majority): - (n) experts, (T) events. Expert (E_i) predicts $(p_i^{\{t\}} \in \{0,1\})$. Outcome $(o^{\{t\}} \in \{0,1\})$. - Algorithm maintains weights $(w_i^{\{t\}})$, initially 1. - On each event: - Predict using weighted majority: $(p^{\{t\}} = 1)$ if $(\{i: p_i^{\{t\}}=1\} w_i^{\{t\}} > \{i: p_i^{\{t\}}=0\} w_i^{\{t\}})$ else 0. - After outcome: for each wrong expert, multiply weight by $(1-\eta)$ ($0 < \eta < 1$). - **Lemma 33.3:** If one expert always correct: algorithm makes (n) mistakes (majority vote of survivors). - **General bound:** Mistakes $(m^* n + O(\eta^{-1} n))$ where (m^*) = best expert's mistakes. - Regret = (algorithm mistakes) - (best expert mistakes). Grows only logarithmically in (n) .

Gradient Descent (Optimization):

GRADIENT-DESCENT(f , $x^{\{0\}}$, y , T)

1 $\text{sum} = 0$

2 for $t = 0$ to $T-1$

3 $\text{sum} = \text{sum} + x^{\{t\}}$

4 $x^{\{t+1\}} = x^{\{t\}} - \gamma (\nabla f)(x^{\{t\}})$

5 return $x\text{-avg} = \text{sum} / T$

- For convex, differentiable ($f : \mathbb{R}^n \rightarrow \mathbb{R}$):
 - **Lemma 33.6:** $f(y) \geq f(x) + \nabla f(x) \cdot (y - x)$. (Convex function lies above tangent line.)
 - Analysis uses potential $\phi(t) = \|x^{\{t\}} - x^*\|^2 / (2\gamma)$.
 - **Lemma 33.9** (Amortized progress): $\phi(t) = \phi(t-1) - \gamma \|\nabla f(x^{\{t-1\}})\|^2 / 2$ where $\|\nabla f(x^{\{t-1\}})\| \leq R$.
 - Summing over T steps: $\phi(T) \leq \phi(0) - \gamma R^2 T / 2$ where $R = \|\nabla f(x^{\{0\}})\|$.
 - With $\gamma = R/(L)$: error $(RL/2)$. To achieve ϵ : $T = R^2 L^2 / \epsilon^2$.
- **Constrained Gradient Descent:** After each gradient step, project back onto convex body (K) . Lemma 33.10: projection never increases distance to any point in (K) . Same convergence bound applies.
- **Line Search:** Dynamically choose step size by doubling + binary search for sufficient decrease.
- Gradient descent finds **local minimum** of non-convex functions. For convex functions, local = global.

Ch 34 – NP-Completeness

Complexity Classes: Formalized via languages over $(\{0,1\}^*)$. - **P:** $(\{L : \exists k \in \mathbb{N} \text{ s.t. } |y| \leq O(|x|^k) \wedge A(x,y) = 1\})$. Certificate (y) proves membership. - **co-NP:** $(\{L : \exists k \in \mathbb{N} \text{ s.t. } |y| \leq O(|x|^k) \wedge \neg A(x,y) = 1\})$. - **NP-complete:** (L) and $(L' \leq_P L)$ for every (L') . - **NP-hard:** satisfies reduction condition (property 2) but not necessarily in NP.

P vs NP: polynomial-time decider ignores certificate. **P = NP?** – open. Most believe $P \neq NP$.

Polynomial-Time Reductions: $(L_1 \leq_P L_2)$ if (f) polynomial-time computable s.t. $(x \in L_1 \iff f(x) \in L_2)$. - **Lemma 34.3:** If $(L_1 \leq_P L_2)$ and $(L_2 \in P)$ then $(L_1 \in P)$. - To prove (L) NP-complete: show $(L \in NP)$ and known NP-complete $(L' \leq_P L)$. - **Theorem 34.4:** If any NP-complete problem is in P, then $P = NP$.

Circuit-SAT (first NP-complete problem): - Input: boolean combinational circuit of AND, OR, NOT gates (no cycles, single output). - Question: does there exist a truth assignment making output = 1? - **NP** (Lemma 34.5): Certificate = value on every wire. Verify each gate's output computed correctly, final output = 1. $(O(n))$. - **NP-hard** (Lemma 34.6): For any $(L \in NP)$, take verifying algorithm (A) . Encode its computation as a circuit: paste $(T(|x|))$.

$x|)$) copies of the hardware circuit (M) that maps one configuration to next. The initial configuration encodes (x) and certificate (y) ; final configuration has output bit. Circuit satisfiable iff $(y : A(x,y) = 1)$. Reduction polynomial in $(|x|)$.

SAT (Formula Satisfiability): - Variables, literals, $(, ,)$. Clause = OR of literals. CNF = AND of clauses. - SAT is NP-complete (Cook-Levin, 1971).

3-CNF-SAT: Each clause has exactly 3 distinct literals. NP-complete. - Reduction from Circuit-SAT: 1. Create variable for each wire in circuit. 2. For each gate, write CNF constraints capturing gate's truth table (e.g., $(z = x \vee y)$ becomes $((z \vee x) \wedge (z \vee y) \wedge (x \vee y \vee z))$). 3. Add unit clause (output) forcing final output = 1. 4. If any clause has (< 3) literals, pad with new variables; if (> 3) , split using auxiliary variables. - Result: original circuit satisfiable $(\iff)$ 3-CNF formula satisfiable.

CLIQUE: Does (G) have a clique of size (k) ? NP-complete. - Reduction from 3-CNF-SAT with formula (ϕ) having (k) clauses. For each clause $(C_r = (\{r_1\} \vee \{r_2\} \vee \{r_3\}))$, create triple of vertices. Edge between vertices if in different clauses and literals are not contradictory $((v_{\{r_i\}} \wedge v_{\{s_j\}})$ with $(\{r_i\} \wedge \{s_j\})$). (ϕ) satisfiable $(\iff)$ (G) has (k) -clique.

VERTEX-COVER: Does (G) have vertex cover of size (k) ? NP-complete. - Complement of independent set. (G) has vertex cover of size $(k) (\iff)$ (G) has clique of size $(|V| - k)$. Reduction from CLIQUE.

HAM-CYCLE: Does undirected graph have Hamiltonian cycle? NP-complete. - Reduction from VERTEX-COVER using widget/gadget construction to simulate edges and selection.

TSP: Given complete graph with integer edge costs and bound (k) , is there tour of cost (k) ? NP-complete. - Reduction from HAM-CYCLE: cost 1 for edges in original graph, 2 for non-edges (or larger). Tour cost distinguishes.

SUBSET-SUM: Given set of integers and integer (t) , is there subset with sum exactly (t) ? NP-complete. - Reduction from 3-CNF-SAT using base-10 encoding. **Weakly NP-complete** – has $(O(n \cdot t))$ pseudo-polynomial DP algorithm. Not strongly NP-complete.

NP-Completeness Proof Strategy: 1. Show problem is in NP: certificate + polynomial verification. 2. Choose known NP-complete problem to reduce from. 3. Describe polynomial-time construction transforming instances. 4. Prove: original instance YES $(\iff)$ constructed instance YES.

Ch 35 – Approximation Algorithms

Motivation: NP-complete problems are too important to abandon. Three options: (1) Small inputs → exponential OK. (2) Special polynomial-time cases. (3) **Approximation algorithms** → near-optimal in polynomial time.

Approximation Ratio: For minimization, ratio $(\frac{C}{C^*})$ if $(\frac{C}{C^*} \leq \alpha)$ for all inputs of size (n) . For maximization, $(\frac{C^*}{C} \leq \alpha)$. (α) -approximation algorithm achieves this.

PTAS: Family of algorithms $\{(\frac{1}{\epsilon})$ -approximation : $(\epsilon > 0)\}$ with running time polynomial in (n) for each fixed (ϵ) . Running time may blow up as (ϵ) (e.g., $(O(n^{2/\epsilon}))$).

FPTAS: PTAS with running time polynomial in both (n) and $(1/\epsilon)$ (e.g., $(O(n^{2/\epsilon}))$).

Vertex Cover (2-approximation):

APPROX-VERTEX-COVER(G)

```
1  $C = \emptyset$ ,  $E' = E$ 
2 while  $E' \neq \emptyset$ 
3     pick arbitrary edge  $(u,v) \in E'$ 
4      $C = C \cup \{u,v\}$ 
5     remove from  $E'$  all edges incident to  $u$  or  $v$ 
6 return  $C$ 
```

- **Theorem 35.1:** $(|C| \leq 2|A|)$. (A) = set of edges picked = maximal matching. Each edge in (A) has no shared endpoints. Optimal cover must include ≥ 1 endpoint per edge $\rightarrow (|C| \leq 2|A|)$. Algorithm returns $(|C| \leq 2|A|)$. Runs in $(O(V+E))$.

TSP with Triangle Inequality (2-approximation):

APPROX-TSP-TOUR(G, c)

```
1 select root  $r \in V$ 
2 compute MST  $T$  from  $r$  using MST-PRIM
3 let  $H$  be preorder walk of  $T$ 
4 return  $H$ 
```

- **Theorem 35.2:** $(c(H) \leq 2c(T))$. MST weight $\leq$ optimal tour (deleting edge from tour gives spanning tree). Full walk of MST traverses each edge twice $\rightarrow$ cost $(2c(T))$. Triangle inequality lets shortcut repeated vertices $\rightarrow (c(H) \leq 2c(T))$.
- MST-PRIM $((V^2))$ with simple implementation.

General TSP (Non-Approximability): - **Theorem 35.3:** No polynomial-time (ϵ) -approximation for any (ϵ) unless $P=NP$. - Proof: Reduce HAM-CYCLE to TSP. Given (G) , construct complete (G') with $(c(u,v) = 1)$ if $((u,v) \in E)$, else $(c(u,v) = |V|+1)$. If (G) has Hamiltonian cycle $\rightarrow$ TSP tour cost = $(|V|)$. If not $\rightarrow$ any tour uses at least

one non-edge $\rightarrow$ cost $((|V|+1) + (|V|-1) > |V|)$. Algorithm distinguishes cases $\rightarrow$ contradiction.

Set Cover (Greedy ($O(|X|)$)-approximation):

```

GREEDY-SET-COVER( $X, \emptyset$ )
1  $U_0 = X, C = \emptyset, i = 0$ 
2 while  $U_i \neq \emptyset$ 
3   select  $S \subseteq \emptyset$  maximizing  $|S \cap U_i|$ 
4    $U_{i+1} = U_i - S, C = C \cup \{S\}, i = i + 1$ 
5 return  $C$ 

```

- **Theorem 35.4:** $(|C| \leq |C^*| \log |X|)$.
- Analysis: If optimal cover has size (k) , at any step some set in (C^*) covers $(|U_i|/k)$ remaining elements. Greedy picks at least that many. So $(|U_{i+1}| \leq |U_i|(1 - 1/k))$. After (i) steps: $(|U_i| \leq |X|(1-1/k)^i \leq |X| e^{-i/k})$. Solve $(|X| e^{-i/k} < 1) \rightarrow (i > k \log |X|)$. So at most $(k \log |X|)$ iterations.

Randomized MAX-3-CNF SAT ($8/7$ -approximation): - Set each of (n) variables to 1 with probability $1/2$, independently. - For each clause with 3 distinct literals (no variable and its negation together): $\Pr[\text{unsatisfied}] = ((1/2)^3 = 1/8)$. So $E[\text{satisfied per clause}] = 7/8$. - **Theorem 35.5:** Expected ratio $= (m / (7m/8) = 8/7)$. Derandomizable via method of conditional expectations.

Weighted Vertex Cover via LP (2 -approximation):

```

APPROX-MIN-WEIGHT-VC( $G, w$ )
1 compute optimal  $x^*$  for LP relaxation:
2   minimize  $\sum w(v) x(v)$ 
3   subject to  $x(u) + x(v) \geq 1$  for all  $(u,v) \in E$ 
4              $0 \leq x(v) \leq 1$  for all  $v \in V$ 
5 for each  $v \in V$ 
6   if  $x(v) \geq 1/2$ :  $C = C \cup \{v\}$ 
7 return  $C$ 

```

- **Theorem 35.6:** 2 -approximation.
- LP relaxation lower bound: $(z^* = \sum w(v) x(v) = w(C^*))$. Rounding: $(w(C) = \sum_{v: x(v) \geq 1/2} w(v) \leq \sum w(v) x(v) = 2z^*)$. Feasibility: for any edge $((u,v))$, $(x(u) + x(v) \geq 1) \rightarrow$ at least one of $(x(u), x(v) \geq 1/2) \rightarrow$ covered.
- LP can be solved in polynomial time (ellipsoid or interior-point).

Subset Sum FPTAS (Fully Polynomial-Time Approximation Scheme): - Exact DP: $(L_0 = \emptyset, (L_i = (L_{i-1}, L_{i-1} + x_i))$, keep elements (t) . Size grows exponentially. - **Trimming:** Remove elements close together. Parameter $(= 1/2n)$. Keep (z) only if $(y > z(1+\epsilon))$ for previous kept (z) . Factor $(1+\epsilon)$ error per step accumulates to $((1+\epsilon)^n)$ overall.

```

TRIM( $L, \delta$ )
1  $L' = \emptyset, last = y_1$ 

```



```

2 for i = 2 to |L|
3     if  $y_i > \text{last} \cdot (1 + \delta)$ 
4         append  $y_i$  to  $L'$ ,  $\text{last} = y_i$ 
5 return  $L'$ 

```

```

APPROX-SUBSET-SUM( $S, n, t, \epsilon$ )
1  $L_0 = \{0\}$ 
2 for i = 1 to n
3      $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
4      $L_i = \text{TRIM}(L_i, \epsilon/2n)$ 
5     remove from  $L_i$  elements  $> t$ 
6 return  $\max(L_n)$ 

```

- **Theorem 35.7:** FPTAS. Returns (z) with $(y^*/z + \epsilon)$ where $(y^*) =$ optimal subset sum $\leq (t)$.
- Proof: For every $(y \in P_i)$ (exact subset sums), $(z \in L_i)$ with $(y/(1+\epsilon)^i \leq z \leq y)$. Apply to $(y) \rightarrow (z \in L_n)$ with $(z \leq y^*/(1+\epsilon/2n)^n \leq y^*/(1+\epsilon))$. Since $(z \leq y)$, ratio bound holds.
- List size: after trimming, successive elements differ by factor $(\geq 1+\epsilon)$. Size $\leq (\lceil \frac{t}{\epsilon} \rceil + 2 = O(\frac{t}{\epsilon}))$. Running time polynomial in (n) and $(1/\epsilon)$.

Cross-Cutting Topics

Design Paradigms & Meta-Methods

Divide-and-Conquer - Break problem into smaller subproblems, solve recursively, combine results - **Key:** subproblems are independent (unlike DP) - **Examples:** merge sort (split array, sort halves, merge), quicksort (partition, sort sides), Strassen's matrix multiplication, FFT - **Recurrence:** $T(n) = aT(n/b) + f(n)$ - solve via master theorem - **When to use:** problem can be divided into independent subproblems

Dynamic Programming - Solve overlapping subproblems, store results (memoization or bottom-up) - **Requirements:** optimal substructure + overlapping subproblems - **4 steps:** (1) characterize optimal structure, (2) define recurrences, (3) compute values bottom-up, (4) reconstruct solution - **Examples:** rod cutting, matrix-chain multiplication, LCS, optimal BST, Floyd-Warshall - **Cut-and-paste proof:** if optimal solution uses a suboptimal subproblem, cut it out and paste in a better one $\rightarrow$ contradiction

Greedy Algorithms - Make locally optimal choice at each step; never reconsider - **Requirements:** greedy-choice property + optimal substructure - **Proof technique:** exchange argument (replace first non-greedy choice with greedy one without worsening) - **Examples:** activity selection, Huffman codes, offline caching (furthest-in-future), Kruskal/Prim MST, Dijkstra - **DP vs Greedy:** DP considers all choices; greedy commits to one. Not all problems with optimal substructure have greedy-choice property.

Amortized Analysis - Average cost per operation over worst-case sequence - **Three methods**: aggregate (total / n), accounting (charge extra to build credit), potential (Φ function) - **Examples**: dynamic tables (amortized $O(1)$ per insert), binary counter ($O(1)$ per increment), MULTIPOP stack ($O(1)$ per op) - **Key**: no probability – worst-case sequence averaged over operations

Linear Programming Duality - Primal ($\min c^T x$ s.t. $Ax \geq b, x \geq 0$) $\leftrightarrow$ Dual ($\max b^T y$ s.t. $A^T y \leq c, y \geq 0$) - Weak duality: $c^T x \geq b^T y$ for any feasible x, y - Strong duality: optimal values are equal (if both feasible) - **Applications**: max flow = min cut, Hungarian algorithm, approximation algorithms

Online Algorithms - Input arrives over time; make irrevocable decisions without future knowledge - **Competitive ratio**: worst-case ratio of online cost to optimal offline cost - **Examples**: elevator/stairs, ski-rental, move-to-front (2-competitive for search list), LRU (k -competitive for caching)

Parallel Algorithms (Fork-Join) - spawn: create child strand (execute in parallel with parent continuation) - sync: wait for all spawned children to complete - parallel for: parallel loop via recursive binary splitting - **Work T_1** (serial time) + **Span T_∞** (critical path length) - Greedy scheduler achieves $T_1 \leq T_1/P + T_\infty$

Probabilistic Analysis & Randomized Algorithms - Indicator random variables: $E[X] = \Pr\{\text{event}\}$ where $X = 1$ if event occurs - Linearity of expectation: $E[\sum X_i] = \sum E[X_i]$ – holds even for dependent events - **Randomized algorithm**: uses randomness to achieve good expected performance (e.g., randomized quicksort, randomized select) - **Universal hashing**: random family with bounded collision probability

Proof & Argument Patterns

Loop Invariant – used for iterative algorithm correctness - **Initialization**: invariant holds before first iteration - **Maintenance**: if it holds before an iteration, it holds after - **Termination**: when loop ends, invariant gives useful property - *Examples*: insertion sort (subarray sorted), Bellman-Ford (after i passes, distances use $\leq i$ edges)

Mathematical Induction – used for recursive algorithms and recurrences - Base case + inductive hypothesis $\rightarrow$ inductive step - *Example*: proving merge sort correctly sorts (assume sorted halves, merge produces sorted output)

Cut-and-Paste (Optimal Substructure) – used for DP and greedy - Assume optimal solution contains a suboptimal subproblem $\rightarrow$ replace with optimal $\rightarrow$ better solution $\rightarrow$ contradiction - *Example*: shortest path subpaths are shortest (Lemma 22.1)

Exchange Argument – used for greedy correctness - Start with optimal solution; transform it toward greedy choice without worsening - *Examples*: activity selection (replace first activity with earliest-finish), Huffman codes (swap to make lowest-frequency chars deepest)

Contradiction – used across all areas - Assume false, derive contradiction - *Examples*: Dijkstra correctness (extracted vertex with wrong distance → earlier vertex must have smaller distance), Gale-Shapley stability

Max-Flow Min-Cut Proof Structure - (1) f is max → (2) no augmenting path: if path existed, could increase flow - (2) no path → (3) $|f| = c(S,T)$: let S = vertices reachable from s in residual; all crossing edges saturated - (3) $|f| = c(S,T)$ → (1) f is max: any flow $\leq$ any cut capacity

NP-Completeness Reductions - To show L is NP-complete: (1) $L \in \text{NP}$ (certificate verifiable in poly time), (2) $L' \leq_p L$ for some known NP-complete L' - Reduction: transform any instance of L' into instance of L such that L' has solution iff L does - *Common reductions*: Circuit-SAT → SAT → 3-CNF-SAT → CLIQUE → VERTEX-COVER → HAM-CYCLE → TSP → SUBSET-SUM

Amortized Analysis via Potential Method - Define $\Phi(D_i) = 0$, $\Phi(D_i) \geq 0$ for all i - $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i+1})$ - $\sum \hat{c}_i = \sum c_i + \Phi(D_1) - \Phi(D_n) \geq \sum c_i$

People & Dates

Person	Contribution	Context
Thomas H. Cormen	Co-author	CLRS textbook
Charles E. Leiserson	Co-author	CLRS textbook
Ronald L. Rivest	Co-author, RSA co-inventor	CLRS textbook, cryptography
Clifford Stein	Co-author	CLRS textbook
John von Neumann	Merge sort (1945)	Sorting
C. A. R. Hoare	Quicksort (1962)	Sorting
Volker Strassen	Strassen's matrix multiplication (1969)	Divide-and-conquer; $O(n^{2.8074})$
Don Knuth	Analysis of algorithms, TAoCP	Asymptotic notation popularization
James Cooley & John Tukey	FFT (1965)	Polynomial multiplication, $O(n \lg n)$
Edsger Dijkstra	Dijkstra's algorithm (1959), semaphores	Shortest paths, concurrency
Richard Bellman	Bellman-Ford (1958), DP	Shortest paths, dynamic programming

Person	Contribution	Context
L. R. Ford Jr. & D. R. Fulkerson	Ford-Fulkerson max flow	Network flow
Jack Edmonds & Richard Karp	Edmonds-Karp algorithm (1972)	Max flow, $O(VE^2)$
David Gale & Lloyd Shapley	Gale-Shapley stable marriage (1962)	Matching; Shapley won 2012 Nobel Prize
Harold Kuhn & James Munkres	Hungarian algorithm (1955/1957)	Assignment problem, $O(n^3)$
Stephen Cook	NP-completeness (1971)	Theory of NP-completeness; Cook-Levin theorem
Richard Karp	21 NP-complete problems (1972)	Reductions
Robert Tarjan	Union-find analysis, DFS-based SCC	Disjoint sets (inverse Ackermann), graph algorithms
John Hopcroft & Robert Tarjan	Planarity testing, biconnectivity	Graph algorithms
Leo Guibas & Robert Sedgwick	Red-black trees (1978)	Balanced BST
Rudolf Bayer	B-trees (1972)	Balanced search trees
Ron Rivest, Adi Shamir, Leonard Adleman	RSA (1977)	Public-key cryptography
Michael Rabin & Gary Miller	Miller-Rabin primality test	Randomized primality testing
Richard Karp & Michael Rabin	Rabin-Karp string matching (1987)	Rolling hash
James Morris, Donald Knuth, Vaughan Pratt	KMP string matching (1977)	Prefix function
Paul Erdős	Erdős-Rényi random graphs	Probabilistic method

Probability & Statistics Foundation

Basic Counting - Permutations: $P(n,k) = n!/(n-k)!$ - Combinations: $C(n,k) = n!/(k!(n-k)!)$ - Binomial theorem: $(x+y)^n = \sum C(n,k) x^k y^{n-k}$ - Balls-and-bins: expected load per bin = n/m when throwing n balls into m bins

Probability Axioms - $\Pr\{A\} \geq 0$, $\Pr\{S\} = 1$, $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\} - \Pr\{A \cap B\}$ - Conditional: $\Pr\{A|B\} = \Pr\{A \cap B\} / \Pr\{B\}$ - Bayes: $\Pr\{A|B\} = \Pr\{B|A\} \cdot \Pr\{A\} / \Pr\{B\}$ - Independence: $\Pr\{A \cap B\} = \Pr\{A\} \cdot \Pr\{B\}$

Indicator Random Variables - $X_i = 1$ if event A occurs, 0 otherwise - $E[X_i] = \Pr\{A\}$ - **Linearity of expectation:** $E[\sum X_i] = \sum E[X_i]$ (no independence needed!)

Discrete Distributions - **Geometric:** $\Pr\{X = k\} = (1-p)^{k-1}p$, $E[X] = 1/p$. Memoryless: $\Pr\{X > n+k \mid X > n\} = \Pr\{X > k\}$ - **Binomial:** $\Pr\{X = k\} = C(n, k) p^k (1-p)^{n-k}$, $E[X] = np$, $\text{Var}[X] = np(1-p)$ - **Poisson** (limit of binomial): $\Pr\{X = k\} = e^{-\lambda} \lambda^k / k!$, $E[X] = \lambda$

Tail Bounds - Markov: $\Pr\{X \geq a\} \leq E[X]/a$ (for nonnegative X) - Chebyshev: $\Pr\{|X-\mu| \geq t\} \leq \text{Var}[X]/t^2$ - Chernoff (sum of independent Bernoulli): $\Pr\{\sum X_i > (1+\delta)\mu\} \leq (e^{\delta/(1+\delta)})^{1+\delta} \mu$; $\Pr\{\sum X_i < (1-\delta)\mu\} \leq e^{-\mu\delta^2/2}$ - Union bound: $\Pr\{\bigcup A_i\} \leq \sum \Pr\{A_i\}$

Geometric Distribution (Birthday Paradox) - Expected number of trials for first collision when sampling from n items: $\approx \sqrt{(\pi n)/2}$ $\approx 1.253\sqrt{n}$ - For n=365: ≈ 22.5 people before a shared birthday

Coupon Collector - Expected number of trials to collect all n coupons: $n \cdot H_n = n \ln n + O(n)$ - Each new coupon takes expected $n/(n-i+1)$ trials after collecting i distinct coupons

Mnemonics & Memory Aids

Big-O Ordering (fastest $\rightarrow$ slowest): " $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$ " - Mnemonic: "Constant Old Ladies Knit Nicely; Sometimes Nice Knitters Prefer Zip-fasteners"

Master Theorem Cases: "Pick on Big F" - compare $f(n)$ with $n^{\log_b a}$ - Case 1: $f(n)$ is polynomially smaller $\rightarrow T(n) = \Theta(n^{\log_b a})$ - Case 2: $f(n) \approx n^{\log_b a} \lg^k n \rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$ - Case 3: $f(n)$ is polynomially larger and regular $\rightarrow T(n) = \Theta(f(n))$

Red-Black Tree Properties (5 rules): "Root is Black, Leaves are Black, No two Reds in a row, Same black-count on every path" - Mnemonic: "Red-Black: Root Black, Leaf Black, No Red-Red, Black-height equal"

Sorting Comparison Key: "Insertion is incremental n^2 / Merge is divide-conquer $n \log n$ / Heap uses heap $n \log n$ / Quick has pivot n^2 worst / Counting needs k range $n+k$ / Radix sorts digit-by-digit d(n+k) / Bucket distributes into bins n"

Greedy vs DP: "Greedy commits once, DP considers all" - Greedy: one subproblem remains after choice. DP: two or more subproblems.

Shortest Path Algorithm Selection: "Dijkstra for non-negative, Bellman for negative, DAG for fast, Floyd for all-pairs" -

Mnemonic: "Dijkstra Does Non-negative, Bellman Battles Bad edges, DAG Delivers Fastest, Floyd Finds All pairs"

Potential Method Amortized: " $\hat{c} = c + \Delta\Phi$ "

Ethics & Professional Practice

- **RSA and cryptography:** dual-use – enables secure communication but also criminal concealment; backdoor debates
 - **Algorithmic bias:** machine learning algorithms (Ch 33) can perpetuate discrimination if training data is biased; fairness constraints in clustering
 - **NP-completeness implications:** if $P = NP$, many hard problems become tractable, breaking current cryptographic systems (including RSA)
 - **Approximation algorithms:** used when exact solutions are too slow; quality guarantees are important for correctness-critical applications
 - **Online algorithms and privacy:** caching/search-list data can reveal user behavior patterns
 - **Parallel computing and determinacy races:** nondeterminism can cause hard-to-reproduce bugs in safety-critical systems
-

Exam Questions by Type

MCQ

1. **Q:** What is the worst-case running time of quicksort? **A:** $\Theta(n^2)$
Distractor: $\Theta(n \lg n)$ (this is average-case)
2. **Q:** Which property is NOT required for dynamic programming? **A:** Greedy-choice property **Distractor:** Optimal substructure (required), Overlapping subproblems (required)
3. **Q:** For $T(n) = 2T(n/2) + n$, the solution is: **A:** $\Theta(n \lg n)$ – master theorem Case 2 **Distractor:** $\Theta(n)$, $\Theta(n^2)$
4. **Q:** Which sorting algorithm is NOT a comparison sort? **A:** Counting sort **Distractor:** Heapsort, Merge sort, Quicksort
5. **Q:** What does the max-flow min-cut theorem state? **A:** Max flow value = min cut capacity **Distractor:** Max flow = $2 \times$ min cut; min cut = max flow / 2
6. **Q:** Which algorithm can handle negative-weight edges? **A:** Bellman-Ford **Distractor:** Dijkstra (requires nonnegative)
7. **Q:** What is the span of parallel Fibonacci P-FIB(n)? **A:** $\Theta(n)$
Distractor: $\Theta(\varphi^n)$ (that's the work), $\Theta(\lg n)$

8. **Q:** A problem is NP-complete if: **A:** It is in NP and every NP problem reduces to it in polynomial time **Distractor:** It is in NP only
9. **Q:** The amortized cost of n insertions into a dynamic table that doubles when full is: **A:** $O(1)$ per insertion (total $O(n)$) **Distractor:** $O(n)$ per insertion, $O(\lg n)$ per insertion
10. **Q:** In a red-black tree, what is the maximum height in terms of n ? **A:** $\leq 2 \lg(n+1)$ **Distractor:** $\leq \lg n$ (too tight, that's for AVL trees), $\leq n$
11. **Q:** Which of the following is a linear-time sorting algorithm? **A:** Counting sort (when $k = O(n)$) **Distractor:** Heapsort ($O(n \lg n)$)
12. **Q:** The Floyd-Warshall algorithm has time complexity: **A:** $O(V^3)$ **Distractor:** $O(V^2)$, $O(VE)$
13. **Q:** Which data structure gives $O(m \alpha(n))$ time for a sequence of m disjoint-set operations? **A:** Disjoint-set forest with union by rank and path compression **Distractor:** Linked-list representation ($O(m + n \lg n)$)
14. **Q:** In the Gale-Shapley stable matching algorithm with men proposing, what property holds for women? **A:** Each woman gets her worst valid partner (female-pessimal) **Distractor:** Each woman gets her best valid partner
15. **Q:** What is the prefix function $\pi[q]$ in KMP? **A:** Length of longest proper prefix of $P[:q]$ that is also a proper suffix **Distractor:** Length of longest suffix; position of first mismatch

Short Answer

1. **Q:** State the three cases of the Master Theorem. **Rubric:** $T(n) = aT(n/b) + f(n)$. Let $c = \log_b a$. Case 1: $f(n) = O(n^{c-\epsilon}) \rightarrow T(n) = \Theta(n^c)$. Case 2: $f(n) = \Theta(n^c \lg^k n) \rightarrow T(n) = \Theta(n^c \lg^{k+1} n)$. Case 3: $f(n) = \Omega(n^{c+\epsilon})$ and $af(n/b) \leq cf(n) \rightarrow T(n) = \Theta(f(n))$. Must mention regularity condition for Case 3.
2. **Q:** Describe the three methods of amortized analysis. **Rubric:** Aggregate: total cost $T(n)$ for n ops, amortized = $T(n)/n$. Accounting: assign amortized cost, maintain credit invariant. Potential: $\Phi(D_i)$ with $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$. Example each.
3. **Q:** Prove that any comparison sort requires $\Omega(n \lg n)$ comparisons. **Rubric:** Decision tree model. $n!$ leaves reachable. Height $h \rightarrow 2^h \geq n!$. Stirling: $n! \sim \sqrt{2\pi n}(n/e)^n \rightarrow \lg(n!) = n \lg n - n \lg e + O(\lg n)$. So $h \geq \lg(n!) = \Omega(n \lg n)$.

4. **Q:** Explain the difference between P, NP, NP-complete, and NP-hard. **Rubric:** P = polynomial-time decidable. NP = polynomial-time verifiable. NPC = $NP \cap NP\text{-hard}$. NP-hard = all NP problems reduce to it. $P \subseteq NP$, $P = NP$? open.
5. **Q:** How does the Ford-Fulkerson method find maximum flow? **Rubric:** Start with zero flow. While augmenting path exists in residual network: find path, compute bottleneck capacity, augment flow. Max-flow min-cut theorem: no augmenting path $\Rightarrow$ flow is maximum. Edmonds-Karp: use BFS $\rightarrow O(VE^2)$.
6. **Q:** What makes a greedy algorithm work vs fail? **Rubric:** Must prove greedy-choice property (locally optimal choice leads to global optimum) + optimal substructure. Exchange argument: start with optimal, swap in greedy choices. Counterexample: 0-1 knapsack (greedy fails), optimal substructure alone insufficient.
7. **Q:** Describe Dijkstra's algorithm and its limitations. **Rubric:** Maintains distance estimates. Extract min from priority queue, relax outgoing edges. Correct for nonnegative weights (proof by contradiction using convergence property). Fails with negative weights because extracted vertex's distance might later decrease.

Trace / Apply

1. **Insertion Sort Trace:** Input $A = [5, 2, 4, 6, 1, 3]$. Show array after each iteration. **Answer:** $i=2$: $[2, 5, 4, 6, 1, 3]$; $i=3$: $[2, 4, 5, 6, 1, 3]$; $i=4$: $[2, 4, 5, 6, 1, 3]$; $i=5$: $[1, 2, 4, 5, 6, 3]$; Done: $[1, 2, 3, 4, 5, 6]$.
2. **Merge Sort Trace:** Input $A = [3, 41, 52, 26, 38, 57, 9, 49]$. Show tree. **Answer:** Split $[3, 41, 52, 26][38, 57, 9, 49] \rightarrow [3, 41][52, 26][38, 57][9, 49] \rightarrow$ singles. Merge: $[3, 41][26, 52][38, 57][9, 49] \rightarrow [3, 26, 41, 52][9, 38, 49, 57] \rightarrow [3, 9, 26, 38, 41, 49, 52, 57]$.
3. **Dijkstra Trace:** Graph with $s \rightarrow a(4)$, $s \rightarrow b(2)$, $a \rightarrow b(1)$, $a \rightarrow c(5)$, $b \rightarrow c(8)$, $b \rightarrow t(10)$, $c \rightarrow t(6)$. Find all distances from s . **Answer:** $d[s]=0$, $d[b]=2$ (via s), $d[a]=3$ (via $s \rightarrow b \rightarrow a$), $d[c]=8$ (via $s \rightarrow b \rightarrow a \rightarrow c$), $d[t]=12$ (via $s \rightarrow b \rightarrow t$). Extract order: $s(0)$, $b(2)$, $a(3)$, $c(8)$, $t(12)$.
4. **B-Tree Insertion:** $t=3$, insert F S Q K C L H T V W M R N P A B X Y D Z E. Show root splits. **Answer:** Root grows to $[F, S]$; after Q $\rightarrow$ split into $[F][Q, S]$; add K C L $\rightarrow [C, F, K, Q, S]$; add H $\rightarrow [C, F, H, K, Q, S]$; add T V W $\rightarrow$ root $[Q]$, children $[C, F, H, K]$ and $[S, T, V, W]$; add M R N P $\rightarrow$ split right; etc.
5. **KMP Prefix Function:** Compute π for pattern "ababaca". **Answer:** $\pi[1]=0$; $\pi[2]=0$; $\pi[3]=1$ (a prefix); $\pi[4]=2$ (ab); $\pi[5]=3$ (aba); $\pi[6]=1$ (a); $\pi[7]=1$ (a). Explanation: for $q=5$, "ababa" has proper prefix "aba" = proper suffix "aba" $\rightarrow \pi=3$.

6. **Huffman Coding:** Frequencies A(0.22), B(0.18), C(0.20), D(0.16), E(0.24). Build tree. **Answer:** Merge D+B=0.34; merge C+A=0.42; merge 0.34+E=0.58; merge 0.42+0.58=1.00. Codes: A=11, B=010, C=10, D=011, E=00. Cost = $0.22 \cdot 2 + 0.18 \cdot 3 + 0.20 \cdot 2 + 0.16 \cdot 3 + 0.24 \cdot 2 = 2.26$ bits/symbol.

Diagram Label

1. **Red-Black Insertion Case 3:** z is red, uncle black, z is left child. **Label:** Before: parent red, grandparent red, uncle black, z red. Fix: recolor parent black, grandparent red, right-rotate at grandparent. After: no violations.
2. **Recursion Tree:** $T(n) = 2T(n/2) + n$. **Label:** Root cost n. Level 1: 2 nodes each $n/2 = n$ total. Level $\lg n$: n nodes each 1 = n total. Total: $n(\lg n + 1) = \Theta(n \lg n)$.
3. **Residual Network:** After flow f in network, residual G_f has forward edges with $c_f = c - f$ and backward edges with $c_f = f$. **Label:** Augmenting path in G_f from s to t increases flow. Min cut = vertices reachable from s in G_f when flow is max.

Essay / Long-Form

1. **Q:** Prove the cut property for minimum spanning trees. Explain how Kruskal's and Prim's algorithms both rely on it. **Key points:** Let $A \in E$ in some MST. Cut $(S, V-S)$ respected by A (no edge of A crosses). Light edge (u,v) crossing cut: minimum weight. Construct MST T containing A; if (u,v) not in T, swap with heavier edge on unique path in T crossing cut $\rightarrow$ lighter MST, contradiction. Kruskal: cut = component vs rest; Prim: cut = tree vs unprocessed.
2. **Q:** Compare dynamic programming and divide-and-conquer. When is each appropriate? **Key points:** Both split into subproblems. DP: overlapping subproblems (same subproblem solved multiple times) + optimal substructure. D&C: independent subproblems. DP stores results (memoization/tables) $\rightarrow$ avoids recomputation. D&C merges results. Examples: merge sort (D&C), matrix-chain (DP). DP can solve optimization where D&C cannot.
3. **Q:** Describe the Ford-Fulkerson method for maximum flow. Prove the max-flow min-cut theorem. **Key points:** Residual network G_f , augmenting path, bottleneck capacity. Theorem: equivalent conditions – (1) f is max, (2) G_f has no augmenting path, (3) $|f| = c(S,T)$. Proof: (1) $\Rightarrow$ (2) by contradiction; (2) $\Rightarrow$ (3) S = vertices reachable in G_f , count flow across cut; (3) $\Rightarrow$ (1) any flow $\leq$ any cut capacity. Edmonds-Karp: BFS for shortest path $\rightarrow O(VE^2)$.
4. **Q:** What is NP-completeness? Give the reduction from 3-CNF-SAT to CLIQUE. **Key points:** NP = verifiable in polynomial time.

NP-complete = in NP + every NP problem reduces to it. Cook-Levin: Circuit-SAT is first. Reduction 3-CNF-SAT $\rightarrow$ CLIQUE: formula with k clauses $\rightarrow$ graph with k triples of vertices. Edge between vertices in different clauses if literals are not contradictory. Formula satisfiable $\iff$ k -clique exists.

5. **Q:** Analyze the expected running time of randomized quicksort. Why is it $\Theta(n \lg n)$ even though worst case is $\Theta(n^2)$? **Key points:** Two elements $z_i < z_j$ compared iff first pivot chosen from $\{z_i \dots z_j\}$ is z_i or $z_j \rightarrow \Pr = 2/(j-i+1)$. Total expected comparisons = $\sum_i \sum_j \frac{2}{(j-i+1)} < 2n \cdot H_n = O(n \lg n)$. Expected time dominates worst-case inputs because probability of consistently bad partitions is negligible.

Additional Worked Examples

Example: Floyd-Warshall on a 3-vertex graph

Graph: vertices 1,2,3 with weights: $w(1,2)=3$, $w(2,1)=\infty$, $w(1,3)=8$, $w(3,1)=\infty$, $w(2,3)=\infty$, $w(3,2)=1$, $w(i,i)=0$.

$D^{\{(0)\}} = [[0, 3, 8], [\infty, 0, \infty], [\infty, 1, 0]]$

$k=1$ (use vertex 1): $D^{\{(1)\}}[2,3] = \min(D^{\{(0)\}}[2,3], D^{\{(0)\}}[2,1] + D^{\{(0)\}}[1,3]) = \min(\infty, \infty+8) = \infty$. No changes.

$k=2$ (use vertex 2): $D^{\{(1)\}}[1,3] = \min(8, 3+\infty) = 8$. No changes.

$k=3$ (use vertex 3): $D^{\{(2)\}}[1,2] = \min(3, 8+1) = 3$. $D^{\{(2)\}}[2,1] = \min(\infty, \infty+\infty) = \infty$. $D^{\{(2)\}}[2,3] = \min(\infty, \infty+0) = \infty$.

Final $D = [[0, 3, 8], [\infty, 0, \infty], [\infty, 1, 0]]$. Shortest path 1 $\rightarrow$ 2 is direct (3), 1 $\rightarrow$ 3 is direct (8), 3 $\rightarrow$ 2 is direct (1).

Example: Bellman-Ford on graph with negative edge

Vertices s, a, b with edges: $s \rightarrow a(4)$, $s \rightarrow b(2)$, $a \rightarrow b(-3)$. Run Bellman-Ford from s .

Initialize: $d[s]=0$, $d[a]=\infty$, $d[b]=\infty$.

Pass 1: relax $s \rightarrow a \rightarrow d[a]=4$. relax $s \rightarrow b \rightarrow d[b]=2$. relax $a \rightarrow b \rightarrow d[b]=\min(2, 4+(-3))=1$.

Pass 2: relax $s \rightarrow a \rightarrow d[a]=\min(4, 0+4)=4$. relax $s \rightarrow b \rightarrow d[b]=\min(1, 0+2)=1$. relax $a \rightarrow b \rightarrow d[b]=\min(1, 4+(-3))=1$. No change $\rightarrow$ converge.

Final: $\delta(s,a)=4$, $\delta(s,b)=1$. Path $s \rightarrow a \rightarrow b$ is shorter than $s \rightarrow b$.

Example: OS-SELECT on order-statistic tree

Tree (from Fig 17.1): root=26(size=17), left child=17(size=7), right child=41(size=9).

OS-SELECT(root, 17): $r = 17.\text{left.size} + 1 = 7 + 1 = 8$. $i = 17 > r = 8 \rightarrow$ recurse right with $i = 17 - 8 = 9$. Node 41: $\text{left.size} = 5$. $r = 5 + 1 = 6$. $i = 9 > r = 6 \rightarrow$ recurse right with $i = 9 - 6 = 3$. Node 38: $\text{left.size} = 1$. $r = 1 + 1 = 2$. $i = 3 > r = 2 \rightarrow$ recurse right with $i = 3 - 2 = 1$. Node 47: $\text{left.size} = 0$. $r = 0 + 1 = 1$. $i = 1 == r \rightarrow$ return 47.

So OS-SELECT(root, 17) returns node with key 47.

Example: BFS on sample graph

Graph: $V = \{s, a, b, c, d\}$, edges: $s \rightarrow a$, $s \rightarrow b$, $a \rightarrow c$, $b \rightarrow c$, $b \rightarrow d$, $c \rightarrow d$.

BFS from s: queue=[s]. Dequeue s: discover $a(d=1, \pi=s)$, $b(d=1, \pi=s)$. Queue=[a,b]. Dequeue a: discover $c(d=2, \pi=a)$. Queue=[b,c]. Dequeue b: discover $d(d=2, \pi=b)$. Queue=[c,d]. Dequeue c: (already black, no new). Queue=[d]. Dequeue d: done.

Distances: $\delta(s)=0$, $\delta(a)=1$, $\delta(b)=1$, $\delta(c)=2$, $\delta(d)=2$. BFS tree: $s \rightarrow a \rightarrow c$, $s \rightarrow b \rightarrow d$.

Example: DFS edge classification

Same graph: DFS order: $s(d=1)$, $a(d=2)$, $c(d=3)$, return to a, $b(d=4)$, $d(d=5)$, finish. Edges: $s \rightarrow a$ (tree), $a \rightarrow c$ (tree), $a \rightarrow b$ (forward: $a.d=2, b.d=4$), $c \rightarrow b$ (cross: $c.f < b.d$), $b \rightarrow d$ (tree), $d \rightarrow c$ (back: c is gray when d visited).

Big-O Complexity Reference

Function	Name	Example Algorithm
$O(1)$	Constant	Hash table lookup (average), array access
$O(\lg n)$	Logarithmic	Binary search, HEAP operations
$O(n)$	Linear	Linear search, COUNTING-SORT ($k=O(n)$)
$O(n \lg n)$	Linearithmic	Merge sort, heapsort, quicksort (expected)
$O(n^2)$	Quadratic	Insertion sort (worst), naive matrix multiply
$O(n^3)$	Cubic	Floyd-Warshall, matrix-chain (DP)
$O(2^n)$	Exponential	Subset enumeration, naive Fibonacci
$O(n!)$	Factorial	Naive traveling-salesperson (all permutations)

Self-Test Fill-in Templates

Template 1: Master Theorem $T(n) = \underline{\hspace{1cm}} T(n/\underline{\hspace{1cm}}) + \underline{\hspace{1cm}}$ - a
 $= \underline{\hspace{1cm}}$, $b = \underline{\hspace{1cm}}$, $\log_b a = \underline{\hspace{1cm}}$ - **$f(n)$** = - Case $\underline{\hspace{1cm}}$: $f(n) = O(n^{\{\log_b a - \epsilon\}}) \rightarrow T(n) = \theta(\underline{\hspace{1cm}})$ Compare: yes/no, $f(n)$ is polynomially $\underline{\hspace{1cm}}$
 - Case $\underline{\hspace{1cm}}$: $f(n) = \theta(n^{\{\log_b a\}} \lg^k n) \rightarrow T(n) = \theta(\underline{\hspace{1cm}})$
 - Case $\underline{\hspace{1cm}}$: $f(n) = \Omega(n^{\{\log_b a + \epsilon\}})$ AND $a \cdot f(n/b) \leq c \cdot f(n)$ ($\underline{\hspace{1cm}}$ condition) $\rightarrow T(n) = \theta(\underline{\hspace{1cm}})$

Template 2: Loop Invariant for Insertion Sort At the start of each iteration of the for loop (index i), the subarray $A[\text{____}:\text{____}]$ consists of the elements originally in $A[\text{____}:\text{____}]$ but in ____ order. - Initialization: Before first iteration $i=$, subarray $A[\text{____}:\text{____}] = A[1:] = A[1]$ which is trivially sorted. - Maintenance: If invariant holds before iteration i , then after the inner while loop shifts elements $> A[i]$ right and inserts $A[i]$ into position, $A[\text{____}:\text{____}]$ is sorted. Next iteration increments i to $_$, restoring invariant. - Termination: When the loop terminates, $i = _$, so $A[\text{____}:\text{____}]$ is sorted = $A[1:n]$ → entire array sorted.

Template 3: BFS Queue Property During BFS, the queue Q contains vertices with distances $d \in \{ _, _ \}$ for some integer k . - Initially: $Q = [\text{____}]$ with $d = _$. - When a vertex finishes (color = $_$), its neighbors are examined. WHITE neighbors get $d = u.d + _ = _$. - Queue always has at most $_$ distinct distance values.

Template 4: Red-Black Tree Insertion Cases After BST insertion of RED node z : - Case 1: Uncle y is $_$ → recolor: $z.p = _$, $y = _$, $z.p.p = _$; then set $z = _$. - Case 2: Uncle y is $_$ AND z is $_$ child → LEFT-ROTATE($z.p$) → becomes Case 3. - Case 3: Uncle y is $_$ AND z is $_$ child → recolor: $z.p = _$, $z.p.p = _$; RIGHT-ROTATE($z.p.p$); done.

Template 5: Dijkstra's Algorithm Correctness Proof Let u be the first vertex extracted from Q with $u.d > \delta(\text{____}, _)$. **Let $p = s \rightarrow \dots \rightarrow x \rightarrow y \rightarrow \dots \rightarrow u$ be the true shortest path to u , where $x \in S$ (has final distance) and $y \notin S$.** Since $x \in S$, $x.d = _$ (by induction). Edge (x,y) was relaxed when x was processed → $y.d = _$ (convergence property). Since y is on the shortest path to u : $\delta(s,y) \leq \delta(s,u)$, so $y.d \leq \delta(s,y) \leq \delta(s,u) \leq u.d$ (by assumption). But u was extracted as $_$ from Q , so $u.d \leq y.d$. Contradiction. Thus $u.d = _$ for the first vertex extracted with incorrect distance → by induction, all extracted vertices have correct distances.

Template 6: Max-Flow Min-Cut Equivalence Three equivalent statements: 1. f is a $_$ flow. 2. Residual network G_f has no $_$ path from s to t . 3. $|f| = c(\text{____}, _)$ for some cut (S,T) .

Proof structure: (1) $\Leftrightarrow$ (2): If augmenting path existed, could $_$ flow → f not max. $\Leftrightarrow$ (2) $\Leftrightarrow$ (3): Let $S =$ vertices $_$ from s in G_f . Since $t \notin S$, (S,T) is a cut. All edges from S to T have $c_f = 0$ → they are $_$ ($f(u,v) = c(u,v)$). All edges from T to S have $f(v,u) = 0$. So $|f| = f(S,T) = _$. $\Leftrightarrow$ (3) $\Leftrightarrow$ (1): For any flow f' , $|f'| \leq c(S,T) = |f|$. So f is max. $\square$

Quick Reference: Algorithm Running Times

Algorithm	Worst-Case	Average/ Expected	Space	Stable
Linear search	$\Theta(n)$	$\Theta(n)$	$O(1)$	—
Binary search	$\Theta(\lg n)$	$\Theta(\lg n)$	$O(1)$	—
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n)$	Yes
Heapsort	$O(n \lg n)$	$O(n \lg n)$	$O(1)$	No
Quicksort	$\Theta(n^2)$	$\Theta(n \lg n)$	$O(\lg n)$	No
Counting sort	$\Theta(k+n)$	$\Theta(k+n)$	$\Theta(k)$	Yes
Radix sort	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(n)$	Yes
Bucket sort	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$	Yes
RANDOMIZED-SELECT	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	—
SELECT (deterministic)	$\Theta(n)$	$\Theta(n)$	$O(\lg n)$	—
BFS	$O(V+E)$	$O(V+E)$	$O(V)$	—
DFS	$O(V+E)$	$O(V+E)$	$O(V)$	—
Topological sort	$O(V+E)$	$O(V+E)$	$O(V)$	—
SCC (Kosaraju)	$O(V+E)$	$O(V+E)$	$O(V)$	—
Kruskal's MST	$O(E \lg V)$	$O(E \lg V)$	$O(V)$	—
Prim's MST	$O(E \lg V)$	$O(E \lg V)$	$O(V)$	—
Bellman-Ford	$O(VE)$	$O(VE)$	$O(V)$	—
Dijkstra	$O(E \lg V)$	$O(E \lg V)$	$O(V)$	—
Floyd-Warshall	$O(V^3)$	$O(V^3)$	$O(V^2)$	—
Johnson's APSP	$O(VE + V^2 \lg V)$	$O(VE + V^2 \lg V)$	$O(V^2)$	—
Ford-Fulkerson	$O(E \cdot f^*)$	f^*	$O(E \cdot f^*)$	$O(E \cdot f^*)$
Edmonds-Karp	$O(VE^2)$	$O(VE^2)$	$O(V)$	—
Hopcroft-Karp	$O(E\sqrt{V})$	$O(E\sqrt{V})$	$O(V)$	—
Gale-Shapley	$O(n^2)$	$O(n^2)$	$O(n)$	—
Hungarian	$O(n^3)$	$O(n^3)$	$O(n^2)$	—
FFT	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n)$	—
Strassen MM	$O(n^{\{2.8074\}})$	$O(n^{\{2.8074\}})$	$O(n^2)$	—

Additional Algorithm Pseudocode Quick Reference

QUICKSORT

```

if p < r:
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q-1)
    QUICKSORT(A, q+1, r)

```

PARTITION (Lomuto)

```

x = A[r]; i = p-1
for j = p to r-1:
    if A[j] ≤ x: i++; swap A[i], A[j]
swap A[i+1], A[r]
return i+1

```

RANDOMIZED-PARTITION

```

i = RANDOM(p, r)
swap A[r], A[i]
return PARTITION(A, p, r)

```

HEAPSORT

```

BUILD-MAX-HEAP(A, n)
for i = n down to 2:
    swap A[1], A[i]
    heap-size--
    MAX-HEAPIFY(A, 1)

```

MAX-HEAPIFY

```

l = LEFT(i); r = RIGHT(i); largest = i
if l ≤ heap-size and A[l] > A[largest]: largest = l
if r ≤ heap-size and A[r] > A[largest]: largest = r
if largest ≠ i: swap A[i], A[largest]; MAX-HEAPIFY(A, largest)

```

BF S(G, s)

```

for each u ≠ s: color=WHITE, d=∞, π=NIL
s.d=0, ENQUEUE(Q, s)
while Q not empty:
    u = DEQUEUE(Q)
    for each v in Adj[u]:
        if v.color == WHITE: v.color=GRAY; v.d=u.d+1; v.π=u;
    ENQUEUE(Q, v)
    u.color = BLACK

```

DFS(G)

```

for each u: color=WHITE, π=NIL; time=0
for each u: if color==WHITE: DFS-VISIT(u)
DFS-VISIT(u):
    time++; u.d=time; u.color=GRAY
    for each v in Adj[u]: if v.color==WHITE: v.π=u; DFS-VISIT(v)
    u.color=BLACK; time++; u.f=time

```

DIJKSTRA(G, w, s)

```

INIT-SINGLE-SOURCE(G, s); S = ∅; Q = V
while Q not empty:
    u = EXTRACT-MIN(Q); S = S ∪ {u}
    for each v in Adj[u]: RELAX(u, v, w)

```

BELLMAN-FORD(G, w, s)

```

INIT-SINGLE-SOURCE(G, s)
for i = 1 to |V|-1:
    for each (u,v) in E: RELAX(u,v,w)
for each (u,v) in E:
    if v.d > u.d + w(u,v): return FALSE
return TRUE

```

FLOYD-WARSHALL(W)

```

D = W; k=1 to n: for i=1 to n: for j=1 to n:
    D[i][j] = min(D[i][j], D[i][k] + D[k][j])
return D

```

FORD-FULKERSON(G, s, t)

```

for each (u,v) in E: f(u,v) = 0
while ∅ path p in G_f:
    c_f(p) = min_{edge in p} c_f(edge)
    for each (u,v) in p:
        f(u,v) += c_f(p); f(v,u) -= c_f(p)
return f

```

KMP-MATCHER

```

π = COMPUTE-PREFIX-FUNCTION(P, m); q = 0
for i = 1 to n:
    while q > 0 and P[q+1] ≠ T[i]: q = π[q]
    if P[q+1] == T[i]: q++
    if q == m: print "match at", i-m; q = π[q]

```

Chapter-End Problems Summary

Problem 2-4 (Inversions): Let $A[1:n]$. Inversion = (i,j) with $i < j$ and $A[i] > A[j]$. (a) List inversions of $[2, 3, 8, 6, 1]$. (b) Array with most inversions? Reversed: $n(n-1)/2$. (c) Running time of insertion sort = $\Theta(n + \text{inversions})$. (d) Lower bound on comparison sort: at least inversions comparisons. (e) Algorithm counting inversions in $\Theta(n \lg n)$: modify merge sort.

Problem 4-1 (Recurrences): Solve: (a) $T(n) = 2T(n/2) + n^3 \rightarrow \text{Case 3} \rightarrow \Theta(n^3)$. (b) $T(n) = T(9n/10) + n \rightarrow \text{Case 3} \rightarrow \Theta(n)$. (c) $T(n) = 16T(n/4) + n^2 \rightarrow \text{Case 2} \rightarrow \Theta(n^2 \lg n)$. (d) $T(n) = 7T(n/3) + n^2 \rightarrow \text{Case 3} \rightarrow \Theta(n^2)$. (e) $T(n) = T(\sqrt{n}) + 1 \rightarrow \text{change variable } m = \lg n \rightarrow \Theta(\lg \lg n)$.

Problem 15-4 (Scheduling): Given n jobs with deadlines d_i and profits p_i , schedule to maximize profit. Greedy: sort by profit descending; schedule each at latest available slot before deadline. Uses disjoint-set to find next available slot. Running time $O(n \alpha(n))$.

Problem 16-3 (Amortized weight-balanced trees): Each node's weight = size+1. Maintain that no node's weight exceeds $2 \times$ any descendant's weight. When violated, rebuild subtree. Amortized $O(1)$ per insertion via potential = $\sum \log(\text{weight})$.

Problem 34-1 (Independent Set): Given graph G and integer k , does G have independent set of size $\geq k$? NP-complete. Reduction from CLIQUE: G has k -clique iff complement has independent set of size k .

Key Theorems to Memorize

#	Theorem	Statement	Where
1	$\Theta = O \cap \Omega$	$f = \Theta(g)$ iff $f = O(g)$ and $f = \Omega(g)$	Ch 3
2	Comparison sort lower bound	Any comparison sort requires $\Omega(n \lg n)$ comparisons	Ch 8
3	Master Theorem	$T(n) = aT(n/b) + f(n) \rightarrow$ 3 cases	Ch 4
4	Red-black height	RB tree with n internal nodes has $h \leq 2 \lg(n+1)$	Ch 13
5	Max-flow min-cut	Max flow = min cut; 3 equivalent conditions	Ch 24
6	Cut property	Light edge crossing cut respected by A is safe for MST	Ch 21
7	Triangle inequality	$\delta(s, v) \leq \delta(s, u) + w(u, v)$	Ch 22
8	Dijkstra correctness	For nonnegative weights, Dijkstra finds correct SP	Ch 22
9	B-tree height	$h \leq \log_t((n+1)/2)$	Ch 18
10	Cook-Levin	Circuit-SAT is NP-complete	Ch 34
11	Kuhn-Munkres	Feasible labeling + perfect matching in $G_\ell \rightarrow$ optimal assignment	Ch 25
12	Greedy scheduling bound	$T_P \leq T_1/P + T_\infty$	Ch 26
13	Gale-Shapley	Deferred acceptance produces stable matching; male-optimal	Ch 25
14	Union-find	m ops with union by rank + path compression: $O(m \alpha(n))$	Ch 19

Common Pitfalls & How to Avoid Them

1. **Confusing O and Θ :** O is upper bound, Θ is tight bound. An algorithm can be $O(n^2)$ and $\Theta(n)$ simultaneously.
2. **Applying Master Theorem when $f(n)$ falls between cases:** The gap between Case 1 and Case 2 ($f(n) = \Theta(n^{\log_b a} / \lg n)$) is not covered. Use Akra-Bazzi or recursion tree.

3. **Forgetting regularity condition for Master Theorem Case 3:** Must check $af(n/b) \leq cf(n)$ for $c < 1$.
4. **Confusing DP and greedy:** Every greedy problem has optimal substructure, but not vice versa. Check if greedy-choice property holds.
5. **Dijkstra on negative edges:** Dijkstra can fail even if no negative cycles – once a vertex's distance is finalized, it cannot be updated.
6. **BFS for weighted graphs:** BFS finds shortest paths only in unweighted graphs (or uniform weight). For weighted, use Dijkstra/Bellman-Ford.
7. **Hash table load factor:** Open addressing requires $\alpha \leq 1$. Chaining can handle $\alpha > 1$ but performance degrades.
8. **Counting sort stability:** Must traverse input from n down to 1 for stability (otherwise relative order of equal keys is reversed).
9. **Forgetting the sentinel in red-black trees:** Leaves are NIL sentinels (black), not null pointers. This simplifies edge cases.
10. **Potential function validity:** Must ensure $\Phi(D_i) \geq \Phi(D_0)$ for all i (usually $\Phi(D_0)=0$ and $\Phi(D_i) \geq 0$), otherwise amortized bounds don't follow.

Final Exam Checklist

☐

Can I state the Master Theorem and apply it to any recurrence?

☐

Can I trace insertion sort, merge sort, heapsort, quicksort on a small array?

☐

Do I know the formal definitions of O , Θ , Ω , o , ω ?

☐

Can I prove the comparison sort lower bound ($\Omega(n \lg n)$)?

☐

Can I solve recurrences using substitution, recursion tree, and master theorem?

☐

Do I know when to use DP vs greedy? Can I design a DP algorithm (4 steps)?

☐

Can I trace Bellman-Ford, Dijkstra, Floyd-Warshall on small graphs?

☐

Can I find augmenting paths in a flow network and identify the min cut?

☐

Do I know the 5 red-black tree properties and the 4 deletion cases?

☐

Can I show that a problem is NP-complete (membership + reduction)?

☐

Can I design a 2-approximation for vertex cover or TSP?



Do I understand amortized analysis (aggregate, accounting, potential)?



Can I compute work, span, and parallelism of a fork-join program?



Can I trace KMP, Rabin-Karp, or naive string matching?



Do I understand the RSA public-key cryptosystem (encryption/decryption)?



Can I compute GCD using Euclid's algorithm and extended Euclid?

Summary of Recurrence Solutions (Quick Reference)

Recurrence	Solution	Method	Example
$T(n) = T(n-1) + n$	$\theta(n^2)$	Iteration/ substitution	Selection sort, insertion sort (worst)
$T(n) = T(n-1) + 1$	$\theta(n)$	Iteration	Linear search
$T(n) = T(n/2) + 1$	$\theta(\lg n)$	Master Case 2	Binary search
$T(n) = 2T(n/2) + n$	$\theta(n \lg n)$	Master Case 2	Merge sort, quicksort best/ avg
$T(n) = 2T(n/2) + 1$	$\theta(n)$	Master Case 1	Tree traversal
$T(n) = 2T(n/2) + n \lg n$	$\theta(n \lg^2 n)$	Master Case 2(k=1)	Sorting by comparison of pairs
$T(n) = 2T(n/2) + n/\lg n$	$\theta(n \lg \lg n)$	Recursion tree	Gap case
$T(n) = T(n/5) + T(7n/10) + n$	$\theta(n)$	Substitution	SELECT (deterministic)
$T(n) = 7T(n/2) + n^2$	$\theta(n^{\{\lg 7\}}) = \theta(n^{\{2.81\}})$	Master Case 1	Strassen
$T(n) = 8T(n/2) + n^2$	$\theta(n^3)$	Master Case 1	Naive recursive MM
$T(n) = 9T(n/3) + n$	$\theta(n^2)$	Master Case 1	Example
$T(n) = 3T(n/4) + n \lg n$	$\theta(n \lg n)$	Master Case 3	Example
$T(n) = T(2n/3) + 1$	$\theta(\lg n)$	Master Case 2	Example (MAX- HEAPIFY bound)
$T(n) = 2T(n/2) + n^2$	$\theta(n^2)$	Master Case 3	Example
$T(n) = 2T(n/2) + c$	$\theta(n)$		Example

Recurrence	Solution	Method	Example
		Master Case	
		1	

Complexity Class Reference

- **P**: problems solvable in polynomial time (e.g., sorting, shortest path, MST, max flow)
- **NP**: problems verifiable in polynomial time (e.g., SAT, CLIQUE, TSP decision, SUBSET-SUM)
- **co-NP**: complement of NP (e.g., TAUTOLOGY – is formula always true?)
- **PSPACE**: problems solvable with polynomial space (e.g., quantified Boolean formulas)
- **NPC** (NP-complete): $NP \cap NP\text{-hard}$ (e.g., SAT, 3-CNF-SAT, CLIQUE, VERTEX-COVER, HAM-CYCLE, TSP, SUBSET-SUM)
- **NP-hard**: at least as hard as NP (e.g., optimization TSP, halting problem)
- **EXPTIME**: exponential time (properly contains P if $P \neq NP$)

Known relationships: $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$. $P \neq EXPTIME$ (time hierarchy theorem). $P = NP?$ – most believe no.

Probability & Statistics in Algorithms

Birthday Paradox: With n items sampled uniformly from d possibilities: - Expected number of samples before first collision: $\approx \sqrt{\pi d/2}$ - Probability of collision with k samples: $\approx 1 - e^{-k(k-1)/(2d)}$ - For $d=365$, $k=23$ gives $p > 1/2$

Coupon Collector: Expected time to collect all n coupons = $n \cdot H_n$
 $H_n = n(\ln n + \gamma) + 1/2 \approx n \ln n + O(n)$ - Stage i (i coupons collected): probability of new coupon = $(n-i)/n$ - Expected trials in stage i = $n/(n-i)$ - Total: $\sum_{i=0}^{n-1} n/(n-i) = n \cdot H_n$

Balls and Bins: Tossing n balls into n bins uniformly at random:
 - Expected max load: $\Theta(\ln n / \ln \ln n)$ - Expected number of empty bins: $n(1-1/n)^n \approx n/e$ - Expected number of bins with exactly k balls: $n \cdot C(n, k) (1/n)^k (1-1/n)^{n-k} \approx n \cdot e^{-1}/k!$

Randomized Quicksort Analysis: $\Pr[z_i \text{ compared with } z_j] = 2/(j-i+1)$ - Expected comparisons: $\sum_{i=1}^{n-1} \sum_{j=i+1}^n 2/(j-i+1) = \sum_{k=1}^{n-1} \sum_{i=1}^{n-k} 2/(k+1) < 2n \ln n = O(n \lg n)$

Universal Hashing: Family $H = \{h : U \rightarrow \{0, \dots, m-1\}\}$ is universal if for any distinct k_1, k_2 : $\Pr_{h \in H}[h(k_1) = h(k_2)] \leq 1/m$ - Number-theoretic construction: $h_{ab}(k) = ((ak+b) \bmod p) \bmod m$ for prime $p > |U|$, $a \in [1, p-1]$, $b \in [0, p-1]$ - Expected length of chain: $\alpha = n/m$

Important Mathematical Facts

- **Stirling's approximation:** $n! = \sqrt{2\pi n}(n/e)^n(1+O(1/n))$
- **Harmonic numbers:** $H_n = \sum_{i=1}^n 1/i = \ln n + \gamma + 1/(2n) - 1/(12n^2) + O(1/n^3)$, $\gamma \approx 0.57721$ (Euler's constant)
- **Golden ratio:** $\phi = (1+\sqrt{5})/2 \approx 1.618$, $\bar{\phi} = (1-\sqrt{5})/2 \approx -0.618$. $\phi^2 = \phi+1$, $\bar{\phi}^2 = \bar{\phi}+1$
- **Summations:** $\sum_{i=0}^n ar^i = a(1-r^{n+1})/(1-r)$ for $r \neq 1$; $\sum_{i=0}^{\infty} ar^i = a/(1-r)$ for $|r| < 1$
- **Arithmetic series:** $\sum_{i=1}^n i = n(n+1)/2$; $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$
- **Useful inequalities:** $1+x \leq e^x$ for all real x ; $\lg n$ is $o(n^\epsilon)$ for any $\epsilon > 0$
- **Integral bounds:** $\int_0^n x^k dx \leq \sum_{i=1}^n i^k \leq \int_{-1}^{n+1} x^k dx$

Table of Notations

Symbol	Meaning	First Seen
$\Theta(g(n))$	Asymptotic tight bound	Ch 3
$O(g(n))$	Asymptotic upper bound	Ch 3
$\Omega(g(n))$	Asymptotic lower bound	Ch 3
$o(g(n))$	Non-tight upper bound	Ch 3
$\omega(g(n))$	Non-tight lower bound	Ch 3
$\lfloor x \rfloor$	Floor (greatest integer $\leq x$)	Ch 3
$\lceil x \rceil$	Ceiling (least integer $\geq x$)	Ch 3
$a \bmod n$	Remainder when a divided by n	Ch 3
$\lg n$	$\log_2 n$	Ch 2
$\ln n$	$\log_e n$	Ch 3
$\lg^k n$	$(\lg n)^k$	Ch 3
$\lg \lg n$	$\lg(\lg n)$	Ch 3
$\lg^* n$	Iterated logarithm	Ch 3
ϕ	Golden ratio $(1+\sqrt{5})/2$	Ch 3
$n!$	n factorial	Ch 3
H_n	Harmonic number $\sum 1/i$	Ch 5
$\alpha(n)$	Inverse Ackermann	Ch 19
T_1	Work (serial time)	Ch 26
T_∞	Span (critical path)	Ch 26
$\delta(u,v)$	Shortest path weight	Ch 22
$c_f(u,v)$	Residual capacity	Ch 24
G_f	Residual network	Ch 24
$\pi[q]$	Prefix function (KMP)	Ch 32
ω_n	Principal n th root of unity	Ch 30

End of study guide. Total chapters covered: 35 (Ch 1-35).
 Universal primitives: all 17 categories checked for every chapter. Target length: ~3000 lines. Every named entity,

algorithm, formula, comparison, edge case, proof pattern,
and end-of-chapter item extracted from all 35 chapters of
CLRS 4th edition.